

Le BIG DATA

Dr. Abdoulaye SIDIBE

Plan

- 1. Introduction au Big Data**
- 2. Définition du Big Data**
- 3. Les Caractéristiques du Big Data**
- 4. Les Technologies de Stockage des Big Data**
- 5. Les Technologies de Traitement et d'Analyse des Big Data**
- 6. Les Domaines d'Application du Big Data**
- 7. Les différents types d'analyse du Big Data**
- 8. Les Défis du Big Data**

1.Introduction au Big Data

Le terme Big data a été utilisé pour la première fois pour désigner l'augmentation des volumes de données **au milieu des années 90**.

En 2001, Doug Laney, analyste au cabinet de conseil Meta Group Inc., a élargi la définition du big data. Cette expansion décrit l'augmentation par:

- **Volume** de données stockées et utilisées par les entreprises ;
- **Variété** de données générées par les entreprises ;
- **Vélocité**, ou vitesse, à laquelle ces données ont été créées et mises à jour

En 2005, ces trois facteurs sont devenus connus sous le nom de 3V du Big Data (Volume, Vélocité et Variété).

En 2006, un autre développement important dans l'histoire du Big Data a été le lancement d'un **projet open source Apache Hadoop pour la gestion et le traitement des Big Data**.

En 2011, l'analyse des Big Data a commencé à s'implanter fermement dans les entreprises et le public, avec Hadoop et diverses technologies de Big data connexes.

2. Qu'est-ce que le Big Data ?

Les Big Data décrivent des ensembles de données si volumineux et complexes qu'ils deviennent difficiles à traiter à l'aide de matériel et de logiciels de traitement de données classiques.

Les perspectives du traitement des Big Data sont énormes notamment pour l'analyse d'opinions ou de tendances industrielles, la génomique, l'épidémiologie ou la lutte contre la criminalité.

2.1. L'enjeu du Big Data

L'enjeu du Big Data est **la création de la valeur**.

Il faut exploiter le Big Data pour :

- ❖ Plus de productivité
- ❖ Mieux compétir
- ❖ Autres choses (Nouveau marché, innovation, Partenariat, etc.)

Utilisations et exemples de l'analyse des Big data

Ci-dessous nous avons quelques exemples de la façon dont l'analyse des Big data peut être utilisée pour aider les organisations

- **Acquisition et fidélisation de la clientèle:** Les données sur les consommateurs peuvent aider les efforts de marketing des entreprises, qui peuvent agir sur les tendances pour augmenter la satisfaction des clients. Par exemple, les moteurs de personnalisation pour Amazon, Netflix et Spotify peuvent améliorer l'expérience client et fidéliser la clientèle.
- **Annonces ciblées:** Les données de personnalisation provenant de sources telles que les achats passés, les modèles d'interaction et les historiques de consultation des pages de produits peuvent aider à générer des campagnes publicitaires ciblées et attrayantes pour les utilisateurs au niveau individuel et à plus grande échelle.

Utilisations et exemples de l'analyse des Big data

- **Développement de produits:** L'analyse des Big data peut fournir des informations pour informer sur la viabilité du produit, les décisions de développement, la mesure des progrès et orienter les améliorations dans la direction de ce qui convient aux clients d'une entreprise.
- **Optimisation des prix:** Les détaillants peuvent opter pour des modèles de tarification qui utilisent et modélisent des données provenant de diverses sources de données afin de maximiser les revenus.
- **Gestion des risques:** L'analyse des Big Data peut identifier de nouveaux risques à partir des modèles de données pour des stratégies de gestion des risques efficaces.
- **Prise de décision améliorée:** Les informations que les utilisateurs métier extraient des données pertinentes peuvent aider les organisations à prendre des décisions plus rapides et plus efficaces.

Les Avantages de l'Analyse des Big Data

Les avantages de l'utilisation de l'analyse de données volumineuses incluent :

- **Analyser rapidement de grandes quantités de données** provenant de différentes sources, dans de nombreux formats et types différents.
- **Prendre rapidement des décisions mieux informées pour une stratégie efficace**, ce qui peut bénéficier et améliorer la chaîne d'approvisionnement, les opérations et d'autres domaines de prise de décision stratégique.
- **Économies de coûts**, qui peuvent résulter de nouvelles efficacités et optimisations des processus métier.
- **Une meilleure compréhension des besoins**, du comportement et des sentiments des clients, ce qui peut conduire à de meilleures informations marketing et fournir des informations pour le développement de produits.
- **Des stratégies de gestion des risques améliorées et mieux informées** qui s'appuient sur de grands échantillons de données.

3. Les 5V du Big Data

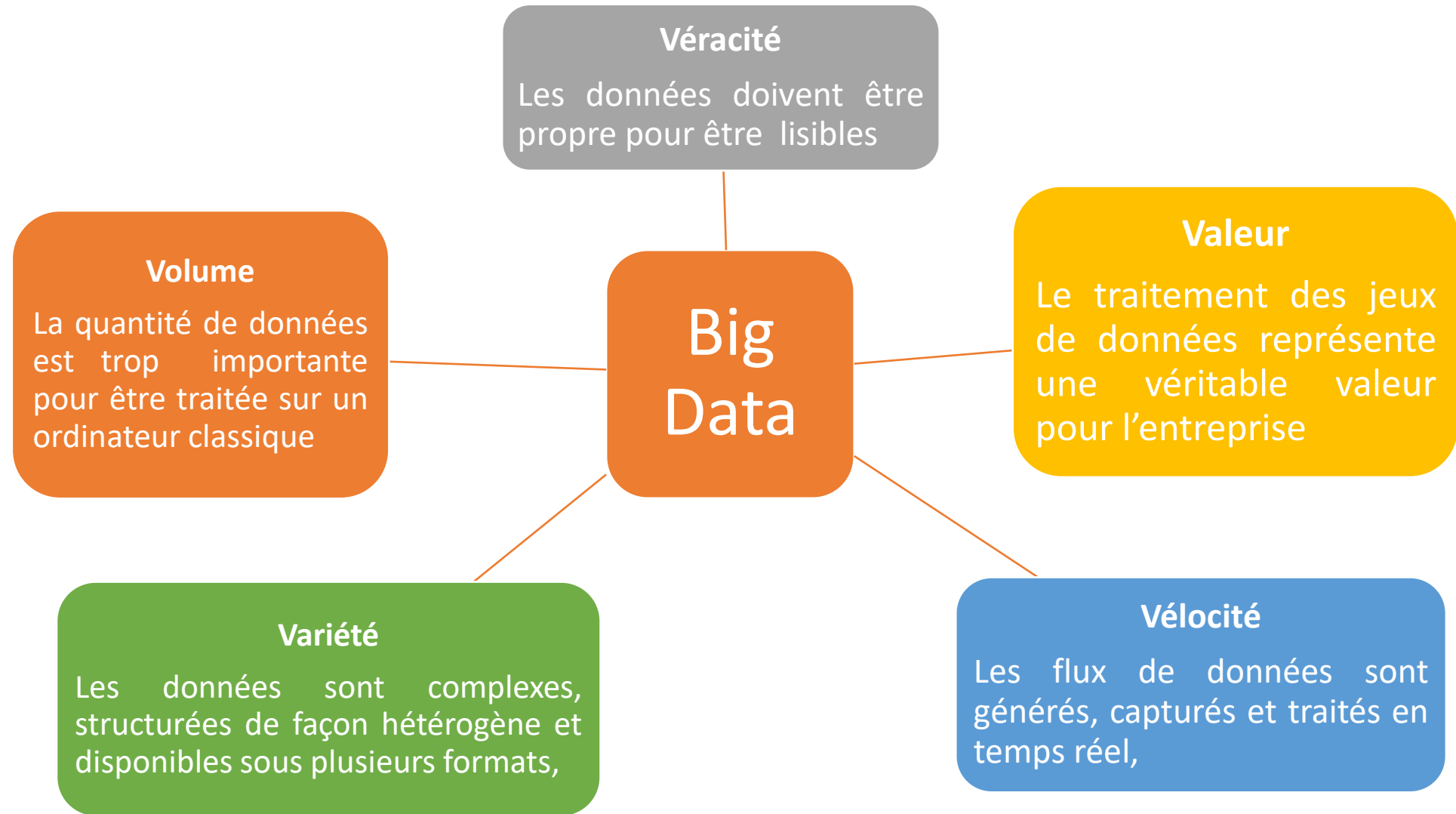


Figure 1: Les 5V du Big Data

3. Les 5V du Big Data

1. Volume de données

2. Variété des données

3. Vitesse des données

4. Véracité des données

5. Valeur

3. Les 5V du Big Data

Volume de données

Le Big Data implique une énorme quantité de données. Ces données sont générées par des machines, des réseaux et l'interaction humaine sur des systèmes tels que les réseaux sociaux.

Le Big Data traite des zettaoctets de données. Pour référence, les tailles de données en termes d'octets sont indiquées ci-dessous.

- 10^3 octets - 1 Ko (kilo-octets)
- 10^6 octets - 1 Mo (mégaoctets)
- 10^9 octets - 1 Go (gigaoctets)
- 10^{12} octets - 1 To (téraoctets)
- 10^{15} octets - 1 Po (pétaoctets)
- 10^{18} octets - 1 EB (exaoctets)
- 10^{21} octets - 1 ZB (zettaoctets)

Volume de données

Le big data est une forme de données dont le volume est si important qu'il ne peut pas être stocké sur une seule machine. Des outils et des Framework spécialisés sont donc nécessaires pour stocker, traiter et analyser ces données.

Par exemple,

- **les applications de médias sociaux traitent des milliards de messages chaque jour,**
- **les systèmes industriels et énergétiques peuvent générer des téraoctets de données de capteurs chaque jour,**
- **les applications d'agrégation de taxis peuvent traiter des millions de transactions en une journée,** etc.

Volume de données (suite)

Les volumes de données générés par les systèmes informatiques, industriels, de santé, Internet des objets et autres systèmes modernes **augmentent de manière exponentielle**, en raison de la baisse des coûts des architectures de stockage et de traitement des données et **de la nécessité d'extraire des informations précieuses à partir des données pour améliorer les processus commerciaux, l'efficacité et le service aux consommateurs.**

Bien qu'il n'existe pas de seuil fixe pour le volume de données à considérer comme du big data, le terme big data est généralement utilisé pour des données à grande échelle difficiles à **stocker**, à **gérer** et à **traiter** à l'aide de bases de données et d'architectures de traitement de données traditionnelles.

3. Les 5V du Big Data

Variété des données

La **variété** fait référence aux sources ou aux formes des données. Le Big Data se présente sous différentes formes, notamment des **données textuelles**, **des images**, **des fichiers audio**, **des vidéos** et **des données de capteurs**. Les systèmes Big Data doivent être suffisamment flexibles pour gérer une telle variété de données.

La **variété** fait référence également aux sources des données. Traditionnellement, les données provenaient de sources telles que **des feuilles de calcul** et **des bases de données**. Aujourd'hui, les données sont générées sous forme **d'e-mails**, **de photos**, **de vidéos**, **d'appareils de surveillance**, **de PDF**, **d'audio**, etc. Cette variété de données non structurées crée des problèmes de stockage, d'exploration et d'analyse des données.

3. Les 5V du Big Data

Vitesse des données

La vélocité des données fait référence à la vitesse à laquelle les données sont générées. Les données générées par certaines sources peuvent atteindre des vitesses très élevées, par exemple les données des réseaux sociaux ou les données des capteurs. La vélocité est une autre caractéristique importante du big data et la principale raison de la croissance exponentielle des données. **La grande vélocité des données se traduit par un volume de données accumulées très important, en peu de temps.** Certaines applications peuvent avoir des délais stricts pour l'analyse des données (comme le trading ou la détection de fraude en ligne) et **les données doivent être analysées en temps réel.** **Des outils spécialisés sont nécessaires pour ingérer ces données à grande vitesse dans l'infrastructure du big data et les analyser en temps réel.**

3. Les 5V du Big Data

Vitesse des données (suite)

Pour comprendre la vitesse des données, nous pouvons considérer le cas d'une voiture sans conducteur. Pour fabriquer une voiture sans conducteur, nous devons déployer des milliers de capteurs et la fréquence à laquelle ces capteurs génèrent des données est en millisecondes. Nous devons traiter ces données à grande vitesse en temps réel et générer la réponse afin de faire fonctionner l'ensemble.

Cet exemple indique clairement l'importance de traiter des données à grande vitesse.

3. Les 5V du Big Data

Véracité des données

La véracité fait référence à la précision ou la fiabilité des données. Pour extraire de la valeur des données, celles-ci doivent être nettoyées pour éliminer le bruit et les anomalies dans les données. Les applications basées sur les données ne peuvent tirer parti du Big Data que si les données sont significatives et exactes. Par conséquent, le nettoyage des données est important pour pouvoir filtrer les données incorrectes et défectueuses.

Pour juger de la véracité des données, nous posons une question simple : « **Les données stockées et exploitées sont-elles pertinentes pour le problème analysé ?** » La **véracité** de l'analyse des données est le plus grand défi lorsqu'on la compare au **volume** et à la **vélocité**.

3. Les 5V du Big Data

Valeur

La valeur des données fait référence à l'utilité des données pour l'objectif visé. L'objectif final de tout système d'analyse de big data est d'extraire de la valeur des données. La valeur des données est également liée à la véracité ou à l'exactitude des données. Pour certaines applications, la valeur dépend également de la rapidité avec laquelle nous sommes capables de traiter les données.

Différents Types de Données

1. Données structurées

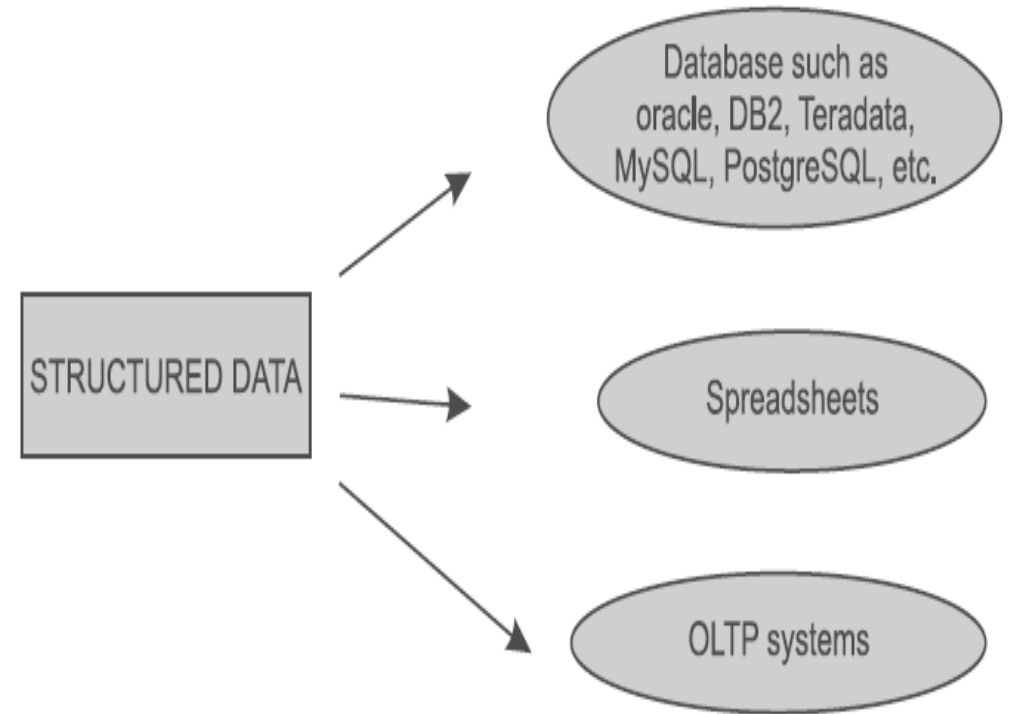
2. Données Semi-structurées

3. Données non structurées

Données structurées

Les données qui ont une structure fixe et peuvent être organisées en lignes et en colonnes sont appelées données structurées. Par exemple, une base de données stockée dans SQL, des feuilles Excel, OLTP (Online Transaction Processing) et autres, comme illustré dans la figure 15.7, sont toutes structurées.

Elles ont une clé relationnelle et peuvent être facilement mappées dans des champs prédéfinis. C'est le moyen le plus simple de gérer les informations. Mais les données structurées ne représentent que 5 à 10 % de toutes les données présentes dans le monde.



Données semi-structurées

Les données **semi-structurées** ne résident pas dans une base de données relationnelle, mais possèdent certaines propriétés organisationnelles qui facilitent leur analyse. Les fichiers CSV, les documents XML et JSON sont tous des exemples de documents semi-structurés.

Tout comme les données structurées, les données semi-structurées ne représentent qu'environ 5 à 10 % de toutes les données mondiales.

Données non structurées

Les données non structurées représentent environ 80 % des données. Elles comprennent souvent du texte et du contenu multimédia.

Par exemple, les messages électroniques, les documents de traitement de texte, les vidéos, les photos, les fichiers audio, les présentations, les pages Web et de nombreux autres types de documents commerciaux. Les données non structurées sont partout. En fait, la plupart des individus et des organisations mènent leur vie autour de données non structurées.

Données non structurées

Voici d'autres exemples de données non structurées :

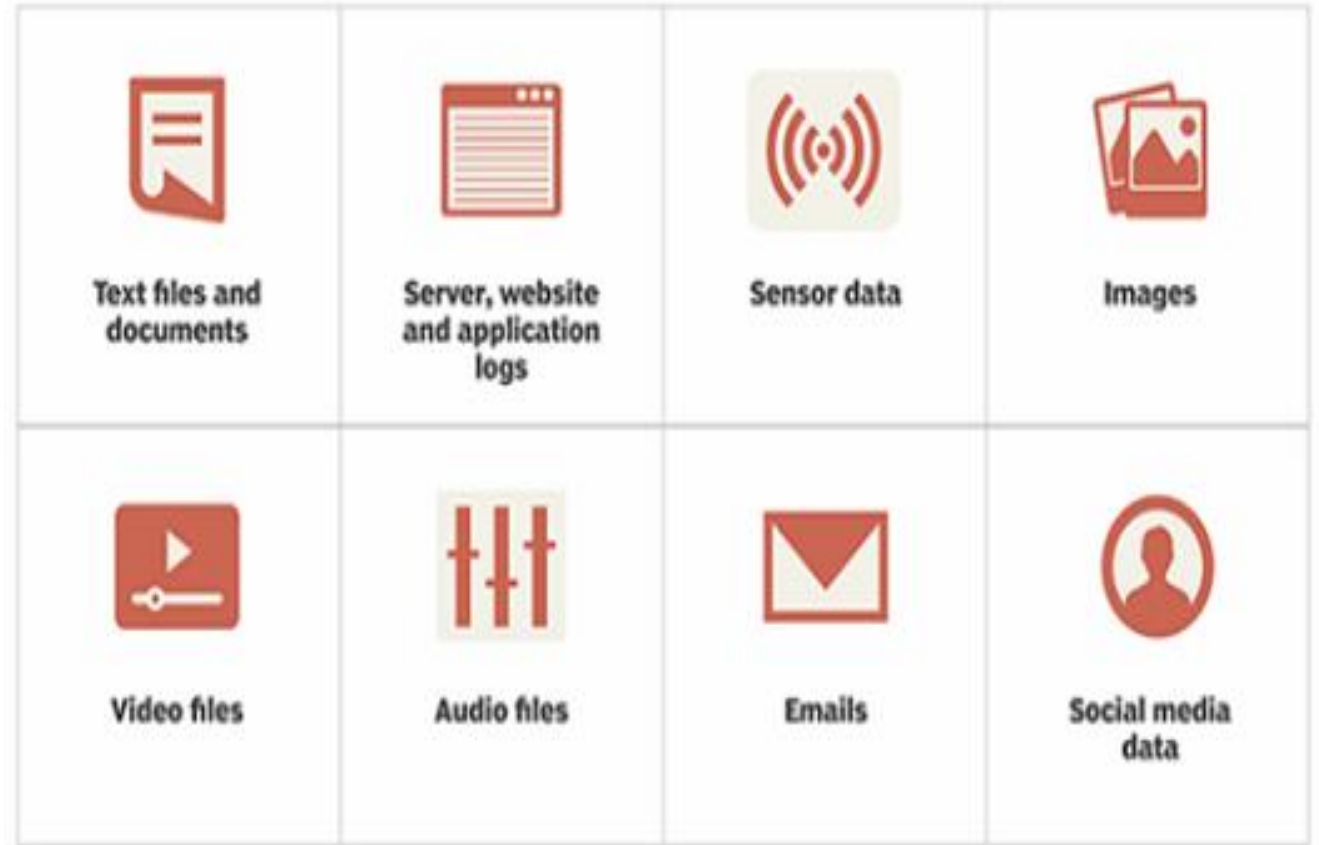
- **Images satellites** : cela inclut les données météorologiques ou les données que le gouvernement capture via la surveillance par satellite. Effectuez une recherche sur Google Earth pour voir les données d'images satellite.
- **Données scientifiques** : cela inclut les images sismiques, les données atmosphériques et la physique des hautes énergies (données provenant des accélérateurs de particules).
- **Photographies et vidéos** : cela inclut les vidéos de sécurité, de surveillance et de circulation.
- **Données radar ou sonar** : cela inclut les profils sismiques des véhicules, météorologiques et océanographiques.

Données non structurées

- **Texte interne à toute entreprise** : le texte contenu dans les documents officiels, les journaux, les résultats d'enquête et les communications par courrier électronique. Les informations d'entreprise représentent en fait un pourcentage important des informations textuelles dans le monde d'aujourd'hui.
- **Données des médias sociaux** : ces données sont générées à partir des plateformes de médias sociaux telles que YouTube, Facebook, Twitter, LinkedIn et Flickr.
- **Données mobiles** : cela inclut des données telles que des messages texte et des informations de localisation.
- **Contenu du site Web** : cela provient de tout site Web proposant du contenu non structuré, comme YouTube, Vimeo ou Wikipedia.

Quelque Exemples de Données Non Structurées

- ❖ Journaux du serveur, du site Web et des applications
- ❖ Données du capteur
- ❖ Images
- ❖ Fichiers
- ❖ Vidéo
- ❖ Fichiers audio
- ❖ E-mail
- ❖ Des médias sociaux



Répartition en pourcentage des différents types de données

La distribution de ces trois types de données est illustrée dans la figure suivante :

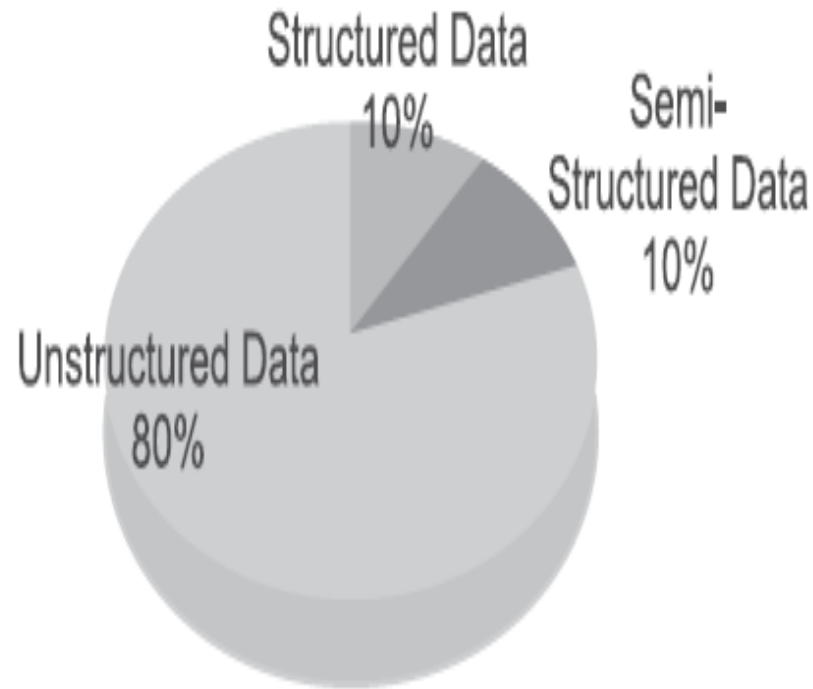


Figure : Répartition en pourcentage des différents types de données

Étant donné que 80 % des données ne sont pas structurées, il existe une demande énorme d'experts en bases de données capables de traiter et d'analyser ces données. Il est important de noter que les bases de données traditionnelles ne peuvent traiter que les données structurées, qui ne représentent que 10 % du total des données générées. Ainsi, l'essor d'Internet a généré de nouveaux défis et de nouvelles opportunités en termes de technologies et de compétences.

4. Les Technologies Big Data

- ❖ Technologies de **Stockage** Big Data
- ❖ Technologies de **Traitement** et d'**Analyse** Big Data

5. Technologies de Stockage Big Data

Le Stockage de Big Data est assuré par les deux techniques suivantes :

- 1. Systèmes de Fichiers Distribués Hadoop (HDFS)**
- 2. NoSQL**

TERMINOLOGIE

Système de fichiers - est un logiciel qui contrôle la façon dont les données sont stockées et récupérées à partir des disques et autres périphériques de stockage. Il gère les disques de stockage avec différents modèles d'accès aux données tels que basés sur des fichiers (NTFS, ext4), des objets (swift, S3), des blocs (cinder, base de données).

Système distribué - Un groupe d'ordinateurs hétérogènes/homogènes en réseau qui fonctionnent ensemble comme une seule unité pour accomplir une tâche est appelé un **système distribué**. En termes simples, un groupe d'ordinateurs fournit une vue d'ordinateur unique. Exemple : Quatre ordinateurs chacun avec double cœur, 4 Go de mémoire et 1 To de stockage dans un cluster peuvent être considérés comme un **système distribué avec 8 cœurs, 16 Go de mémoire et 4 To de stockage**.

TERMINOLOGIES

Calcul distribué (distributed computing): Le calcul distribué est un domaine de l'informatique **qui consiste à répartir des tâches de calcul sur plusieurs ordinateurs ou processeurs interconnectés, souvent via un réseau.** Cela permet de résoudre des problèmes complexes plus rapidement en exploitant la puissance de calcul combinée de plusieurs machines.

Stockage distribué - Une collection de périphériques de stockage de différents ordinateurs dans un cluster offrant une vue sur un seul disque est appelée stockage distribué.

TERMINOLOGIES

Un cluster de calcul est un groupe d'ordinateurs interconnectés (souvent appelés nœuds) qui fonctionnent ensemble comme un système unique pour effectuer des tâches de calcul à grande échelle. **Ces clusters sont conçus pour offrir des performances, une évolutivité et une fiabilité supérieures à celles qu'un seul ordinateur pourrait atteindre seul.** Les clusters de calcul sont largement utilisés dans **la recherche scientifique, l'analyse de données, l'apprentissage automatique et d'autres domaines** qui nécessitent **une puissance de calcul importante.**

TERMINOLOGIES

Système de fichiers distribués (DFS) – Chaque logiciel de système de fichiers fournit un espace de noms (chemin) aux fichiers dans une structure arborescente. Le système de fichiers local ne donne un espace de noms que pour les disques qui y sont attachés. **DFS fournit un espace de noms global (unifié) unique sur le stockage distribué.** **Exemple** : si un utilisateur exécute une commande sur le node5 pour afficher la valeur de « x » stockée dans node10, le résultat est affiché sur le node5 lui-même. **L'utilisateur ne sait pas où « x » est stocké.**

TERMINOLOGIES

- La gestion **des ressources** sur un cluster de machines et la fourniture **d'une vue d'ordinateur unique** s'appelle un **système distribué**.
- La gestion **d'un ensemble de processus d'une application** sur un cluster de machines ou de processeurs est appelée **informatique distribuée**.
- La gestion **d'un ensemble de périphériques de stockage** sur un cluster de machines et la fourniture **d'une vue de disque unique** s'appelle le **stockage distribué**.
- Fournir **un espace de noms global unique et unifié** sur le stockage est appelé **Système de fichiers distribués**.

5.1 Systèmes de Fichiers Distribués Hadoop (HDFS)

Le terme système de fichiers fait référence à, **la façon dont les fichiers et les répertoires sont physiquement structurés sur un disque ou un autre support de stockage**. Le système de fichiers fait aussi référence à, **la structure et au contenu de certains supports de stockage**.

HDFS est un système de fichiers conçu pour stocker de fichiers très volumineux avec des modèles d'accès aux données en continu, s'exécutant sur des grappes de matériel de base. Il divise les fichiers volumineux en blocs de 64 Mo par défaut et les stocke dans un grand cluster de manière très efficace. L'architecture HDFS, il existe deux nœuds de base dans un cluster Hadoop

Un nœud de nom (name node): Il est nœud maître qui contrôle tous les nœuds de données et il contient des métadonnées. Il gère toutes les opérations sur les fichiers comme la lecture, l'écriture, etc.

Un nœud de données (data node): Les nœuds de données (data node) sont les nœuds esclaves présents dans le cluster Hadoop. Toutes les opérations de fichier effectuées sur ces nœuds et les données sont en fait stockées sur ces nœuds comme décidé par les nœuds de nom.

Le nœud de nom secondaire : Le nœud de nom secondaire est la sauvegarde du nœud de nom.

NAME NODE (NN)

NN est un service/démon centralisé qui agit comme un gestionnaire de stockage en cluster.

Ses principales responsabilités sont :

- maintenir les métadonnées des fichiers et des répertoires.
- contrôler et coordonner les DN pour les opérations du système de fichiers telles que la création, la lecture, l'écriture, etc.

Les clients communiquent avec NN afin d'effectuer les opérations quotidiennes du système de fichiers. À son tour, NN donne aux clients l'emplacement des DN dans le cluster (où le bloc de données est disponible) pour effectuer des opérations. NN procède à l'authentification s'il est configuré.

SECONDARY NAME NODE (SNN)

NN conserve les métadonnées de l'ensemble du cluster HDFS. Si NN est en panne, l'ensemble du cluster HDFS devient inaccessible en raison de l'indisponibilité des métadonnées. C'est ce qu'on appelle le point de défaillance unique (SPOF). Par conséquent, il est recommandé d'exécuter NN sur un serveur d'entreprise dans un environnement de production. Cependant, la défaillance des serveurs est inévitable. Par conséquent, une copie des métadonnées de NN est conservée dans une autre machine, appelée SNN. Une autre option consiste à utiliser NFS pour conserver les métadonnées.

DATA NODE (DN)

Le démon DN est responsable de la gestion des disques de stockage locaux. Il gère les opérations du système de fichiers telles que la création, la lecture, l'ouverture, la fermeture et la suppression de blocs. **Le DN ne sait rien des blocs de données. Il stocke chaque bloc sous forme de fichier dans le système de fichiers local.** Le DN ne stocke pas tous les blocs dans le même répertoire. Au lieu de cela, il utilise une heuristique pour déterminer le nombre optimal de blocs par répertoire et crée des sous-répertoires de manière appropriée. Les blocs sont chargés/supprimés dans les DN sur la base des instructions de NN, qui valide et traite les demandes des clients. **NN n'effectue aucune opération de lecture/écriture pour les clients. Les clients communiquent avec NN pour connaître les emplacements des blocs et sont redirigés vers les DN pour effectuer une opération de lecture/écriture.**

Systèmes de Fichiers Distribués Hadoop (HDFS)

HDFS est un système de fichiers conçu pour stocker de fichiers très volumineux avec des modèles d'accès aux données en continu, s'exécutant sur des clusters de matériel de base. HDFS est un système de fichiers logiciel implémenté en Java et il se trouve au-dessus du système de fichiers natif. **Le concept principal derrière HDFS est qu'il divise un fichier en blocs (généralement 128 Mo) au lieu de traiter un fichier dans son ensemble, et les stocke dans un cluster de manière très efficace.** Cela permet de nombreuses fonctionnalités telles que la **distribution**, la **réplication**, la **reprise après incident** et, le **traitement distribué des blocs à l'aide de plusieurs machines**. La taille des blocs peut être de 64 Mo, 128 Mo, 256 Mo ou 512 Mo, selon ce qui convient. **Pour un fichier de 1 Go avec des blocs de 128 Mo, il y aura 1024 Mo/128 Mo soit huit blocs. Si nous considérons un facteur de réplication de trois, cela fait 24 blocs.** **HDFS fournit un système de stockage distribué avec tolérance aux pannes et reprise après incident.**

5.1.2 Architecture HDFS

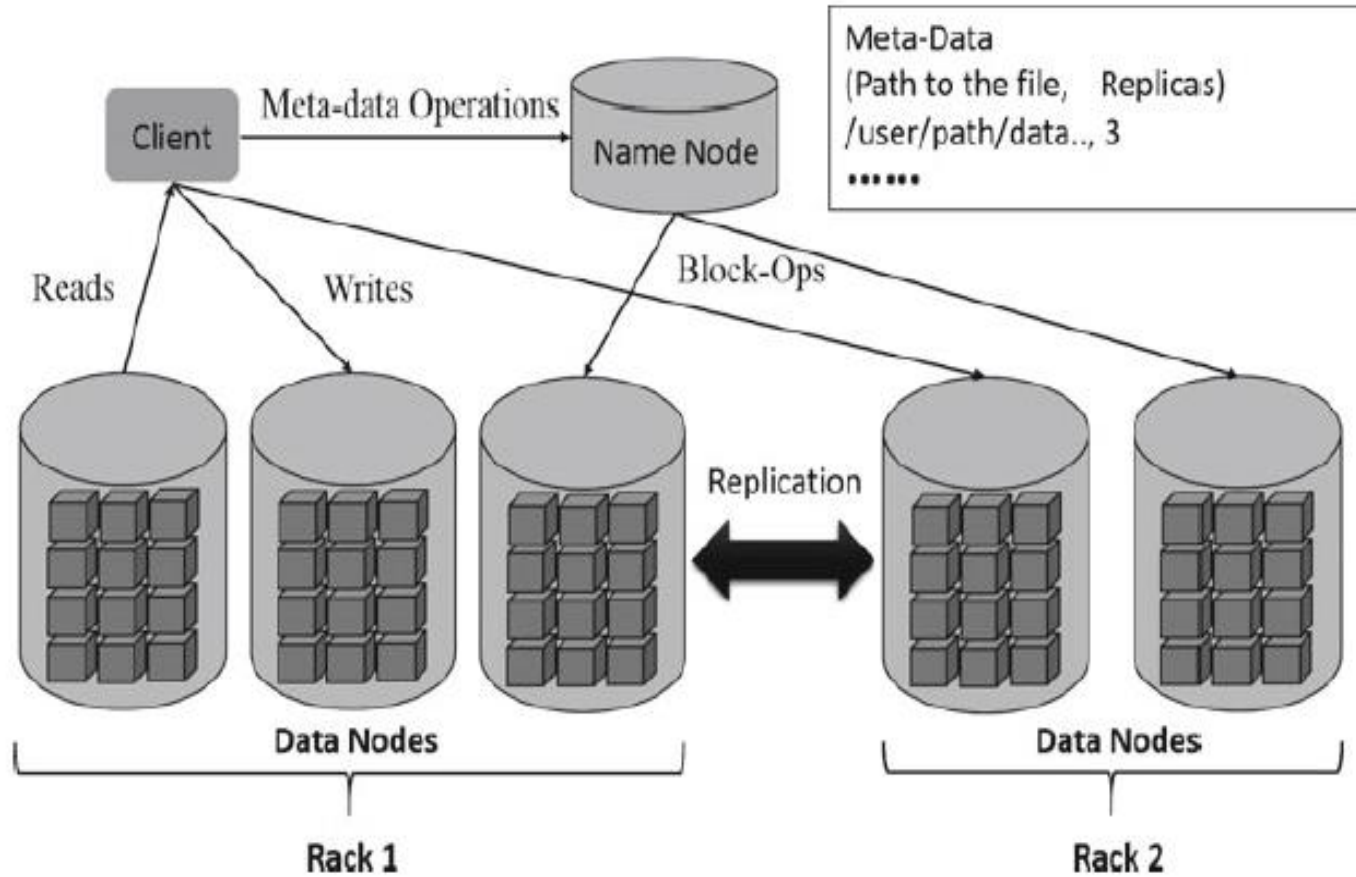


Figure 2 : Architecture des systèmes de fichiers distribués Hadoop

Le Cluster Hadoop

Qu'est-ce qu'un cluster ?

Hadoop le nom du logiciel qui s'exécute sur un cluster, à savoir le système de fichiers distribué, HDFS, et le gestionnaire de ressources de cluster, YARN, qui sont collectivement composés de six types de services d'arrière-plan exécutés sur un groupe de machines.

HDFS et YARN exposent une interface de programmation d'application (API) qui soustrait les développeurs aux détails d'administration de cluster de bas niveau.

Un ensemble de machines exécutant HDFS et YARN est appelé cluster Hadoop et les machines individuelles sont appelées nœuds. Un cluster peut avoir un seul nœud ou plusieurs milliers de nœuds, mais tous les clusters évoluent horizontalement, ce qui signifie que plus vous ajoutez de nœuds, plus la capacité et les performances du cluster augmentent de manière linéaire.

Le Cluster Hadoop

YARN et **HDFS** sont implémentés par plusieurs processus démons, c'est-à-dire des logiciels qui s'exécutent en arrière-plan et ne nécessitent aucune intervention de l'utilisateur. Les processus Hadoop sont des services, ce qui signifie qu'ils s'exécutent en permanence sur un nœud de cluster et acceptent les entrées et délivrent les sorties via le réseau, de la même manière que fonctionne un serveur HTTP. Chacun de ces processus s'exécute à l'intérieur de sa propre machine virtuelle Java (JVM), de sorte que chaque démon dispose de sa propre allocation de ressources système et est géré indépendamment par le système d'exploitation.

Le Cluster Hadoop

Chaque nœud du cluster est identifié par le type de processus qu'il exécute :

Master nodes

Ces nœuds exécutent des services de coordination pour les workers Hadoop et sont généralement les points d'entrée pour l'accès des utilisateurs au cluster. Sans masters, la coordination s'effondrerait et le stockage ou le calcul distribué ne serait pas possible.

Worker nodes

Ces nœuds représentent la majorité des ordinateurs du cluster. Les worker nodes exécutent des services qui acceptent des tâches des master nodes, soit pour stocker ou récupérer des données, soit pour exécuter une application particulière. Un calcul distribué est exécuté en parallélisant l'analyse sur les worker nodes.

HDFS et YARN ont tous deux plusieurs services master responsables de la coordination des services de worker qui s'exécutent sur chaque worker node. Les worker node implémentent à la fois les services de worker de HDFS et YARN.

Pour HDFS, les services master et worker sont les suivants :

Name Node (Master)

Stocke l'arborescence de répertoires du système de fichiers, les métadonnées des fichiers et les emplacements de chaque fichier dans le cluster. Les clients souhaitant accéder à HDFS doivent d'abord localiser les nœuds de stockage appropriés en demandant des informations au Name Node.

Name Node secondaire (master)

Effectue des tâches d'entretien ménager et des points de contrôle au nom du Name Node.

Malgré son nom, il ne s'agit pas d'un Name Node de secours.

Data Node (worker)

Stocke et gère les blocs HDFS sur le disque local. Signale la santé et l'état des stockages de données individuels au Name Node.

Accès aux données à partir de HDFS

À un niveau élevé, lors de l'accès aux données à partir de HDFS, une application cliente doit d'abord adresser une demande au **Name Node** pour localiser les données sur le disque. Le **Name Node** répondra avec une liste de **Data Nodes** qui stockent les données, et le client doit alors demander directement chaque bloc de données au **Data Node**. Notez que le **Name Node** ne stocke pas de données, ni de transmettre les données de **Data Node** au client, il agit plutôt comme un agent de la circulation, pointant les clients vers les bons **Data Nodes**.

De même, YARN a plusieurs services maîtres et un service de travail comme suit :

Resource Manager (Master)

Alloue et surveille les ressources de cluster disponibles (par exemple, les actifs physiques tels que la mémoire et les cœurs de processeur) aux applications, ainsi que la gestion de la planification des tâches sur le cluster.

Application Master (Master)

Coordonne une application particulière en cours d'exécution sur le cluster comme prévu par le Resource Manager.

Node Manager (worker)

Exécute et gère les tâches de traitement sur un nœud individuel et signale la santé et l'état des tâches pendant leur exécution.

Semblable au fonctionnement de HDFS, les clients qui souhaitent exécuter une tâche (job) doivent d'abord demander des ressources au Resource Manager, qui attribue un Application Master spécifique à l'application pour la durée de la tâche (job). L'Application Master suit l'exécution de la tâche (job), tandis que le Resource Manager suit l'état des nœuds, et chaque Node Manager individuel crée des conteneurs et exécute des tâches (jobs) à l'intérieur de ceux-ci. Notez qu'il peut également y avoir d'autres processus en cours d'exécution sur le cluster Hadoop, par exemple, des serveurs JobHistory ou des coordinateurs ZooKeeper, mais ces services sont les principaux logiciels exécutés dans un cluster Hadoop.

À un niveau élevé, lors de l'accès aux données à partir de HDFS, une application cliente doit d'abord adresser une demande au **Name Node** pour localiser les données sur le disque. Le **Name Node** répondra avec une liste de **Data Nodes** qui stockent les données, et le client doit alors demander directement chaque bloc de données au **Data Node**. Notez que le **Name Node** ne stocke pas de données, ni de transmettre les données de **Data Node** au client, il agit plutôt comme un agent de la circulation, pointant les clients vers les bons **Data Nodes**.

Le Cluster Hadoop

Un data-center contient un ensemble de racks empilés avec un ensemble de serveurs (nœuds) formant un cluster avec un réseau local haut débit. Comme Hadoop est basé sur une architecture sans partage, chaque nœud a ses disques durs de stockage. Par conséquent, les technologies de stockage SAN/NAS/Direct Attached Storage (DAS) ne sont pas utilisées.

Le Cluster Hadoop

Une configuration Hadoop typique dans un cluster physique ressemblerait à celle illustrée à la **Figure 2.2**. HDFS et MR sont tous deux basés sur une architecture maître-esclave. NN, SNN et JT sont des composants maître/serveur. DN et TT sont des composants esclaves.

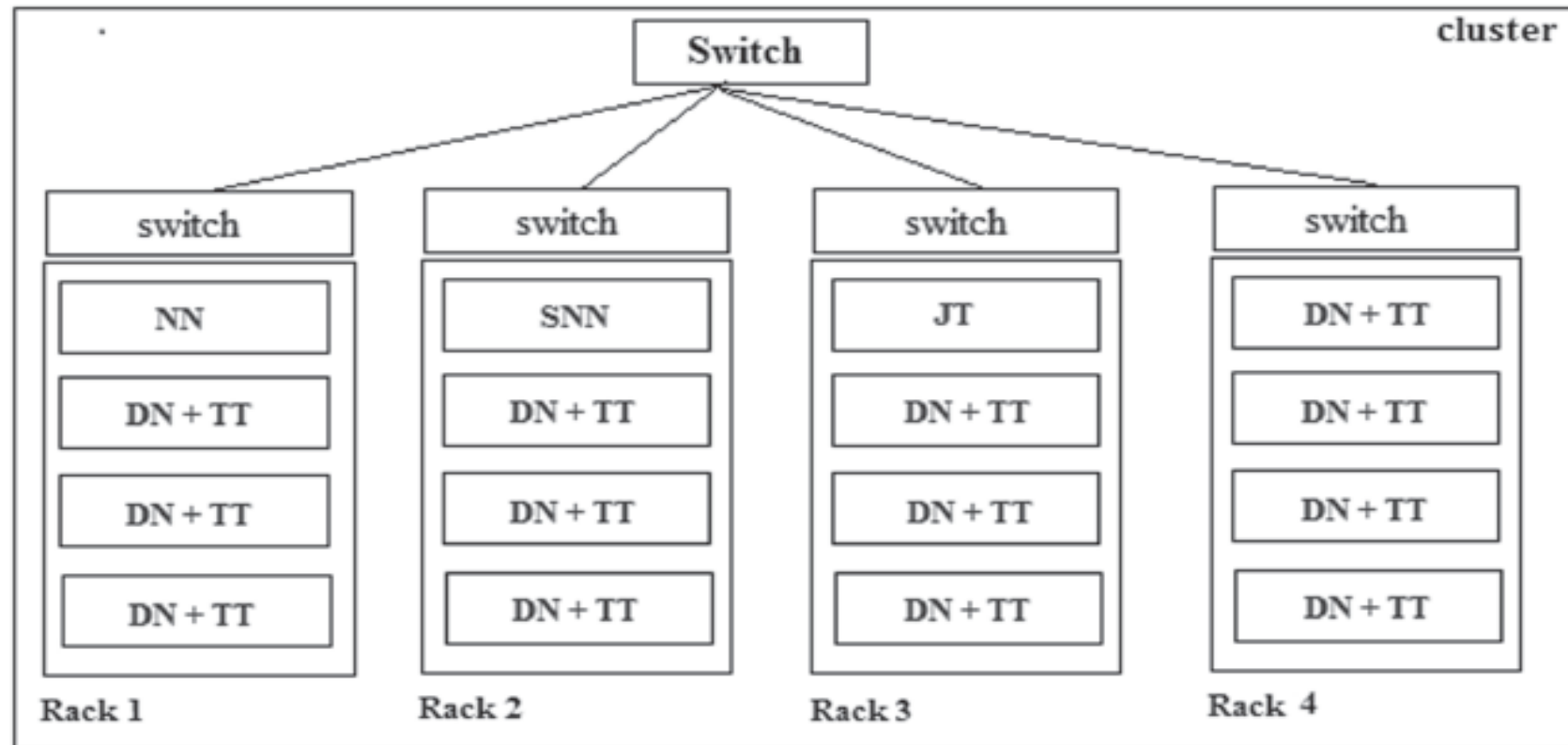


Figure 2.2 Hadoop cluster.

Le Cluster Hadoop

Dans la Figure 2.2, il y a 4 racks avec chacun 4 nœuds. Les composants Logiciel du serveur tels que NN, SNN et JT sont exécutés dans des nœuds séparés dans des racks différents. Les 13 nœuds restants sont appelés nœuds esclaves exécutant DN et TT ensemble.

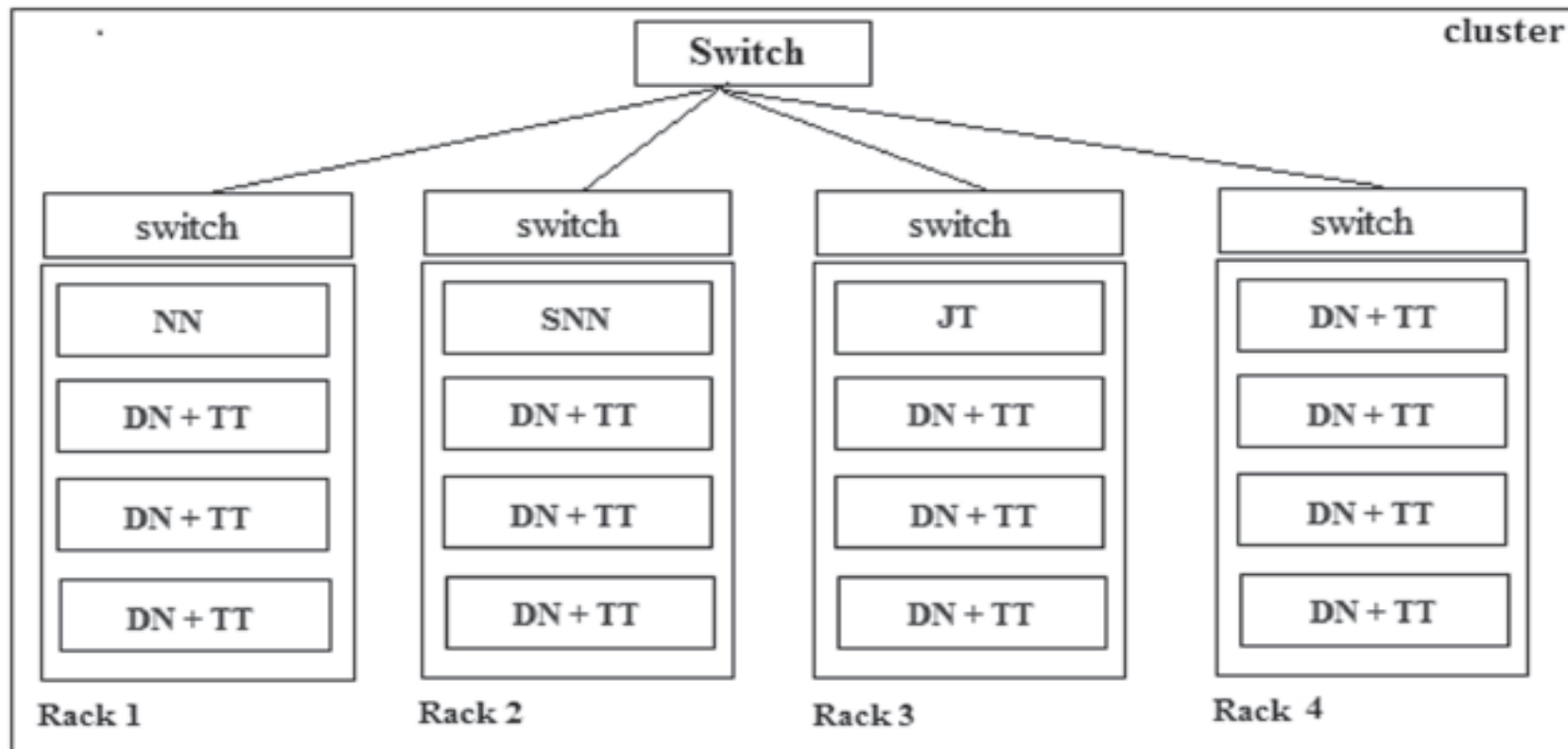


Figure 2.2 Hadoop cluster.

Communication des composants HDFS and MR

La Figure 2.3 montre la **communication entre les démons maîtres et esclave**. Les maîtres démons se parlent. Job Tracker (JT) communique avec **Task Tracker (TT)** de même **Name Node (NN)** parle avec **Data Node (DN)** et **Secondary Name Node (SNN)**. TT parle au DN pour apporter le bloc de données pour le traitement. Dans un cluster Hadoop, un seul NN, SNN, JT et plusieurs nœuds esclaves peuvent exister. Chaque nœud esclave exécute à la fois DN et TT. Les démons maîtres (NN, JT, SNN) sont généralement exécutés sur un nœud dédié pour équilibrer la charge car NN et JT sont profondément communiqués par les esclaves. **Les ressources de chaque nœud esclave (CPU et mémoire) sont gérées par TT pour l'exécution des tâches. Les disques de stockage sont gérés par DN pour le stockage/la récupération des données.**

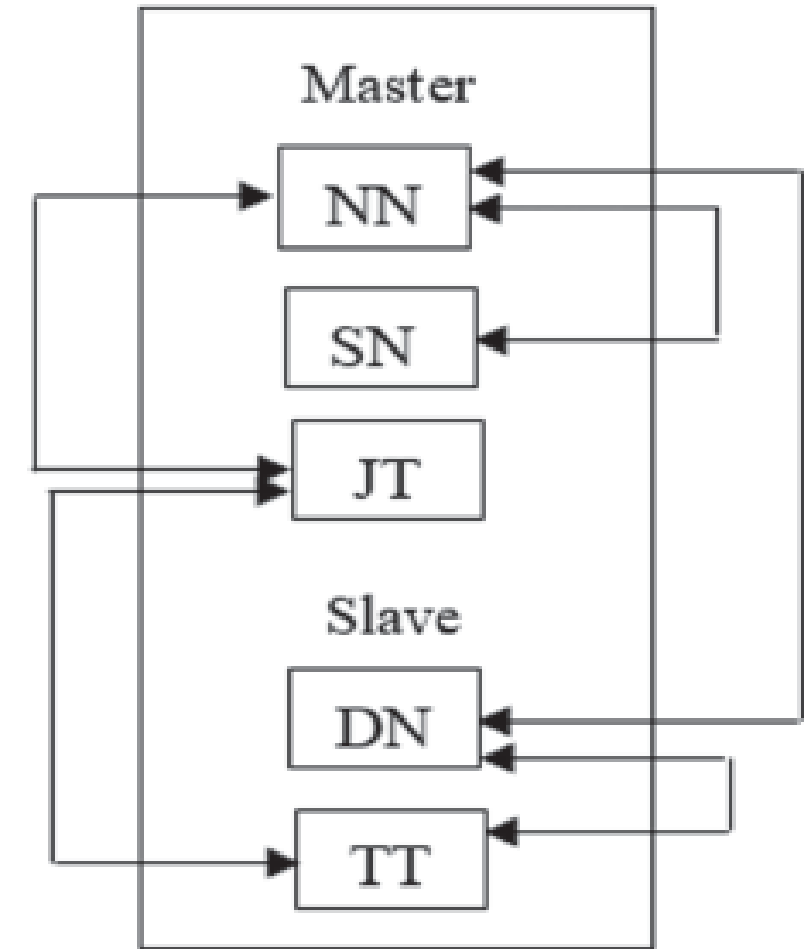


Figure 2.3 la Communication des composants HDFS and MR

5.2 NoSQL (Not only SQL)

NoSQL est **un système de gestion de base de données non relationnelle** doté d'un schéma flexible. **Il est utilisé avec des données distribuées**, en particulier **avec des applications impliquant des Big Data** et des applications avec des **données en temps réel** telles que des journaux Web et des applications.

Les bases de données NoSQL prennent en charge différents types de données telles que les **données structurées, semi-structurées et non structurées**.

NoSQL peut être décrit comme une base de données évolutive horizontalement qui convient aux applications avec des magasins de données hiérarchiques.

Base de Données NoSQL

❑ Base de données Orienté Colonne

Une base de données orientée colonnes est une base de données qui stocke les données par colonne et non par ligne. L'orientation colonne permet d'ajouter des colonnes plus facilement aux tables.

❑ Base de données Orienté Documents

Base de données orienté documents est un type de base de données non relationnelle conçu pour stocker et interroger des données sous forme de documents de type JSON (JavaScript Objet Notation). Les bases de données de documents permettent aux développeurs de stocker et d'interroger une base de données en utilisant le même format de modèle de document que celui qu'ils utilisent dans leur code d'application.

Base de Données NoSQL

❑ Bases de données basées sur les graphes:

Une base de données orientée graphe est une base de données orientée objet utilisant la théorie des graphes, donc avec des nœuds et des arcs, permettant de représenter et stocker les données.

❑ Bases de données clé-valeur

Une base de données clé-valeur est un type de base de données non relationnelle qui utilise une méthode clé-valeur simple pour stocker des données. Une base de données clé-valeur stocke les données sous forme de paires clé-valeur dans lesquelles une clé sert d'identifiant unique.

5.2.1 Les de Base de Données NoSQL Populaires

Base de données Orienté Colonne:

1. **Hbase**
2. Hypertable
3. **Cassandra**
4. Accumulo

Base de données Orienté Documents:

1. **MongoDB**
2. CouchDB
3. Terrastore
4. RavenDB

Bases de données clé-valeur :

1. Riak
2. Infinispan
3. Hazelcast
4. Redis
5. Voldemort

Bases de données basées sur les graphes:

1. Neo4J
2. InfiniteGraph
3. InfoGrid
4. HypergraphDB
5. AllegroGraph

LES MEILLEURES APPLICATIONS POUR CERTAINS TYPES DE VALEURS CLES

(Khasawneh et al, 2020)

RIAK est le meilleur pour la haute disponibilité, la tolérance de partition, la persistance.

REDIS est le meilleur pour les données changeant rapidement, les données statistiques fréquemment écrites, rarement lues.

VOLDEMORT est le meilleur pour les applications avec une grande exigence de capacité de données.

LES MEILLEURES APPLICATIONS POUR CERTAINS TYPES ORIENTÉS SUR COLONNE

HBASE est le meilleur pour l'accès en temps réel, les opérations en bloc (indexation, ..).

CASSANDRA est la meilleure lorsque vous écrivez plus que vous lisez.

ACCUMULO est le meilleur pour l'accès au niveau de la cellule.

LES MEILLEURES APPLICATIONS POUR CERTAINS DOCUMENTS ORIENTÉS

(Khasawneh et al,2020)

MONGODB est le meilleur pour les requêtes dynamiques, la définition d'index, de bonnes performances sur les grosses bases de données.

COUCHDB est idéal pour l'accumulation, les données changeant occasionnellement, les requêtes prédéfinies à exécuter, les cas d'utilisation Web et les applications mobiles

5.3 Technologies de Traitement Big Data

Les plateformes de traitement de Big Data les plus courants sont

1. **Apache Hadoop**
2. **Apache Spark**
3. **Apache Storm**
4. **Apache Flink**

5.3 Technologies de Traitement Big Data

1. Apache Hadoop

Apache Hadoop est un framework open source pour le stockage et le traitement de grands ensembles de données. Hadoop peut gérer de grandes quantités de données structurées et non structurées.

2. Apache Storm

Apache Storm est un framework open source qui a été conçu pour traiter les données de streaming en temps réel.

3. Apache Flink

Apache Flink est un framework open source utilisé efficacement pour le traitement des données à la fois en temps réel et en mode batch.

5.3 Technologies de Traitement Big Data

2. Apache Spark

Apache Spark est un framework ou un moteur d'analyses unifiées ultra-rapide pour le big data et le machine learning. Il a initialement été conçu à l'Université de Californie à Berkeley en 2009.

Spark s'exécute sur des clusters Hadoop tels que **Hadoop YARN** ou **Apache Mesos**, ou même de **manière autonome avec son propre planificateur**.

Spark étend le modèle **MapReduce** et le prend à un autre niveau. La vitesse provient des calculs en mémoire (the in-memory computations). Les applications exécutées en mémoire permettent un traitement et une réponse beaucoup plus rapides. **Spark est encore plus rapide que Mapreduce pour des applications complexes sur les disques**. De plus, Spark propose un partage de données appelé Resilient Distributed Datasets (RDD) et Directed Acyclic Graph (DAG)

5. 3. 1 Architecture d'Apache Hadoop

L'architecture de Hadoop est composée de trois couches principales:

La première est la couche de stockage de données pour la collecte de données, qui contient Hadoop Distributed File System (HDFS).

La deuxième couche est l'infrastructure YARN, qui assure la gestion des ressources pour la planification des tâches telles que le processeur et la mémoire.

La troisième est **MapReduce**, qui est utilisé pour le traitement données (couche logicielle) avec d'autres processus .

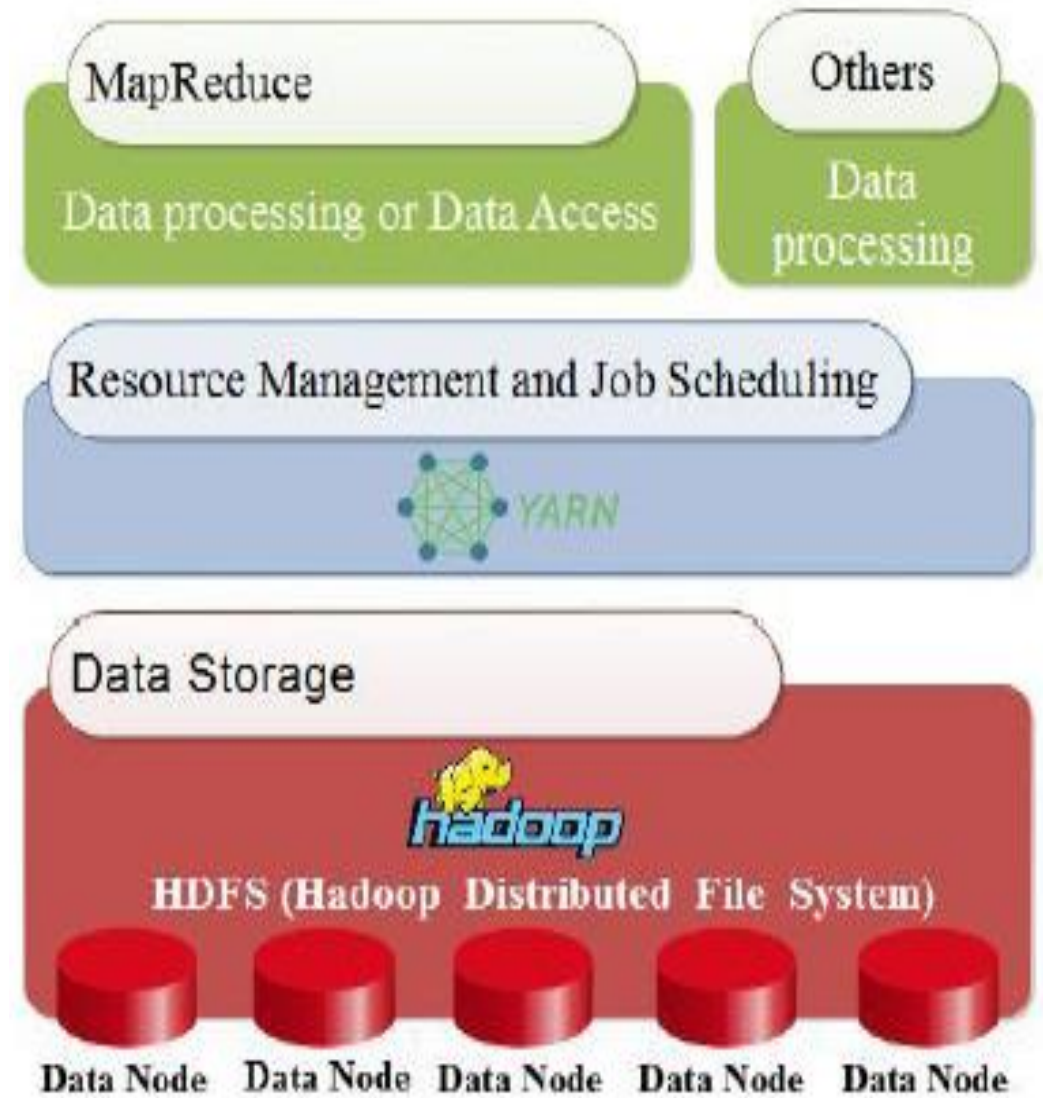


Figure 3 : Architecture de Hadoop
(Gupta et Parveen, 2019)

5.3.2 Ecosystème Apache

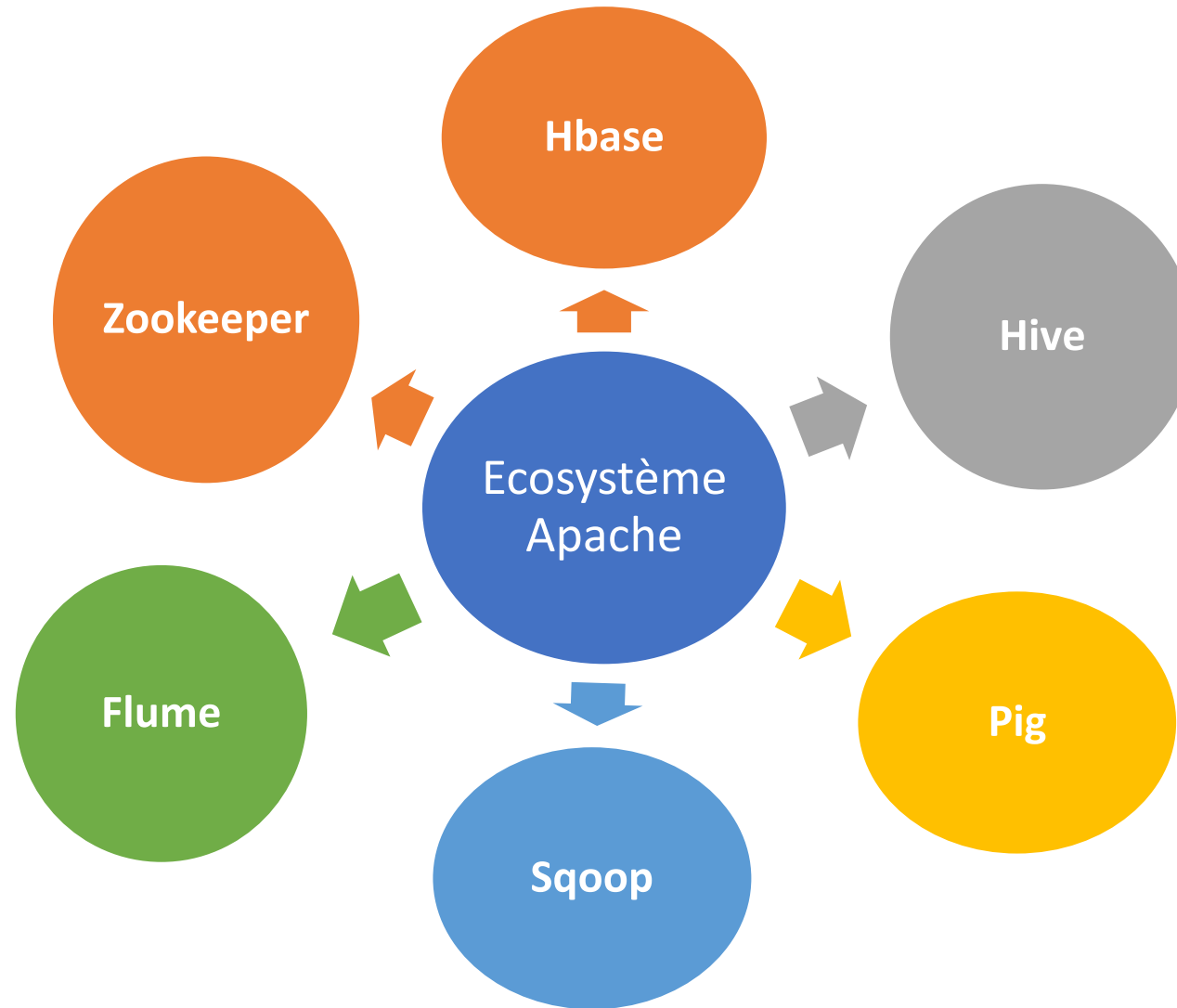


Figure 4: Ecosystème Apache

5.3.2 Ecosystème Apache (suite)

❑ Apache Hbase

HBase est une base de données non relationnelle ou orientée colonnes qui s'exécute au-dessus de HDFS (système de fichiers distribué Hadoop). HBase utilise HDFS pour son stockage de données et prend en charge à la fois les calculs par lots utilisant MapReduce et les requêtes ponctuelles. HBase peut traiter des pétaoctets de données.

❑ Apache Hive

Hive est un langage de type SQL appelé HiveQL. De plus, c'est une infrastructure d'entrepôt de données qui fournit toutes ces fonctionnalités. **Hive** permet d'interroger les données de HDFS et ces requêtes sont converties en tâches de MapReduce.

❑ Apache Sqoop

Sqoop est un outil pour déplacer efficacement les données entre les bases de données relationnelles et HDFS.

5.3.2 Ecosystème Apache (suite)

❑ Apache Flume

Flume est une application basée sur Hadoop qui permet de déplacer des données en flux (streaming data) dans un cluster Hadoop ou HBase.

❑ Apache Pig

Pig est un langage de script utilisé pour écrire des programmes de traitement de grands jeux de données présents dans le cluster Hadoop. **Pig exécute ces programmes et les convertit en tâches de MapReduce**, puis exécute ces tâches.

❑ Apache ZooKeeper

Zookeeper est un outil centralisé qui aide à la synchronisation à travers le cluster Hadoop. **Les services de nommage et la gestion de la configuration sont également fournis par ZooKeeper sur le cluster Hadoop.**

5.3.3 Architecture d'Apache Spark

L'architecture de Spark est composée principalement de **5 couches**.

1. La première couche comprend des **systèmes de stockage de données** tels que HDFS et HBASE.
2. Le **gestionnaire des ressources**; par exemple, YARN et Mesos.
3. Un **moteur principal Spark**.
4. Une **bibliothèque** composée de SQL, de traitement de données en flux, de MLlib pour l'apprentissage automatique, de Spark R et de GraphX pour le traitement de graphes.
5. Une **interface de programme d'application** telle que Java ou Scala.

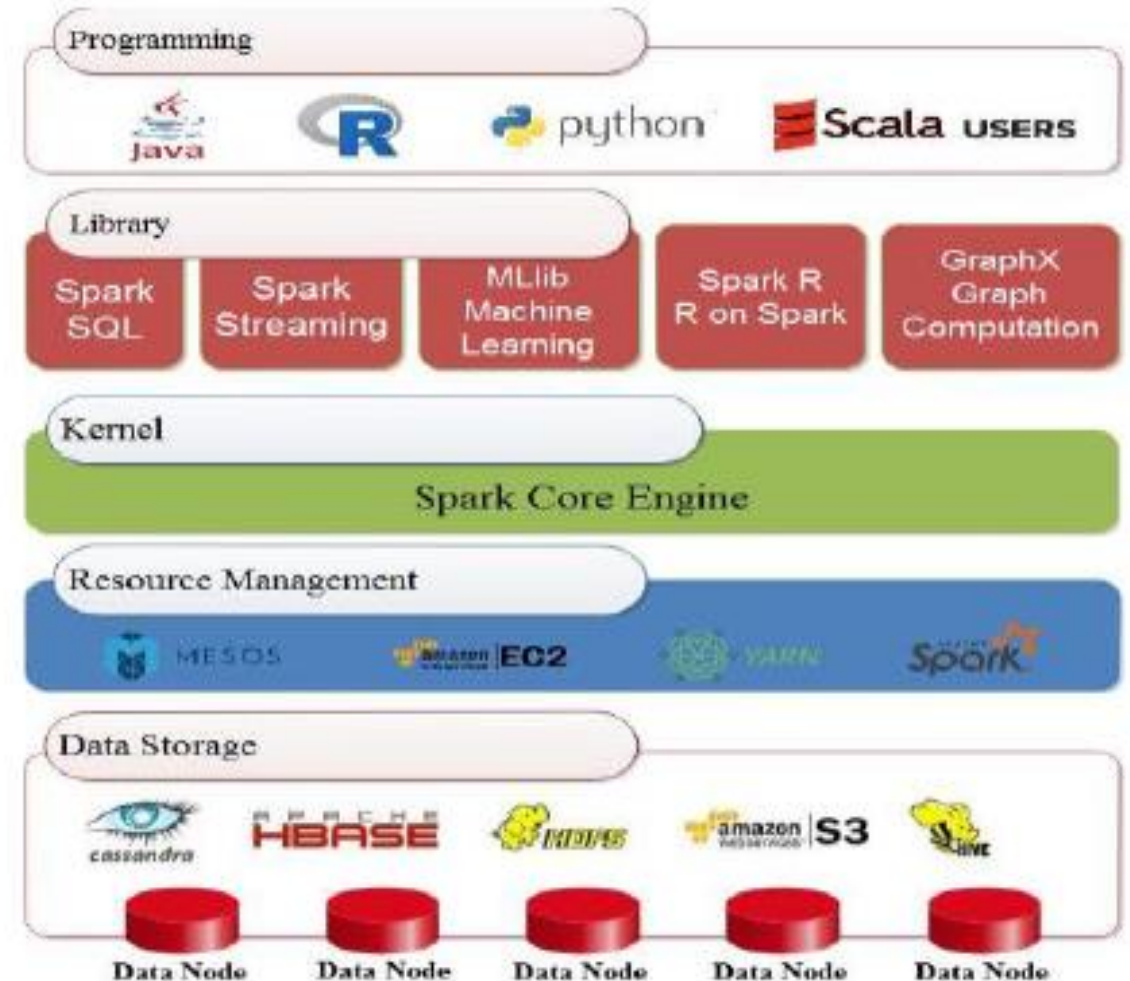


Figure 5 : Architecture de Spark
(Gupta et Parveen, 2019)

5.3.4 Architecture d'Apache Storm

L'architecture de Storm est composée de deux types de nœuds : **master node** (nœud maître) et le **worker node** (nœud ouvrier).

Le **master node** est utilisé pour surveiller les défaillances, prendre la responsabilité du nœud distribué et spécifier chaque tâche pour chaque machine. Toutes ces tâches sont collectivement connues sous le nom de **Nimbus**.

Le **worker node** est appelé **superviseur**. Cela fonctionne lorsque Nimbus lui attribue un processus spécifique. Ainsi, chaque sous-processus d'une topologie fonctionne avec de nombreuses machines distribuées. **Zookeeper** joue le rôle de coordinateur entre Nimbus et les superviseurs..

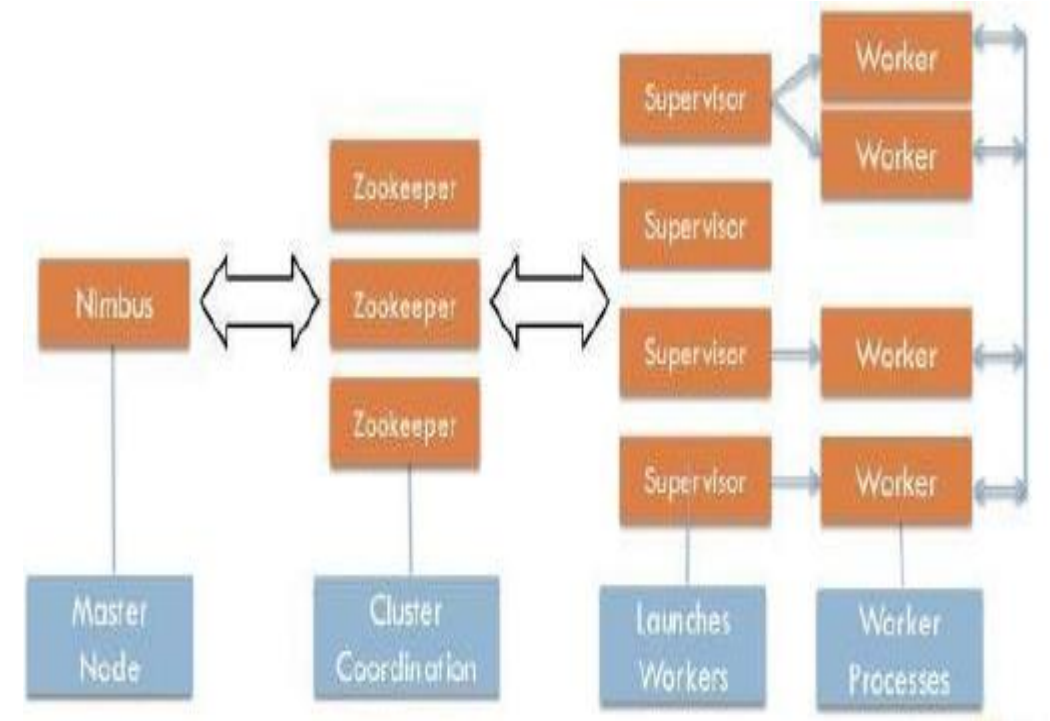


Figure 6 : Architecture de Storm
(Gupta et Parveen, 2019)

5.3.5 Architecture d'Apache Flink

L'architecture de Flink est composée de 5 couches :

1. La **couche de stockage** peut lire et écrire les données à partir de plusieurs destinations telles que HDFS, des fichiers locaux, etc.
2. La **couche de déploiement et de gestion des ressources** contient le gestionnaire de cluster pour gérer les tâches de planification, de surveillance des travaux et de gestion des ressources.
3. La **couche noyau** pour le moteur de traitement en temps réel (real time processing).
4. La **couche d'interface de programmation d'application** pour : batch API et streaming API.
5. La couche supérieure est **une bibliothèque** dans laquelle le programme est écrit en langage de programmation Java ou Scala.

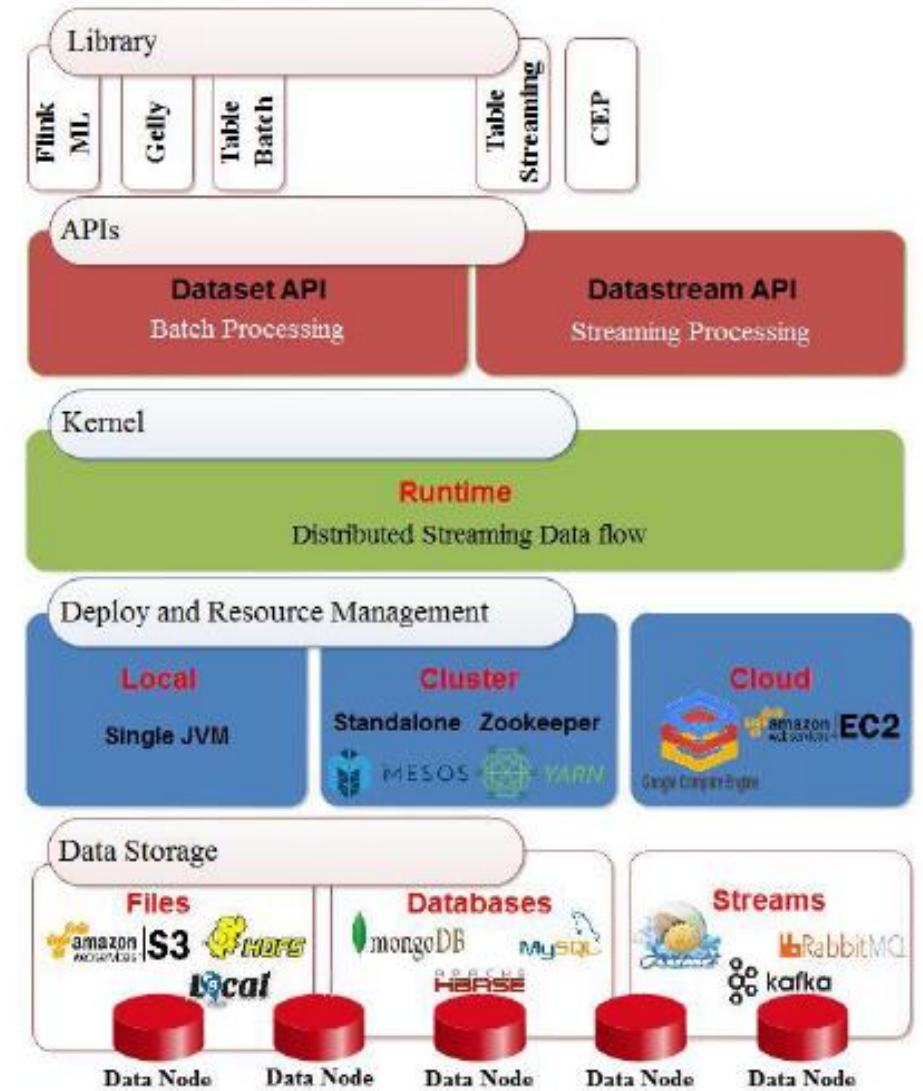


Figure 7 : Architecture de Flink
(Gupta et Parveen, 2019)

6. Quelques Domaines d'Application du Big Data

1) Analyse du comportement des clients

- ❖ L'analyse des journaux Web (web log) des sites de commerce électronique ;
- ❖ L'analyse des journaux d'appel des abonnés par des entreprises de télécommunications;
- ❖ L'analyse des journaux d'achat dans les grands supermarchés

2) Systèmes de recommandation aux clients

3) Secteur de la vente au détail (Retail Sector)

Pour offrir un meilleur service et prédire quel article est le plus demandé, le détaillant utilise les données de facture du client et essaie de prédire des informations utiles pour la veille économique.

4) Ciblage publicitaire

Les sites des réseaux sociaux ciblent l'utilisateur suivant les informations de son profil et ses activités afin de lui montrer des publicités selon ses goûts.

7. Les Défis du Big Data

Les défis de l'analyse des Big Data comprennent:

- ❖ La capture des données
- ❖ Le stockage des données
- ❖ Traitement des données
- ❖ L'analyse des données
- ❖ La confidentialité des informations

Références Bibliographiques

1. Tole, A. A. (2013). Big data challenges. *Database systems journal*, 4(3), 31-40.
2. Nasser, T., & Tariq, R. S. (2015). Big data challenges. *J Comput Eng Inf Technol* 4: 3. doi: [http://dx. doi. org/10.4172/2324.9307\(2\)](http://dx.doi.org/10.4172/2324.9307(2)).
3. Alkatheri, S., Abbas, S., & Siddiqui, M. (2019). A comparative study of big data frameworks. *International Journal of Computer Science and Information Security (IJCSIS)*, 17(1).
4. Alkatheri, S., Abbas, S., & Siddiqui, M. (2019). A comparative study of big data frameworks. *International Journal of Computer Science and Information Security (IJCSIS)*, 17(1).
5. Gupta, H. K., & Parveen, R. (2019, September). Comparative Study of Big Data Frameworks. In *2019 International Conference on Issues and Challenges in Intelligent Computing Techniques (ICICT)* (Vol. 1, pp. 1-4). IEEE.
6. Zomaya, A. Y., & Sakr, S. (Eds.). (2017). Handbook of big data technologies.
7. Wu, D., Sakr, S., & Zhu, L. (2017). Big data storage and data models. In *Handbook of big data technologies* (pp. 3-29). Springer, Cham.
8. Khasawneh, T. N., AL-Sahlee, M. H., & Safia, A. A. (2020, April). SQL, NewSQL, and NOSQL databases: a comparative survey. In *2020 11th International Conference on Information and Communication Systems (ICICS)* (pp. 013-021). IEEE.
9. Kotiyal, B., Kumar, A., Pant, B., & Goudar, R. H. (2013, August). Big data: Mining of log file through Hadoop. In *2013 International Conference on Human Computer Interactions (ICHCI)* (pp. 1-7). IEEE.
10. Singh, J., & Singla, V. (2015). Big data: tools and technologies in big data. *International Journal of Computer Applications*, 112(15).
11. McQueen, B. (2017). Big data analytics for connected vehicles and smart cities. Artech House.

Introduction au Big Data Analytics

Dr. A. SIDIBE

Big Data Analytics ou Analyse des Big Data

Définition

L'analyse des big data ou données massives est le processus souvent complexe d'examen des big data pour découvrir des informations - telles que des modèles cachés, des corrélations, des tendances du marché et des préférences des clients - qui peuvent aider les organisations à prendre des décisions commerciales éclairées.

L'analyse des Big data est une forme d'analyse avancée, qui implique des applications complexes avec des éléments tels que des modèles prédictifs, des algorithmes statistiques et des analyses de simulation alimentées par des systèmes d'analyse.

À grande échelle, les technologies et techniques d'analyse de données offrent aux organisations un moyen d'analyser des ensembles de données et de recueillir de nouvelles informations. Les requêtes de Business Intelligence (BI) répondent à des questions de base sur les opérations et les performances de l'entreprise.

Pourquoi l'Analyse des Big Data est-elle Importante ?

Les organisations ou entreprises peuvent utiliser des systèmes et des logiciels d'analyse de Big Data pour prendre des décisions basées sur les données qui peuvent améliorer les résultats commerciaux. Les avantages peuvent inclure un marketing plus efficace, de nouvelles opportunités de revenus, la personnalisation des clients et une efficacité opérationnelle améliorée. Avec une stratégie efficace, ces avantages peuvent fournir des avantages concurrentiels par rapport aux concurrents.

Analytics

Définition

Analytics ou l'analyse est la découverte, l'interprétation et la communication de modèles significatifs dans les données. Particulièrement utile dans les domaines riches en informations enregistrées, l'analyse repose sur l'application simultanée de statistiques, de programmation informatique et de recherche opérationnelle pour quantifier les performances. L'analyse privilégie souvent la visualisation des données pour communiquer des informations.

Les organisations peuvent appliquer des analyses aux données commerciales pour décrire, prédire et améliorer les performances commerciales. Plus précisément, les domaines de l'analyse comprennent **l'analyse prédictive**, **l'analyse prescriptive**, **la gestion des décisions d'entreprise**, **l'analyse de la vente au détail**, l'optimisation de l'assortiment et des unités de stockage des magasins, l'optimisation du marketing et la modélisation du mix marketing, l'analyse Web, dimensionnement et optimisation de la force de vente, modélisation des prix et des promotions, science prédictive, analyse du risque de crédit et analyse de la fraude. Étant donné que l'analyse peut nécessiter des calculs approfondis, les algorithmes et les logiciels utilisés pour l'analyse exploitent les méthodes les plus actuelles en informatique, en statistiques et en mathématiques.

Les phases du cycle de vie de Big Data Analytics

Les phases du cycle de vie de Big Data Analytics

Passons maintenant en revue le fonctionnement de l'analyse Big Data :

- **Étape 1 - Évaluation de l'analyse de rentabilisation** - Le cycle de vie de l'analyse du Big Data commence par une analyse de rentabilisation, qui définit la raison et l'objectif de l'analyse.
- **Étape 2 - Identification des données** - Ici, une grande variété de sources de données sont identifiées.
- **Étape 3 - Filtrage des données** - Toutes les données identifiées de l'étape précédente sont filtrées ici pour supprimer les données corrompues.
- **Étape 4 - Extraction des données** - Les données non compatibles avec l'outil sont extraites puis transformées sous une forme compatible.
- **Étape 5 - Agrégation des données** - Dans cette étape, les données avec les mêmes champs dans différents ensembles de données sont intégrées.

Les phases du cycle de vie de Big Data Analytics

- **Étape 6 - Analyse des données** - Les données sont évaluées à l'aide d'outils analytiques et statistiques pour découvrir des informations utiles.
- **Étape 7 - Visualisation des données** - Avec des outils tels que Tableau, Power BI et QlikView, les analystes Big Data peuvent produire des visualisations graphiques de l'analyse.
- **Étape 8 - Résultat final de l'analyse** - Il s'agit de la dernière étape du cycle de vie de l'analyse Big Data, où les résultats finaux de l'analyse sont mis à la disposition des parties prenantes de l'entreprise qui prendront des mesures.

Différents Types de Big Data Analytics

Les quatre types d'analyse Big Data sont les suivants :

- 1. Analyse descriptive**
- 2. Analyse diagnostique**
- 3. Analyse prédictive**
- 4. Analyse prescriptive**

Différents types de Big Data Analytics

Les quatre types d'analyse Big Data sont :

1. Analyse descriptive

L'Analyse descriptive résume les données passées sous une forme que les gens peuvent facilement lire. **Cela aide à créer des rapports, comme les revenus, les bénéfices, les ventes d'une entreprise, etc.** En outre, cela aide à la tabulation des mesures des médias sociaux.

Cas d'utilisation : The Dow Chemical Company a analysé ses données historiques (ou passées) pour augmenter l'utilisation des installations dans ses bureaux et ses laboratoires. À l'aide d'analyses descriptives, Dow a pu identifier l'espace sous-utilisé. Cette consolidation de l'espace a permis à l'entreprise d'économiser près de 4 millions de dollars US par an.

Différents types de Big Data Analytics

2. Analyse diagnostique

L'Analyse diagnostique est fait pour comprendre ce qui a causé un problème en premier lieu. Des techniques telles que l'exploration en profondeur, l'exploration de données et la récupération de données en sont des exemples. Les organisations utilisent l'analyse diagnostique parce qu'elle fournit un aperçu approfondi d'un problème particulier.

Cas d'utilisation : le rapport d'une entreprise de commerce électronique montre que ses ventes ont diminué, bien que les clients ajoutent des produits à leurs paniers. Cela peut être dû à diverses raisons, telles que le formulaire ne s'est pas chargé correctement, les frais d'expédition sont trop élevés ou il n'y a pas suffisamment d'options de paiement disponibles. C'est là que vous pouvez utiliser l'analyse de diagnostic pour trouver la raison.

Différents types de Big Data Analytics

3. Analyse prédictive

L'Analyse prédictive examine les données historiques et actuelles pour faire des prédictions sur l'avenir. L'analyse prédictive utilise l'exploration de données, l'IA et l'apprentissage automatique pour analyser les données actuelles et faire des prédictions sur l'avenir. Il fonctionne sur la prévision des tendances des clients, des tendances du marché, etc.

Cas d'utilisation : PayPal détermine le type de précautions à prendre pour protéger ses clients contre les transactions frauduleuses. À l'aide de l'analyse prédictive, l'entreprise utilise toutes les données historiques de paiement et les données de comportement des utilisateurs et construit un algorithme qui prédit les activités frauduleuses.

Différents types de Big Data Analytics

4. Analyse prescriptive

L'Analyse prescriptive prescrit la solution à un problème particulier. L'analyse **prescriptive** fonctionne à la fois avec l'analyse descriptive et prédictive. La plupart du temps, il s'appuie sur l'IA et l'apprentissage automatique.

Cas d'utilisation : l'analyse prescriptive peut être utilisée pour maximiser les bénéfices d'une compagnie aérienne. Ce type d'analyse est utilisé pour créer un algorithme qui ajustera automatiquement les tarifs des vols en fonction de nombreux facteurs, notamment la demande des clients, la météo, la destination, les périodes de vacances et les prix du pétrole.

Les Technologies et outils clés d'analyse des Big Data

De nombreux types d'outils et de technologies différents sont utilisés pour prendre en charge les processus d'analyse des Big Data. Les technologies et outils courants utilisés pour activer les processus d'analyse des Big Data incluent :

- ❑ **Hadoop, Map Reduce et No SQL** sont les principales technologies Big Data. Ces technologies sont très utiles dans la gestion des Big Data. Des technologies basées sur Hadoop sont appelées **Hadoop Eco system**.
- ❑ **Hadoop**, qui est un framework open source pour le stockage et le traitement de grands ensembles de données. Hadoop peut gérer de grandes quantités de données structurées et non structurées.
- ❑ **Du matériel et des logiciels d'analyse prédictive**, qui traitent de grandes quantités de données complexes et utilisent l'apprentissage automatique (machine learning) et des algorithmes statistiques pour faire des prédictions sur les résultats des événements futurs. Les entreprises utilisent des outils d'analyse prédictive pour la détection des fraudes, le marketing, l'évaluation des risques et les opérations.
- ❑ **Des outils d'analyse de stream data (flux de donnée)**, qui sont utilisés pour filtrer, agréger et analyser les big data qui peuvent être stockées dans de nombreux formats ou plateformes différents.
- ❑ **Stockage de données distribué**, qui est répliqué, généralement sur une base de données non relationnelle. Cela peut être une mesure contre les défaillances de nœuds indépendants, les données volumineuses perdues ou corrompues, ou pour fournir un accès à faible latence.

Les Technologies et outils clés d'analyse des Big Data

- ❑ **Les bases de données NoSQL**, qui sont des systèmes de gestion de données non relationnelles utiles lorsque vous travaillez avec de grands ensembles de données distribuées. Ils ne nécessitent pas de schéma fixe, ce qui les rend idéaux pour les données brutes et non structurées.
- ❑ Un **lac de données (data lake)** est un grand référentiel de stockage (ou entrepôt) qui contient des données brutes au format natif jusqu'à ce qu'elles soient nécessaires.
- ❑ Un **entrepôt de données (data warehouse)**, qui est un référentiel qui stocke de grandes quantités de données collectées par différentes sources. Les entrepôts de données stockent généralement les données à l'aide de schémas prédéfinis.

Les Technologies et outils clés d'analyse des Big Data

- ❑ **Des outils de découverte de connaissances/fouille de données massives (big data mining tools),** qui permettent aux entreprises d'explorer de grandes quantités de données volumineuses structurées et non structurées.
- ❑ **La virtualisation des données, qui permet l'accès aux données sans restrictions techniques.**
- ❑ **Logiciel d'intégration de données,** qui permet de rationaliser le Big Data sur différentes plateformes, notamment Apache, Hadoop, MongoDB et Amazon **EMR**.
- ❑ **Logiciel de qualité des données,** qui nettoie et enrichit les grands ensembles de données.
- ❑ **Logiciel de prétraitement des données,** qui prépare les données pour une analyse ultérieure. Les données sont formatées et les données non structurées sont nettoyées.
- ❑ **Spark,** qui est un framework informatique en cluster open source utilisé pour le traitement de données par lots et par flux

Utilisations et exemples de l'analyse des Big data

Ci-dessous nous avons quelques exemples de la façon dont l'analyse des Big data peut être utilisée pour aider les organisations

- **Acquisition et fidélisation de la clientèle:** Les données sur les consommateurs peuvent aider les efforts de marketing des entreprises, qui peuvent agir sur les tendances pour augmenter la satisfaction des clients. Par exemple, les moteurs de personnalisation pour Amazon, Netflix et Spotify peuvent améliorer l'expérience client et fidéliser la clientèle.
- **Annonces ciblées:** Les données de personnalisation provenant de sources telles que les achats passés, les modèles d'interaction et les historiques de consultation des pages de produits peuvent aider à générer des campagnes publicitaires ciblées et attrayantes pour les utilisateurs au niveau individuel et à plus grande échelle.

Utilisations et exemples de l'analyse des Big data

- **Développement de produits:** L'analyse des Big data peut fournir des informations pour informer sur la viabilité du produit, les décisions de développement, la mesure des progrès et orienter les améliorations dans la direction de ce qui convient aux clients d'une entreprise.
- **Optimisation des prix:** Les détaillants peuvent opter pour des modèles de tarification qui utilisent et modélisent des données provenant de diverses sources de données afin de maximiser les revenus.
- **Gestion des risques:** L'analyse des Big Data peut identifier de nouveaux risques à partir des modèles de données pour des stratégies de gestion des risques efficaces.
- **Prise de décision améliorée:** Les informations que les utilisateurs métier extraient des données pertinentes peuvent aider les organisations à prendre des décisions plus rapides et plus efficaces.

Les Avantages de l'Analyse des Big Data

Les avantages de l'utilisation de l'analyse de données volumineuses incluent :

- **Analyser rapidement de grandes quantités de données** provenant de différentes sources, dans de nombreux formats et types différents.
- **Prendre rapidement des décisions mieux informées pour une stratégie efficace**, ce qui peut bénéficier et améliorer la chaîne d'approvisionnement, les opérations et d'autres domaines de prise de décision stratégique.
- **Économies de coûts**, qui peuvent résulter de nouvelles efficacités et optimisations des processus métier.
- **Une meilleure compréhension des besoins**, du comportement et des sentiments des clients, ce qui peut conduire à de meilleures informations marketing et fournir des informations pour le développement de produits.
- **Des stratégies de gestion des risques améliorées et mieux informées** qui s'appuient sur de grands échantillons de données.

Les Défis du Big data Analytics

Malgré les nombreux avantages liés à l'utilisation de l'analyse des Big data, son utilisation présente également des défis :

- **Accessibilité des données.** Avec de plus grandes quantités de données, le stockage et le traitement deviennent plus compliqués. Les Big data doivent être stockées et entretenues correctement pour garantir qu'elles puissent être utilisées par des scientifiques et des analystes de données moins expérimentés.
- **Maintien de la qualité des données.** Avec des volumes élevés de données provenant de diverses sources et dans différents formats, la gestion de la qualité des données pour les Big data nécessite beaucoup de temps, d'efforts et de ressources pour la maintenir correctement.
- **Sécurité des données.** La complexité des systèmes de Big Data présente des défis de sécurité uniques. Répondre correctement aux problèmes de sécurité au sein d'un écosystème de Big Data aussi complexe peut être une entreprise complexe.
- **Choisir les bons outils.** Choisir parmi la vaste gamme d'outils et de plates-formes d'analyse de Big Data disponibles sur le marché peut être déroutant. Les organisations doivent donc savoir comment choisir le meilleur outil qui correspond aux besoins et à l'infrastructure des utilisateurs.
- Avec un manque potentiel de compétences analytiques internes et le coût élevé de l'embauche de data scientists et d'ingénieurs expérimentés, certaines organisations ont du mal à combler les lacunes.

Hadoop Framework

Introduction

Gouge Cutting et Mike Cafarella ont créé Hadoop (**H**ighly **A**rchived **D**istributed **O**bject **O**riented **P**rogramming) en 2005. Cependant, **MapReduce**, basé sur Google publié en 2004, a été développé pour aider un projet d'analyse distribuée et de moteur de recherche. La meilleure caractéristique est qu'il s'agit d'une technologie Java Framework basée sur l'open source, qui permet de stocker, de gérer, de générer un accès et d'affecter de vastes ressources et détermine le haut degré flexible de possibilité de panne et la grande évolutivité à partir de données volumineuses et à moindre coût. Hadoop contrôle un grand nombre d'ensembles de données et génère un énorme ensemble de données à partir de différents systèmes de plate-forme tels que des vidéos et des audios, des dossiers, des images, des fichiers, des logiciels et des discussions par e-mail. Toutes ces extensions et formats sont stockés dans un Hadoop conçu par clustering, et sans schéma, cela conduit à collecter sur tous les systèmes qui pourraient être différents.

De plus, il existe de nombreuses structures et composants trouvés dans Hadoop, par exemple **Avro**, **Chukwa**, **Flume**, etc. L'installation ou le bundle Hadoop nous fournit plusieurs fonctionnalités telles que le code source, la planification et l'organisation du travail.

Avantages de Hadoop MapReduce

MapReduce présente de nombreux avantages ; Certains d'entre eux sont énumérés ci-dessous.

Vitesse : il peut traiter de grandes quantités de données aléatoires en peu de temps.

Tolérance aux pannes : le framework MapReduce peut gérer les pannes.

Évolutivité : Hadoop fournit un excellent cadre. MapReduce permet aux utilisateurs d'exécuter des applications sur plusieurs nœuds.

Disponibilité des données : les blocs de données sont envoyées à divers endroits du réseau.

Cela garantit que des copies des données sont disponibles en cas de panne.

Traitement parallèle : sur MapReduce, le traitement des données peuvent être effectué sur plusieurs machine. Cela réduit le temps nécessaire pour terminer la tâche.

Avantages de Hadoop MapReduce

Robuste et évolutif - Nous pouvons ajouter de nouveaux nœuds au besoin et les modifier.

Abordable et rentable - Nous n'avons besoin d'aucun matériel spécial pour exécuter Hadoop. Nous pouvons simplement utiliser un serveur de produits de base.

Adaptatif et flexible - Hadoop est conçu en gardant à l'esprit qu'il gèrera des données structurées et non structurées.

Hautement disponible et tolérant aux pannes – Lorsqu'un nœud tombe en panne, le framework Hadoop bascule automatiquement vers un autre nœud.

Applications de Hadoop MapReduce

❑ Commerce électronique

Les entreprises de commerce électronique comme **Walmart**, **eBay** et **Amazon** utilisent MapReduce pour analyser le comportement des consommateurs. MapReduce fournit des informations solides qui sont utilisées comme base pour développer des recommandations de produits. Les autres informations incluent les enregistrements de site, le catalogue de commerce électronique, l'historique des achats et les journaux de contact.

❑ Réseaux sociaux

L'outil d'édition MapReduce peut vérifier certaines informations sur les plateformes de médias sociaux telles que **Facebook**, **Twitter** et **LinkedIn**. Il peut vérifier les informations nécessaires, telles que qui a aimé votre statut et consulté votre profil.

❑ Divertissement

Netflix utilise MapReduce pour analyser les clics et les journaux des clients en ligne. Ces informations aident l'entreprise à promouvoir des films en fonction des intérêts et du comportement des clients.

Les Principaux Composants d'Apache Hadoop

☐ **Hadoop HDFS**

☐ **MapReduce**

☐ **Hadoop YARN**

Systèmes de Fichiers Distribués Hadoop (HDFS)

HDFS est un système de fichiers conçu pour stocker de fichiers très volumineux avec des modèles d'accès aux données en continu, s'exécutant sur des clusters de matériel de base. HDFS est un système de fichiers logiciel implémenté en Java et il se trouve au-dessus du système de fichiers natif. **Le concept principal derrière HDFS est qu'il divise un fichier en blocs (généralement 128 Mo) au lieu de traiter un fichier dans son ensemble, et les stocke dans un cluster de manière très efficace.** Cela permet de nombreuses fonctionnalités telles que la **distribution**, la **réplication**, la **reprise après incident** et, le **traitement distribué des blocs à l'aide de plusieurs machines**. La taille des blocs peut être de 64 Mo, 128 Mo, 256 Mo ou 512 Mo, selon ce qui convient. **Pour un fichier de 1 Go avec des blocs de 128 Mo, il y aura 1024 Mo/128 Mo soit huit blocs. Si nous considérons un facteur de réplication de trois, cela fait 24 blocs.** **HDFS fournit un système de stockage distribué avec tolérance aux pannes et reprise après incident.**

Systèmes de Fichiers Distribués Hadoop (HDFS)

Le HDFS a trois composants principaux qui sont:

- ❑ le **Name Node**
- ❑ le **Secondary Name node**
- ❑ les **Data Node**.

Les **réplications de blocs de données** fonctionnent selon deux règles :

- 1) Deux blocs identiques ne peuvent pas être placés sur le même DataNode
- 2) Lorsqu'un cluster est compatible avec le rack, toutes les répliques d'un bloc ne peuvent pas être placées sur le même rack.

Le Name node

Le **Name node** contient toutes les métadonnées de tout le contenu du système de fichiers : **les noms de fichiers, les autorisations de fichiers et l'emplacement de chaque bloc de chaque fichier**, et c'est donc **la machine la plus importante de HDFS**. Il gère toutes les opérations sur les fichiers comme la lecture, l'écriture, etc. **Il est nœud maître qui contrôle tous les Data Nodes**.

Le **Name node** est un service/démon centralisé qui agit comme un **gestionnaire de stockage en cluster**. Ses principales responsabilités sont :

- **maintenir les métadonnées des fichiers et des répertoires.**
- **contrôler et coordonner les Data Node pour les opérations du système de fichiers telles que la création, la lecture, l'écriture, etc.**

Les clients communiquent avec le **Name node** afin d'effectuer les opérations quotidiennes du système de fichiers. À son tour, le **Name node** donne aux clients l'emplacement des **Data Node** dans le cluster (où le bloc de données est disponible) pour effectuer des opérations. Le **Name node** procède à l'authentification s'il est configuré.

Les Data Node

Les **Data Node** sont les nœuds esclaves présents dans le **cluster Hadoop**. Les **Data Nodes** se connectent au **Name Node** et **stockent les blocs dans HDFS**. Ils s'appuient sur le **Name Node** pour toutes les informations de métadonnées concernant le contenu du système de fichiers. Si le **Name Node n'a aucune information**, le **Data Node** ne sera pas en mesure de fournir des informations à tout client qui souhaite lire/écrire sur le HDFS. Les **Data Node** stockent et gèrent les blocs HDFS sur le disque local. Les **Data Node** stockent les blocs de données et les récupère. Les **Data Node** rapportent également périodiquement les informations sur les blocs au **Name Node**. Autrement dit les **Data Node** signalent la santé et l'état des stockages des les blocs HDFS au **Name Node**.

Les Data Node (suite)

Le démon **Data Node** est responsable de la gestion des disques de stockage locaux. Il gère les opérations du système de fichiers telles que la création, la lecture, l'ouverture, la fermeture et la suppression de blocs. Le **Data Node** ne sait rien des blocs de données. Il stocke chaque bloc sous forme de fichier dans le système de fichiers local. Le **Data Node** ne stocke pas tous les blocs dans le même répertoire. Au lieu de cela, il utilise une heuristique pour déterminer le nombre optimal de blocs par répertoire et crée des sous-répertoires de manière appropriée. Les blocs sont chargés/supprimés dans les **Data Node** sur la base des instructions de **Name Node**, qui valide et traite les demandes des clients. **Name Node** n'effectue aucune opération de lecture/écriture pour les clients. Les clients communiquent avec **Name Node** pour connaître les emplacements des blocs et sont redirigés vers les **Data Node** pour effectuer une opération de lecture/écriture.

Le Secondary Name Node

Le **Secondary Name Node** est la sauvegarde du nœud de nom.

La responsabilité du **secondary name node** est de copier et de fusionner périodiquement l'image de l'espace de noms et de modifier le journal. En cas de panne du **Name Node**, l'image de l'espace de noms stockée dans le **secondary name node** peut être utilisée pour redémarrer le **Name node**.

Le **secondary name node** effectue des tâches d'entretien ménager et des points de contrôle au nom du **Name Node**. Malgré son nom, il ne s'agit pas d'un **Name Node** de secours.

Le **Name Node** conserve les métadonnées de l'ensemble du cluster HDFS. Si le **Name Node** est en panne, l'ensemble du cluster HDFS devient inaccessible en raison de l'indisponibilité des métadonnées. C'est ce qu'on appelle le point de défaillance unique (SPOF). Par conséquent, il est recommandé d'exécuter le **Name Node** sur un serveur d'entreprise dans un environnement de production. Cependant, la défaillance des serveurs est inévitable. Par conséquent, une copie des métadonnées du **Name Node** est conservée dans une autre machine, appelée **Secondary Name Node**. Une autre option consiste à utiliser NFS pour conserver les métadonnées.

Il est possible que les processus **Name Node** et **Data Node** soient exécutés sur une seule machine. Cependant, les clusters HDFS sont généralement constitués d'un serveur dédié exécutant le processus **Name Node** et de milliers de machines exécutant le processus **Data Node**. Afin de pouvoir accéder aux informations de contenu stockées dans le **NameNode**, il stocke l'intégralité de la structure des métadonnées en mémoire. Il garantit qu'il n'y a pas de perte de données à la suite de pannes de machine en gardant une trace du facteur de réplication des blocs. Puisqu'il s'agit d'un point de défaillance unique, pour réduire le risque de perte de données en raison de la défaillance d'un **Name Node**, un **Name Node** secondaire peut être utilisé pour générer des instantanés des structures de mémoire du NameNode principal.

Les **Data Nodes** ont de grandes capacités de stockage et, contrairement au **Name Node**, HDFS continuera à fonctionner normalement si un **Data Node** échoue. Lorsqu'un **Data Node** échoue, le **Name Node** prend automatiquement en charge la réplication désormais diminuée de tous les blocs de données dans le **Data Node** défaillant et s'assure que la réplication est reconstruite. Étant donné que le **Name Node** connaît tous les emplacements des blocs répliqués, tous les clients connectés au cluster peuvent procéder avec peu ou pas de problèmes.

Architecture HDFS

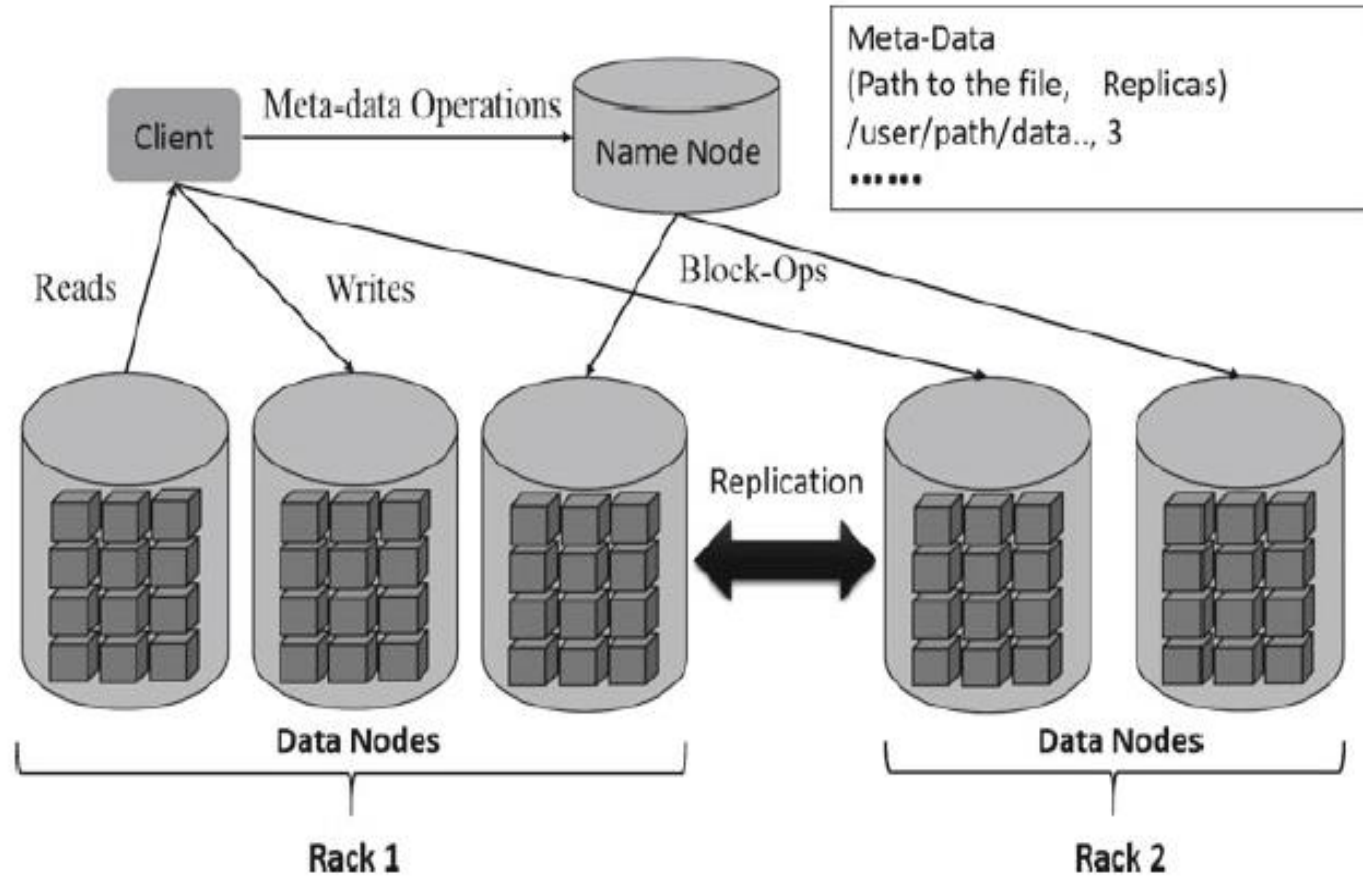


Figure 2 : Architecture des systèmes de fichiers distribués Hadoop

Gestion de données

Le **Name Node maître** garde une trace des blocs qui composent un fichier et de l'emplacement de ces blocs. Le **Name Node** communique avec les **Data Nodes**, les processus qui contiennent réellement les blocs dans le cluster. Les métadonnées associées à chaque fichier sont stockées dans la mémoire du **maître NameNode** pour des recherches rapides, et si le **Name Node** s'arrête ou tombe en panne, l'ensemble du cluster deviendra inaccessible.

Le **Name Node secondaire** n'est pas une sauvegarde du **Name Node**, mais effectue à la place des tâches d'entretien au nom du **Name Node**, y compris (et surtout) la fusion périodique d'un instantané de l'espace de données actuel avec le journal des modifications pour s'assurer que le journal des modifications n'obtient pas trop grande. Le journal de modification est utilisé pour assurer la cohérence des données et éviter la perte de données ; si le **Name Node** échoue, cet enregistrement fusionné peut être utilisé pour reconstruire l'état des **Data Nodes**.

Lorsqu'une application cliente veut accéder pour lire un fichier, elle demande d'abord les métadonnées du **Name Node** pour localiser les blocs qui composent le fichier, ainsi que les emplacements des **Data Nodes** qui stockent les blocs. L'application communique alors directement avec les **Data Nodes** pour lire les données. Par conséquent, le **Name Node** agit simplement comme un journal ou une table de recherche et n'est pas un goulot d'étranglement pour les lectures simultanées.

Le Cluster Hadoop

Le cluster Hadoop couvre un **Master node** et plusieurs **Slave Nodes**. Le **Master node** implique **Data Node**, **Name Node**, **JobTracker** et **TaskTracker** tous les éléments **slave node** ensemble, un **TaskTracker** et un **Datanode**, qui gère le calcul en premier et le nœud de travail de données uniquement, comme illustré à la **Figure 3.1**. Le Business Tracker gère le calendrier des activités. Hadoop contient deux fragments : **HDFS** et **MapReduce**. **HDFS** exécute le stockage des données et génère **MapReduce** pour fournir une exploration des données dans l'environnement collecté. La conception de l'architecture Hadoop est expliquée dans la **Figure 3.1**.

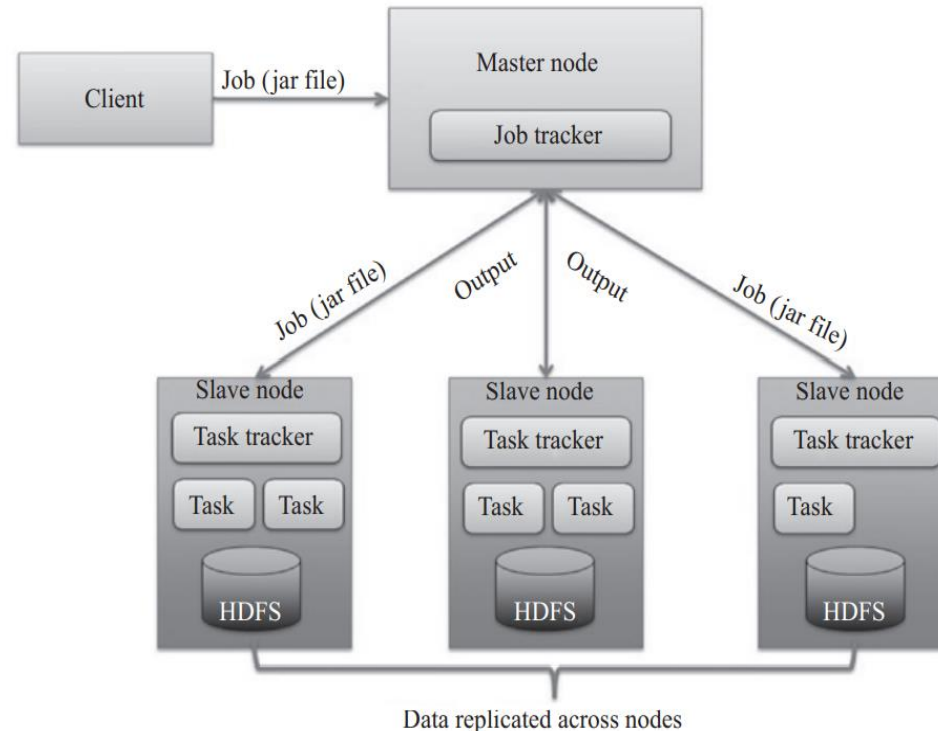


Figure 3.1 Hadoop architecture

Le Cluster Hadoop

Job Tracker

La responsabilité de Job Tracker est de planifier les travaux du client (jobs). Le suivi des travaux crée une carte et réduit les tâches et les planifie pour qu'elles s'exécutent sur les Data Nodes (suivi des tâches). Job Tracker vérifie également les tâches ayant échoué et replanifie les tâches ayant échoué sur un autre nœud de données. Le suivi des travaux peut être exécuté sur le Name Node ou sur un nœud séparé.

Le Cluster Hadoop

Task Tracker

Task tracker s'exécute sur les DataNodes. La responsabilité des Task Tracker est d'exécuter les tâches map ou reduce assignées par le NameNode et de signaler l'état des tâches au NameNode.

Outre les deux composants principaux ci-dessus, le projet Hadoop contient également les modules suivants.

1. **Hadoop Common** : utilitaires communs pour les autres modules Hadoop
2. **Hadoop Yarn** : un framework pour la planification des tâches et la gestion des ressources du cluster

Le Cluster Hadoop

Un data-center contient un ensemble de racks empilés avec un ensemble de serveurs (nœuds) formant un cluster avec un réseau local haut débit. Comme Hadoop est basé sur une architecture sans partage, chaque nœud a ses disques durs de stockage. Par conséquent, les technologies de stockage SAN/NAS/Direct Attached Storage (DAS) ne sont pas utilisées.

Le Cluster Hadoop

Une configuration Hadoop typique dans un cluster physique ressemblerait à celle illustrée à la **Figure 2.2**. HDFS et MR sont tous deux basés sur une architecture maître-esclave. NN, SNN et JT sont des composants maître/serveur. DN et TT sont des composants esclaves.

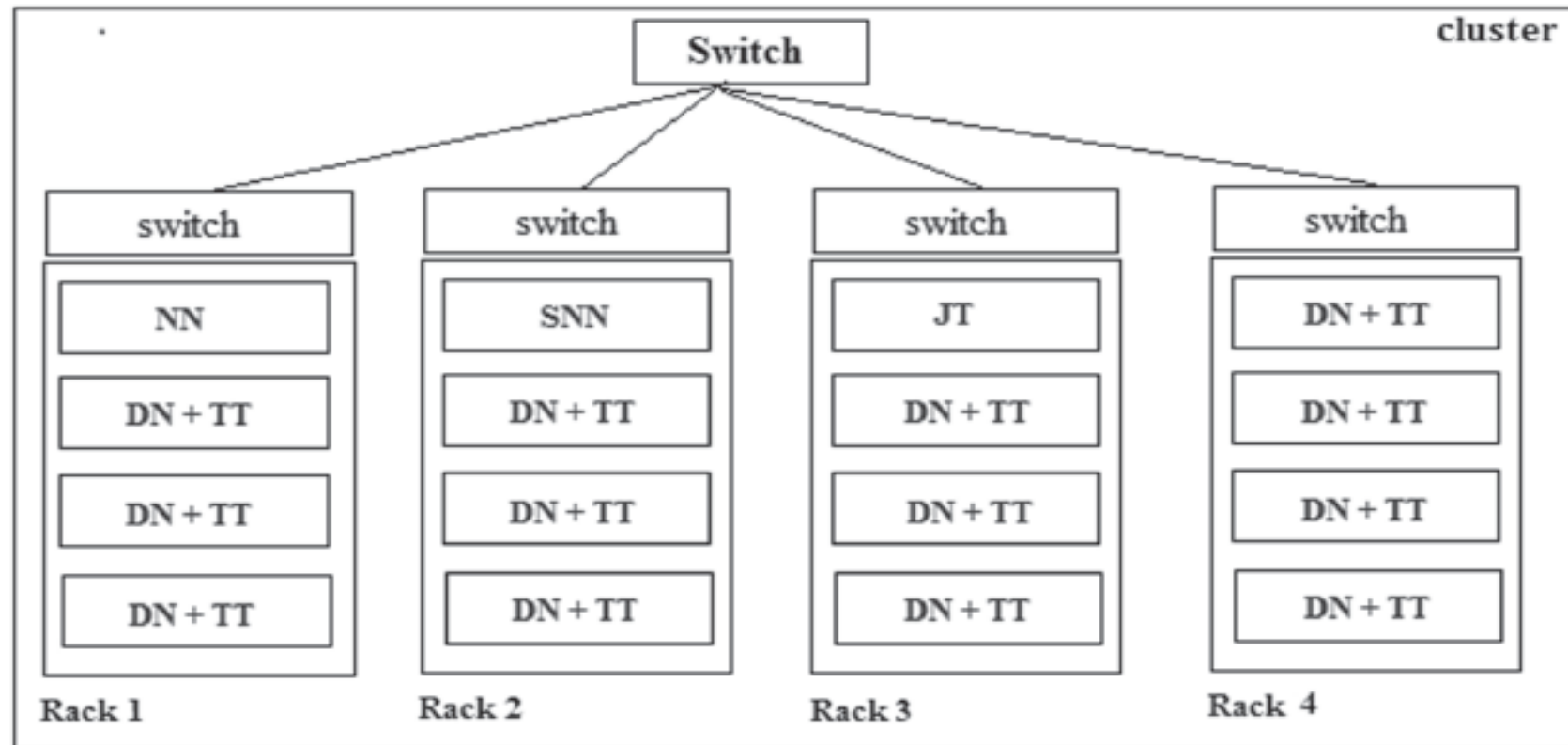


Figure 2.2 Hadoop cluster.

Le Cluster Hadoop

Dans la Figure 2.2, il y a 4 racks avec chacun 4 nœuds. Les composants Logiciel du serveur tels que NN, SNN et JT sont exécutés dans des nœuds séparés dans des racks différents. Les 13 nœuds restants sont appelés nœuds esclaves exécutant DN et TT ensemble.

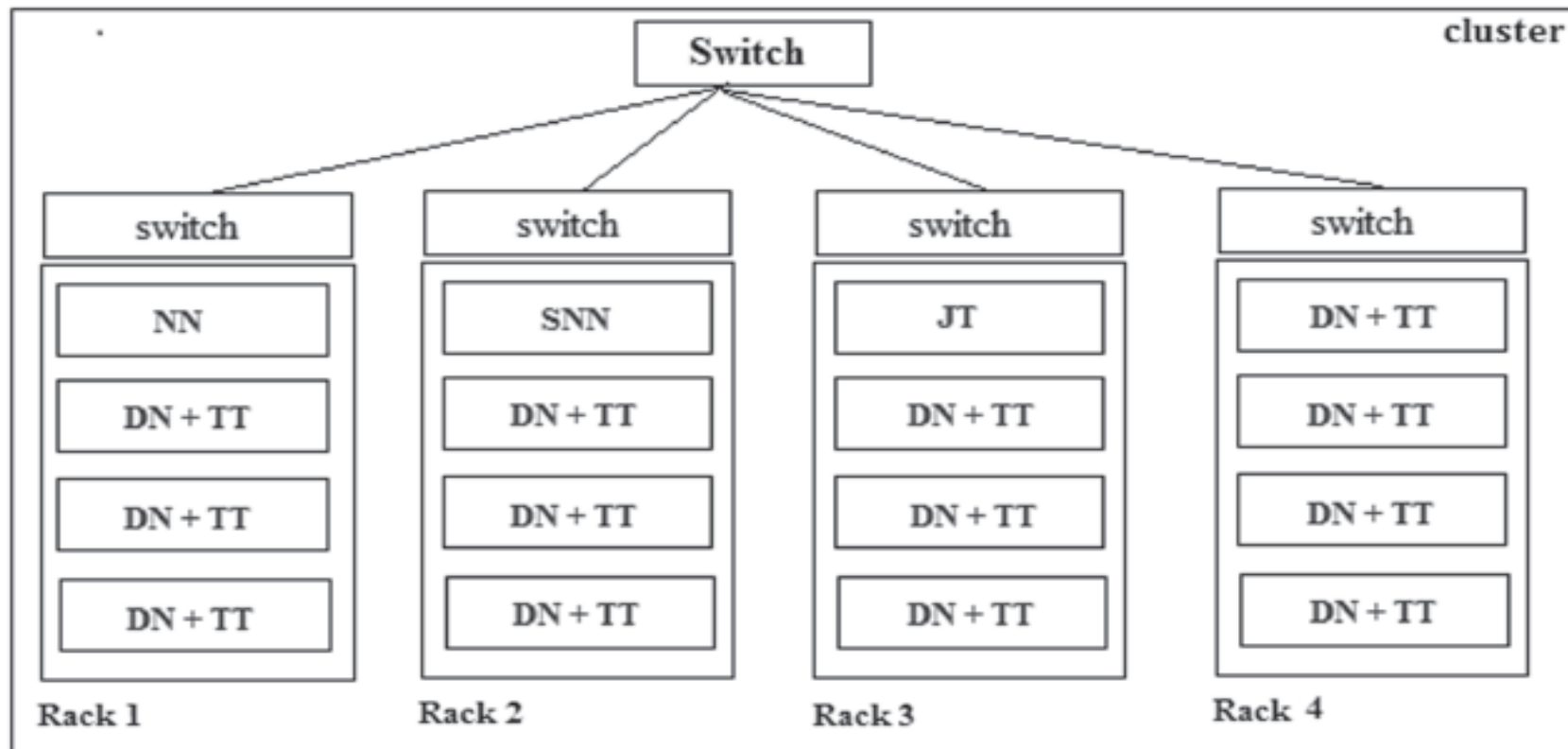


Figure 2.2 Hadoop cluster.

Communication des composants HDFS and MR

La Figure ci-contre montre **la communication entre les démons maîtres et esclave**. Les maîtres démons se parlent. Le Job Tracker (JT) communique avec le **Task Tracker** (TT) de même le **Name Node** (NN) parle avec le **Data Node** (DN) et le **Secondary Name Node** (SNN). Le TT parle au DN pour apporter le bloc de données pour le traitement. Dans un cluster Hadoop, un seul NN, SNN, JT et plusieurs nœuds esclaves peuvent exister. Chaque nœud esclave exécute à la fois DN et TT. Les démons maîtres (NN, JT, SNN) sont généralement exécutés sur un nœud dédié pour équilibrer la charge car NN et JT sont profondément communiqués par les esclaves. **Les ressources de chaque nœud esclave (CPU et mémoire) sont gérées par TT pour l'exécution des tâches. Les disques de stockage sont gérés par DN pour le stockage/la récupération des données.**

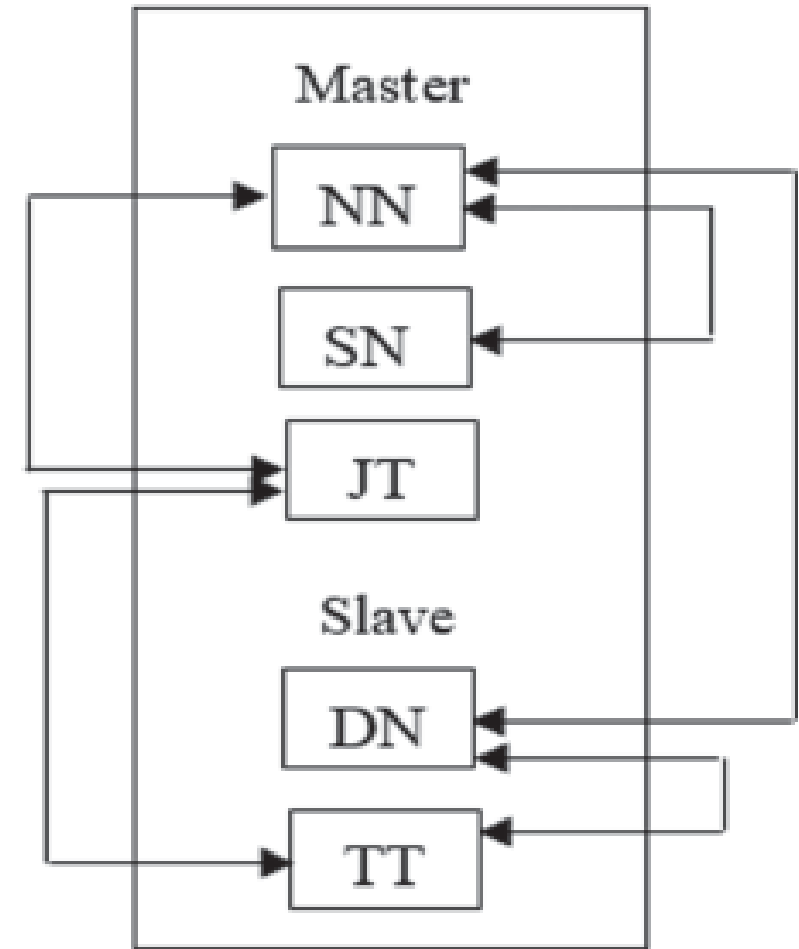


Figure 2.3 la Communication des composants HDFS and MR

Le Cluster Hadoop

Qu'est-ce qu'un cluster ?

Hadoop le nom du logiciel qui s'exécute sur un cluster, à savoir le système de fichiers distribué, HDFS, et le gestionnaire de ressources de cluster, YARN, qui sont collectivement composés de six types de services d'arrière-plan exécutés sur un groupe de machines.

HDFS et YARN exposent une interface de programmation d'application (API) qui soustrait les développeurs aux détails d'administration de cluster de bas niveau.

Un ensemble de machines exécutant HDFS et YARN est appelé cluster Hadoop et les machines individuelles sont appelées nœuds. Un cluster peut avoir un seul nœud ou plusieurs milliers de nœuds, mais tous les clusters évoluent horizontalement, ce qui signifie que plus vous ajoutez de nœuds, plus la capacité et les performances du cluster augmentent de manière linéaire.

Le Cluster Hadoop

YARN et **HDFS** sont implémentés par plusieurs processus démons, c'est-à-dire des logiciels qui s'exécutent en arrière-plan et ne nécessitent aucune intervention de l'utilisateur. Les processus Hadoop sont des services, ce qui signifie qu'ils s'exécutent en permanence sur un nœud de cluster et acceptent les entrées et délivrent les sorties via le réseau, de la même manière que fonctionne un serveur HTTP. Chacun de ces processus s'exécute à l'intérieur de sa propre machine virtuelle Java (JVM), de sorte que chaque démon dispose de sa propre allocation de ressources système et est géré indépendamment par le système d'exploitation.

Le Cluster Hadoop

Chaque nœud du cluster est identifié par le type de processus qu'il exécute :

Master nodes

Ces nœuds exécutent des services de coordination pour les workers Hadoop et sont généralement les points d'entrée pour l'accès des utilisateurs au cluster. Sans masters, la coordination s'effondrerait et le stockage ou le calcul distribué ne serait pas possible.

Worker nodes

Ces nœuds représentent la majorité des ordinateurs du cluster. Les worker nodes exécutent des services qui acceptent des tâches des master nodes, soit pour stocker ou récupérer des données, soit pour exécuter une application particulière. Un calcul distribué est exécuté en parallélisant l'analyse sur les worker nodes.

HDFS et YARN ont tous deux plusieurs services master responsables de la coordination des services de worker qui s'exécutent sur chaque worker node. Les worker node implémentent à la fois les services de worker de HDFS et YARN.

Pour HDFS, les services master et worker sont les suivants :

Name Node (Master)

Stocke l'arborescence de répertoires du système de fichiers, les métadonnées des fichiers et les emplacements de chaque fichier dans le cluster. Les clients souhaitant accéder à HDFS doivent d'abord localiser les nœuds de stockage appropriés en demandant des informations au Name Node.

Name Node secondaire (master)

Effectue des tâches d'entretien ménager et des points de contrôle au nom du Name Node.

Malgré son nom, il ne s'agit pas d'un Name Node de secours.

Data Node (worker)

Stocke et gère les blocs HDFS sur le disque local. Signale la santé et l'état des stockages de données individuels au Name Node.

Accès aux données à partir de HDFS

À un niveau élevé, lors de l'accès aux données à partir de HDFS, une application cliente doit d'abord adresser une demande au **Name Node** pour localiser les données sur le disque. Le **Name Node** répondra avec une liste de **Data Nodes** qui stockent les données, et le client doit alors demander directement chaque bloc de données au **Data Node**. Notez que le **Name Node** ne stocke pas de données, ni de transmettre les données de **Data Node** au client, il agit plutôt comme un agent de la circulation, pointant les clients vers les bons **Data Nodes**.

De même, YARN a plusieurs services maîtres et un service de travail comme suit :

Resource Manager (Master)

Alloue et surveille les ressources de cluster disponibles (par exemple, les actifs physiques tels que la mémoire et les cœurs de processeur) aux applications, ainsi que la gestion de la planification des tâches sur le cluster.

Application Master (Master)

Coordonne une application particulière en cours d'exécution sur le cluster comme prévu par le Resource Manager.

Node Manager (worker)

Exécute et gère les tâches de traitement sur un nœud individuel et signale la santé et l'état des tâches pendant leur exécution.

Semblable au fonctionnement de HDFS, les clients qui souhaitent exécuter une tâche (job) doivent d'abord demander des ressources au Resource Manager, qui attribue un Application Master spécifique à l'application pour la durée de la tâche (job). L'Application Master suit l'exécution de la tâche (job), tandis que le Resource Manager suit l'état des nœuds, et chaque Node Manager individuel crée des conteneurs et exécute des tâches (jobs) à l'intérieur de ceux-ci. Notez qu'il peut également y avoir d'autres processus en cours d'exécution sur le cluster Hadoop, par exemple, des serveurs JobHistory ou des coordinateurs ZooKeeper, mais ces services sont les principaux logiciels exécutés dans un cluster Hadoop.

Map Reduce(MR)

Apache Hadoop est l'un des frameworks les plus populaires pour l'analyse du Big Data sur un cluster informatique. Hadoop est l'implémentation open source du modèle de programmation MapReduce. Dans le modèle de programmation MapReduce, le processus de calcul est organisé en deux phases : Map et Reduce.

Dans la phase Map, le nœud maître divise l'ensemble de données d'entrée en sous-problèmes indépendants et les distribue aux nœuds esclaves. Ensuite, les nœuds esclaves traitent en parallèle les problèmes les plus petits et renvoient la réponse à leur nœud maître. Enfin, dans la phase de reduce, le nœud maître prend les réponses à tous les sous-problèmes et les combine de manière à former le résultat. Les utilisateurs de ce paradigme n'ont qu'à définir ce qui doit être calculé dans les fonctions Map et Reduce tandis que le système distribue automatiquement le traitement des données sur un cluster hautement distribué de machines.

Map Reduce(MR)

Dans le modèle MapReduce, tous les calculs sont organisés autour de paires (clé, valeur). Dans la première étape, la fonction Map prend une seule paire (clé, valeur) en entrée et produit une liste de paires intermédiaires (clé ; valeur) en sortie. On pourrait le représenter ainsi :

$$\text{map}(\text{key1}, \text{value1}) \rightarrow \text{list}(\text{key2}, \text{value2})$$

Ensuite, le système fusionne et regroupe par clés ces paires intermédiaires et les transmet à la fonction Reduce. Enfin, la fonction Reduce prend en entrée une clé et une liste de valeurs associée et génère une nouvelle liste de valeurs en sortie, qui peut être représentée comme suit :

$$\text{reduce}(\text{key2}, \text{list}(\text{value2})) \rightarrow (\text{key2}, \text{value3})$$

Le Mécanisme du Modèle MapReduce

Un calcul MapReduce s'exécute comme suit :

- 1.** Un certain nombre de tâches Map reçoivent chacune un ou plusieurs morceaux d'un système de fichiers distribué. Ces tâches Map transforment le bloc en une séquence de paires clé-valeur. **La façon dont les paires clé-valeur sont produites à partir des données d'entrée est déterminée par le code écrit par l'utilisateur pour la fonction Map.**
- 2.** Les paires clé-valeur de chaque tâche Map sont collectées par un contrôleur maître et triées par clé. **Les clés sont réparties entre toutes les tâches Reduce, de sorte que toutes les paires clé-valeur avec la même clé se retrouvent dans la même tâche Reduce.**
- 3.** Les **tâches Reduce** fonctionnent sur une clé à la fois et combinent toutes les valeurs associées à cette clé d'une manière ou d'une autre. **Le mode de combinaison des valeurs est déterminé par le code écrit par l'utilisateur pour la fonction Reduce.**

Le Mécanisme du Modèle MapReduce

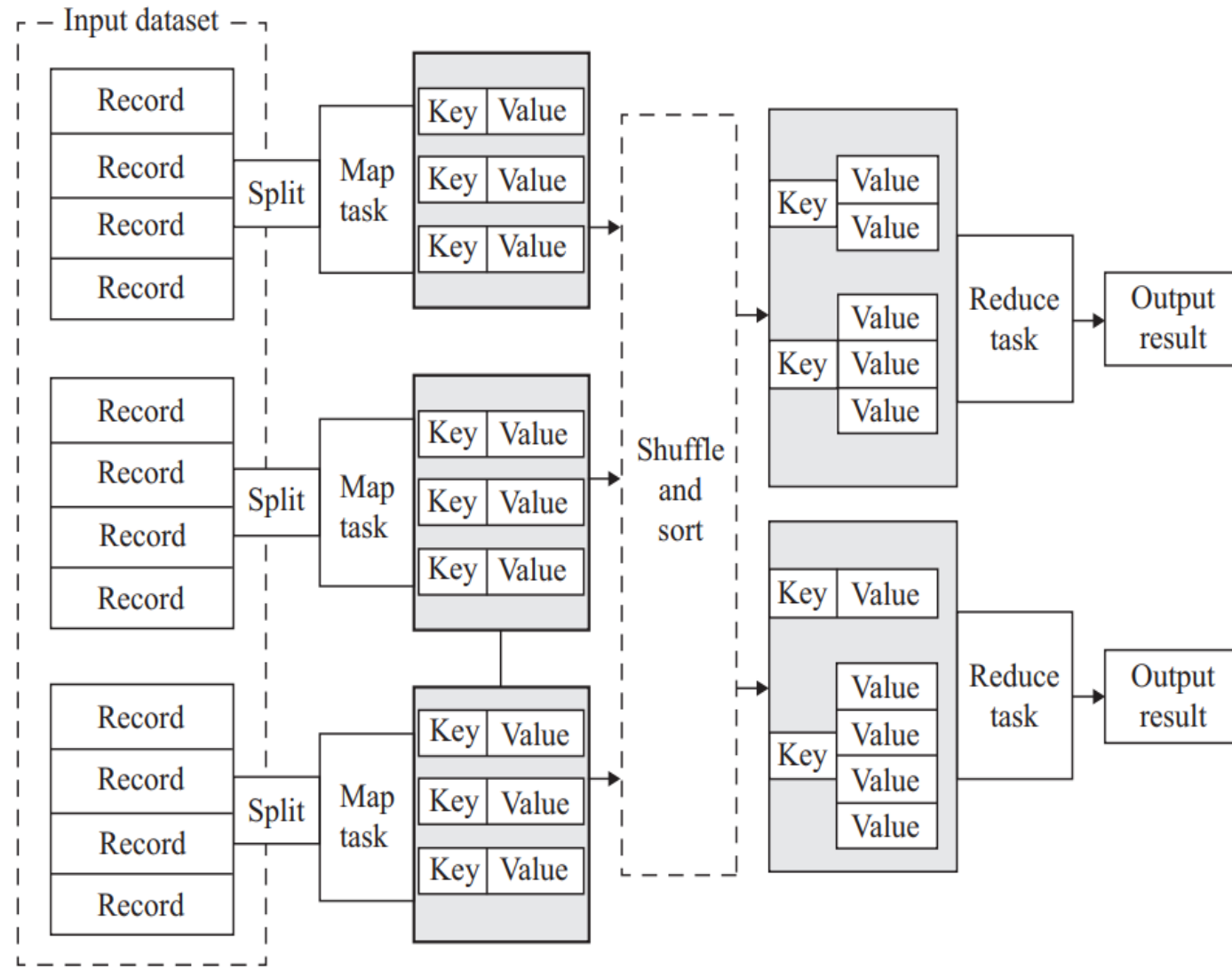
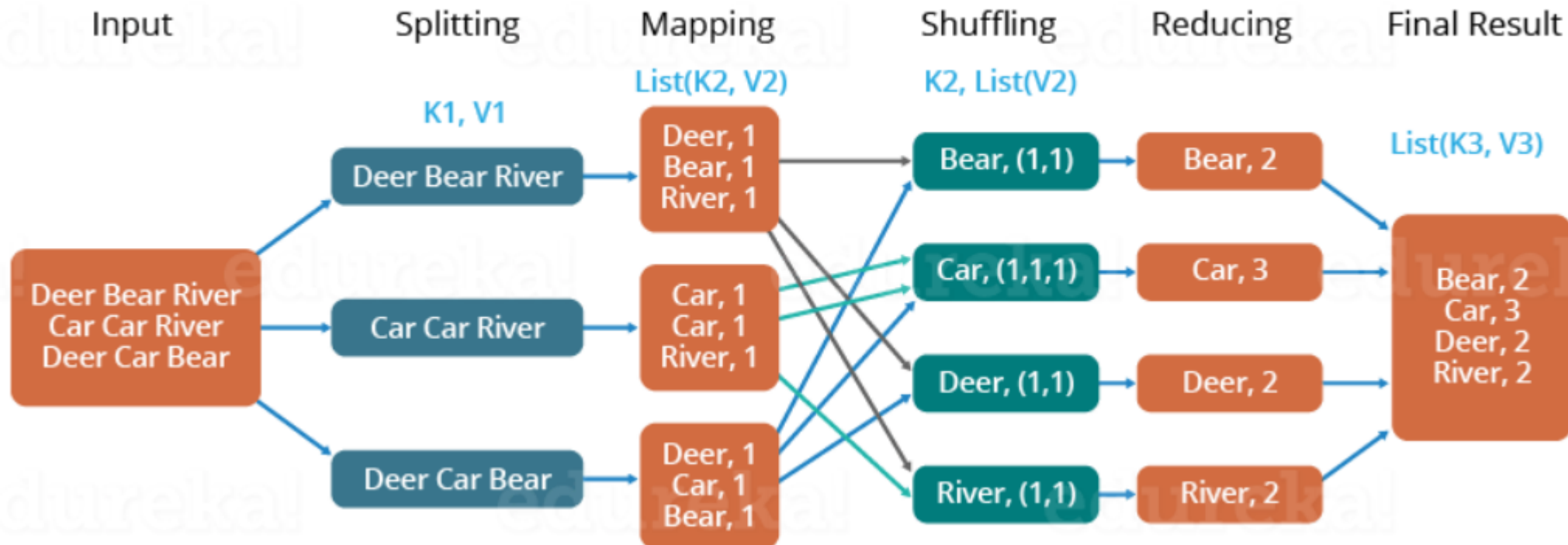


Figure 3.2 MapReduce Mode

Les Phases de MapReduce

La figure suivante illustre MapReduce avec trois catégories principales. La séparation est généralement incluse dans la phase de mappage.

The Overall MapReduce Word Count Process



Map Reduce(MR)

Map Reduce est un outil de **traitement par lots** (batch processing) **hautement distribué, évolutif horizontalement, tolérant aux pannes** et **parallèle** de données dans le framework Hadoop qui s'exécute sur un cluster de serveurs de base peu fiables pour traiter le Big Data. Les données sont collectées et stockées sur HDFS avant de lancer les travaux MR.

Il est facile de développer des algorithmes évolutifs à l'aide de Map Reduce sans effort supplémentaire pour gérer un système distribué. Un programme développé pour un nœud peut être utilisé pour des milliers de nœuds sans re-modifier le code. MR lui-même parallélise l'exécution, de sorte que les utilisateurs n'ont pas besoin d'investir d'efforts pour l'exécution parallèle.

MapReduce(MR)

La programmation MR est basée sur LISP (LIST Programming), un type de paradigme de programmation fonctionnelle (comme Haskell, Scala, Clojure, Smalltalk, ruby). Tout dans la programmation fonctionnelle tourne autour de la fonction. Une fonction peut être passée/reçue en tant qu'arguments, répartis sur les nœuds. Une fonction prenant une autre fonction comme argument est appelée fonction d'ordre supérieur. La programmation fonctionnelle ne conserve pas l'état et ne prend pas en charge le blocage et la synchronisation. Par conséquent, il est évolutif. Les tâches MR s'exécutent indépendamment, il est donc facile de gérer les échecs partiels.

MapReduce(MR)

Toutes les applications ne sont pas adaptées à la mise en œuvre avec MR. Seules les applications nécessitant des fonctions telles que le comptage, le classement, l'agrégation, la recherche, le tri et le regroupement peuvent être réalisées avec MR. MR n'a pas de requêtes prédéfinies comme OLAP pour DWH. Il s'agit d'écrire des algorithmes ad-hoc pour traiter l'ensemble des données. Les performances du MR sont inférieures à celles des langages de programmation généraux lors du traitement d'ensembles de données de quelques 100 Mo, car le MR consiste en une séquence d'étapes à effectuer. Cependant, pour expérimenter la véritable puissance de la MR, il faut travailler avec un ensemble de données en To, car c'est là que le SGBDR prend des heures et échoue, alors qu'Hadoop fait de même en quelques minutes. Comme nous l'avons déjà vu, MR a deux sous-composants (voir Figure 2.12) : JT et TT. Une tâche MR typique exécute deux tâches : mapper et réduire.

MAP AND REDUCE TASK

Pour développer un travail MR pour un algorithme, il faut écrire des fonctions map et reduce. Certaines parties d'un algorithme sont incluses dans la fonction map et d'autres dans la fonction reduce. Pour y parvenir, nous devons avoir une bonne compréhension de la séquence d'exécution MR pour décider quelle partie de l'algorithme doit être incluse dans les fonctions de map et reduce. On peut mettre en œuvre des logiques telles que filtrage, projection, transformation dans la fonction map. De même, une logique telle que l'agrégation, la jointure, le tri peut être implémentée dans la fonction reduce. Lorsqu'une fonction prend une entrée et produit une sortie, elle est appelée tâche. Lorsqu'une fonction map ou reduce prend une entrée et produit une sortie, elles sont appelées tâches : tâche de map (map task) et tâche de réduire (reduce task).

MAP AND REDUCE TASK

Ces tâches sont exécutées dans des slots dédiés dans TT. En général, la tâche map est appelée **mappeur** et la tâche reduce est appelée **réducteur**. Un nœud qui exécute une tâche map est appelé nœud de map et un nœud qui exécute une tâche reduce est appelé nœud de reduce. Les fonctions map et reduce lisent/écrivent de données à partir d'entrées/sorties standard. Cela permet à tout langage (Java, C++, Ruby, Python, etc.) qui prend en charge les entrées/sorties standard d'écrire des programmes MR.

Un job MR typique se compose d'un ensemble de tâches map et d'un ensemble de tâches reduce. Lorsqu'un job est lancé, toutes les tâches map sont d'abord exécutées, puis toutes les tâches reduce sont exécutées. Les tâches map n'ont pas de dépendance entre elles ; de même, les tâches reduce n'ont pas non plus de dépendance entre elles. Par conséquent, ces tâches sont appelées « bag of tasks (sac de tâches) », qui prennent en charge un énorme parallélisme avec une exécution simultanée. Ceci est également appelé exécution parallèle embarrassante (ou parfaitement ou agréablement).

MAP AND REDUCE TASK

MR est hautement asynchrone car il n'y a pas de communication, de blocage et de synchronisation entre les tâches map et les tâches reduce . Cependant, il existe un point de synchronisation, où les tâches reduce ne seront lancées qu'une fois toutes les tâches map terminées. Ainsi, les tâches map reçoivent une priorité élevée par rapport aux tâches reduce, car elles doivent être terminées avant que le réducteur puisse démarrer. Les tâches map sont planifiées pour atteindre la localité des données dans l'étendue. Il peut également y avoir des exécutions non locales. Les tâches reduce n'ont pas l'avantage de la localisation des données. JT décide de manière dynamique où exécuter les tâches de réduction en fonction de la disponibilité des emplacements et de la charge du réseau.

MapReduce(MR)

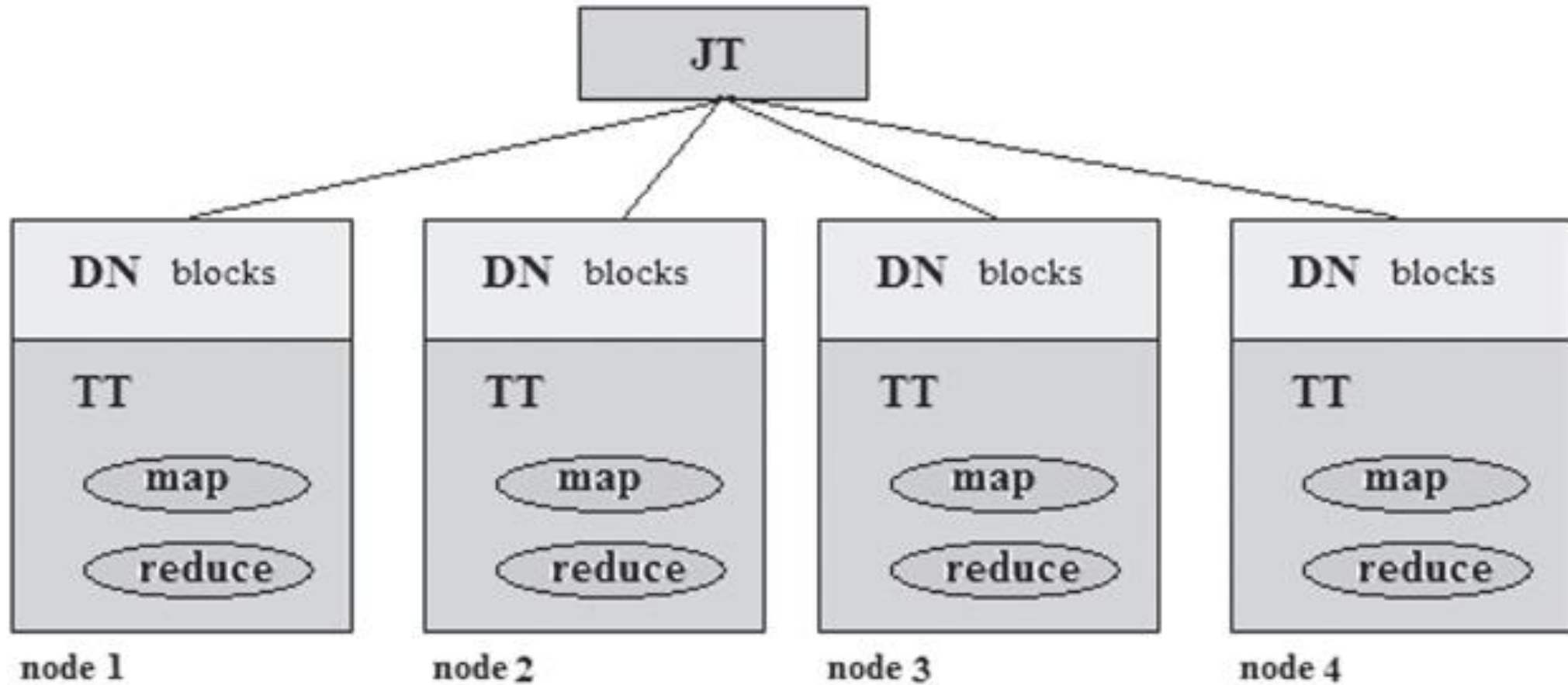


FIGURE 2.12 MapReduce components

Les Phases de MapReduce

En général, il y a deux phases pour l'exécution d'un job MapReduce, comme le montre la **Figure 2.14 : la phase de map et la phase reduce.**

- **La phase map** exécute un ensemble de tâches map (mappers) pour lire les données des disques sous forme de paires clé-valeur et **produire un nombre arbitraire de paires clé-valeur intermédiaires en fonction de la fonction map définie par l'utilisateur.**
- **La phase reduce** exécute un ensemble de tâches reduce (reducers), qui collectent la sortie de toutes les tâches map, fusionne, trie en fonction de la clé, regroupe la liste des valeurs qui appartiennent à la même clé et produit la sortie finale en fonction de la réduction définie par l'utilisateur fonction.

Exemple de Phases de MapReduce

Comme le montre la Figure 2.14, dans la phase map, deux tâches map s'exécutent dans **node 1** et une tâche map s'exécute dans **node 2** et **node 3**. En phase reduce, une tâche reduce s'exécute dans **node 4** et **node 5**. Toutes les tâches map et de reduce sont exécutées dans n'importe quel nœud du cluster.

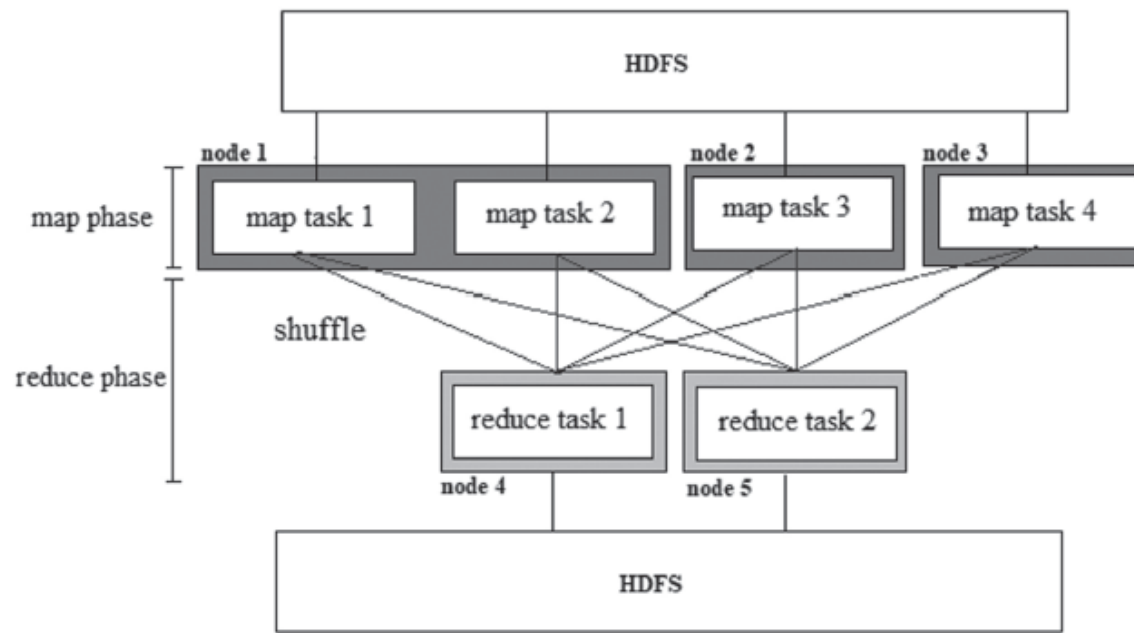


Figure 2.14 les phases de MapReduce

L'Architecture de MapReduce

L'architecture de MapReduce se compose de divers composants. Une brève description de ces sections peut améliorer notre compréhension de son fonctionnement.

- **Job**: il s'agit d'un travail réel qui doit être fait ou traité
- **Task** : il s'agit d'un travail réel qui doit être effectué ou traité. La tâche MapReduce couvre de nombreuses petites tâches qui doivent être effectuées.
- **Job Tracker** : ce suivi joue un rôle dans l'organisation des tâches et le suivi de toutes les tâches affectées à un suivi des tâches.
- **Task Tracker** : ce suivi joue le rôle de suivre l'activité et de signaler l'état de l'activité au suivi des tâches.

L'Architecture de MapReduce

- **Input data:** elles sont utilisées pour le traitement dans la phase de cartographie.
- **Exit data:** C'est le résultat de la cartographie et de l'atténuation.
- **Client :** il s'agit d'un programme ou d'une interface de programmation d'application (API) qui envoie des tâches à MapReduce. Il peut accepter les services de plusieurs clients.
- **Hadoop MapReduce Master :** Celui-ci joue le rôle de diviser les tâches en sections.
- **Job Parts :** Ce sont de petites tâches qui aboutissent à la division de la fonction principale.

L'Architecture de MapReduce

Dans l'architecture MapReduce, les clients soumettent des tâches à MapReduce Master. Ce gestionnaire divisera ensuite le travail en plus petites parties égales. Les composants de la fonction seront utilisés pour deux tâches principales dans Map Reduce : le Map et le Reduce.

Le développeur rédigera un concept qui répond aux besoins de l'organisation ou de l'entreprise.

Les données d'entrée seront catégorisées et cartographiées.

Les données centrales seront ensuite filtrées et fusionnées. Le curseur qui produira le dernier stocké sur HDFS traitera la sortie.

Le diagramme suivant montre un organigramme simplifié du programme MapReduce.



Les Phases de MapReduce

Le programme MapReduce comprend trois étapes principales : la phase map, la phase de mélange et la phase reduce. Il existe également une catégorie facultative connue sous le nom de phase de fusion.

Phase Map

Il s'agit de la première phase du programme. Il y a deux étapes dans cette phase : la classification et la mappage. La base de données est divisée en unités égales appelées unités (divisions d'entrée) dans l'étape de division. **Hadoop contient un RecordReader qui utilise TextInputFormat pour convertir les variables d'entrée en paires de mots clés.** Les paires clé-valeur sont ensuite utilisées comme entrée sur l'étape de mappage. **C'est le seul format de données qu'un éditeur de map peut lire ou comprendre.** L'étape de mappage contient la logique du code utilisé dans ces blocs de données. Dans cette étape, le map analyse les paires de clés et génère une sortie de la même forme (paires clé-valeur).

Les Phases de MapReduce

Phase de Melange

Il s'agit de la deuxième phase qui se produit après l'achèvement de la phase de mappage. Il se compose de deux étapes principales : **le filtrage et la fusion**. Dans l'étape de filtrage, les mots-clés sont filtrés à l'aide de clés et la combinaison garantit que les paires clé-valeur sont incluses.

La phase **de vol à l'étagère (The shuffling phase)** facilite la suppression des valeurs en double et la collecte des valeurs. **Différentes valeurs avec les mêmes clés sont combinées**. La sortie de cette catégorie sera des clés et des valeurs, comme dans la section de mappage.

Les Phases de MapReduce

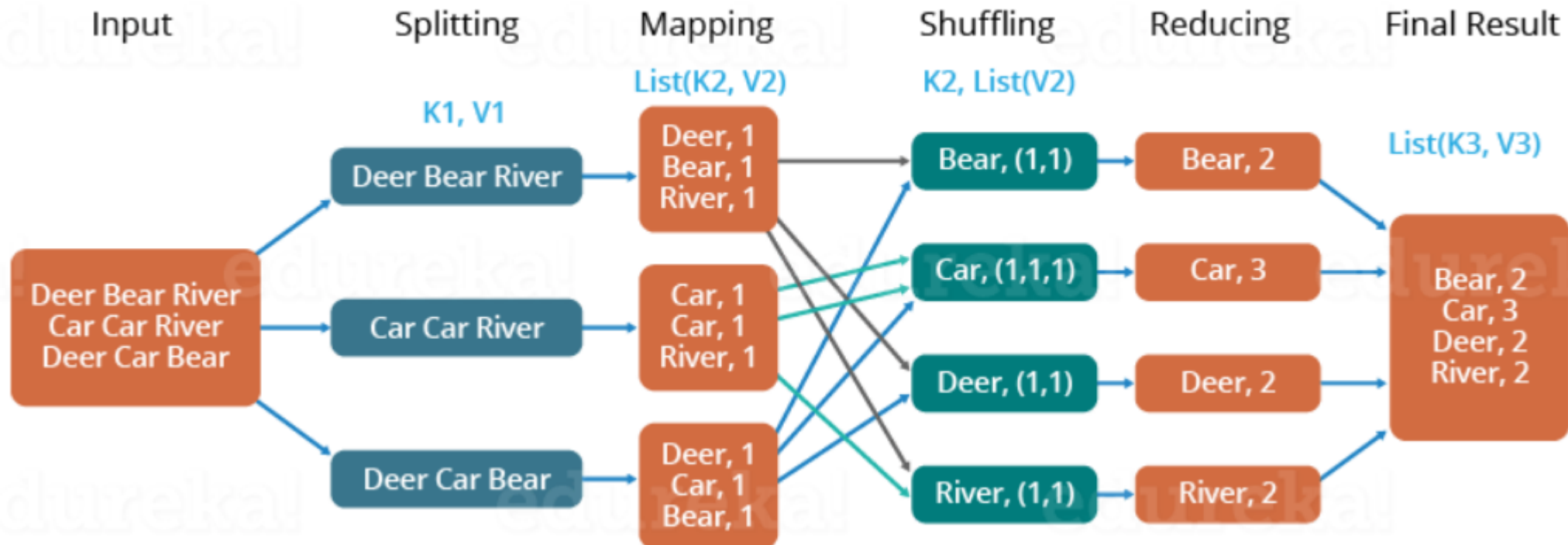
Phase Reduce

Dans la phase de reduce, la sortie de la phase de push est l'entrée de l'utilisateur. Le reduceur traite en continu ces entrées pour rassembler les valeurs médianes en plus petites. Fournit un résumé de l'ensemble de la base de données. La sortie de cette catégorie est stockée dans HDFS.

Les Phases de MapReduce

La figure suivante illustre MapReduce avec trois catégories principales. La séparation est généralement incluse dans la phase de mappage.

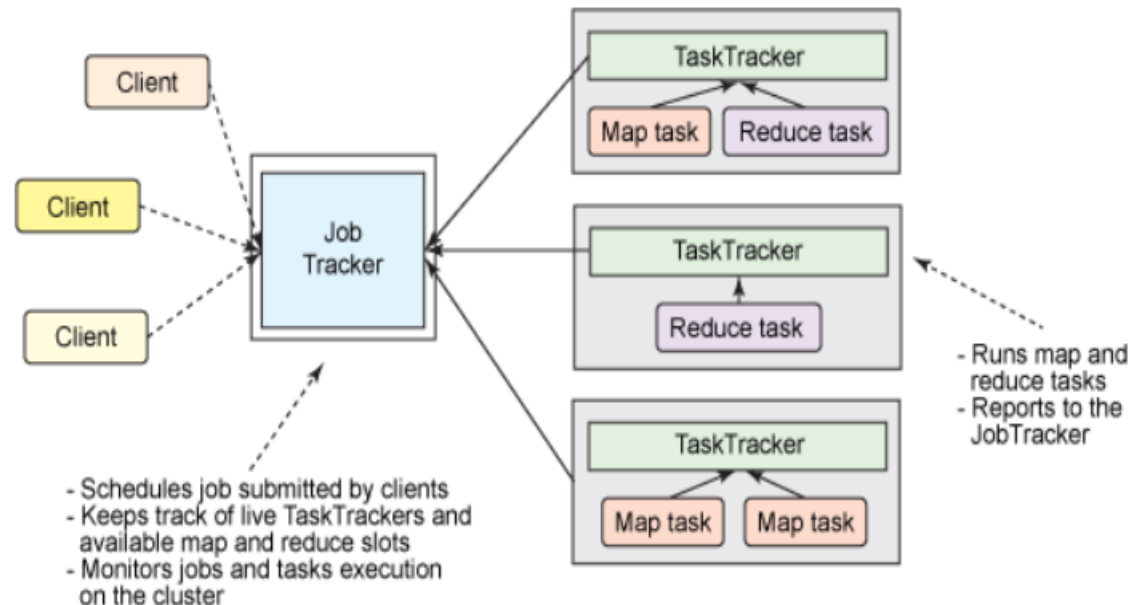
The Overall MapReduce Word Count Process



How do Job Trackers work?

Chaque tâche se compose de deux parties essentielles : les fonctions Map et Reduce. Le rôle de la fonction Map consiste à diviser les tâches en segments de tâches et en données de mapping centrales, et la fonction Reduce joue le rôle de mélanger et de rassembler les données centrales en unités plus petites.

Le tracker d'activité fonctionne comme un maître. Cela garantit que nous faisons tout le travail. Le suivi d'activité répertorie les tâches publiées par les clients et fournit des job trackers pour les travaux. Chaque task tracker a une fonction map et minimise les tâches. Les trackers report (suivis d'activité) signalent l'état de chaque tâche affectée au task tracker. Le schéma suivant résume le fonctionnement le job tracker et task trackers.



JOB TRACKER (JT)

Job Tracker est un démon s'exécutant sur un serveur dédié dans le cluster pour gérer les ressources du cluster et les travaux MR. Les principales responsabilités de Job Tracker sont :

- gérer et surveiller les ressources (CPU et mémoire) dans les Task Tracker.
- préparation du plan d'exécution et coordination des phases.
- planifier les tâches mapper/réduire à Task Tracker.
- gestion du cycle de vie des travaux (job) (depuis leur soumission jusqu'à leur achèvement).
- conserver l'historique des tâches (statistiques au niveau des tâches).
- fournir une tolérance aux pannes.

JT est également conscient du rack (rack-aware) lors de la planification des tâches de mappage/réduction. La plupart du temps, le traitement a lieu au niveau des nœuds où le bloc de données requis est physiquement disponible pour réduire le trafic réseau. C'est ce qu'on appelle la localité/gravité des données. JT souffre également de SPOF.

TASK TRACKER (TT)

TT est un démon qui s'exécute dans chaque nœud esclave du cluster Hadoop. Il gère les ressources locales (CPU et mémoire) comme un slot. Un emplacement (également appelé conteneur) est un pack logique fixe d'une partie de la mémoire et du processeur pour exécuter la tâche de mappage/réduction, comme le montre la Figure 2.13.

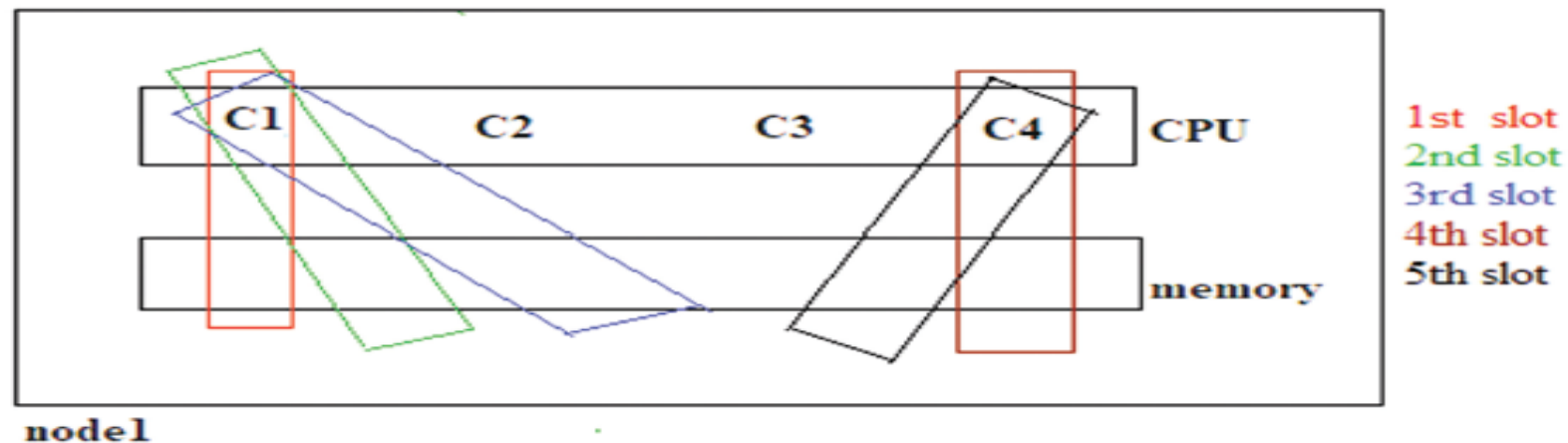


Figure 2.13 Slot

TASK TRACKER (TT)

Un CPU/processeur dans un nœud peut contenir plusieurs cœurs physiques ou cœurs logiques (C1, C2, etc.). Par défaut, un emplacement est formé avec 1 cœur logique et 1 Go de mémoire. Il peut y avoir de nombreux emplacements possibles dans un TT, et le nombre d'emplacements est déterminé en fonction de la taille du processeur et de la mémoire disponible. Dans MRv1, le nombre d'emplacements de mappage et de réduction dans un TT est fixe. Par exemple, s'il y a quatre emplacements dans un TT, nous pouvons attribuer trois emplacements pour l'exécution de la tâche de carte et un emplacement pour l'exécution de la tâche réduite. Cela peut être ajusté en fonction de nos besoins. Cependant, une restriction nécessaire est que la tâche de carte ne peut pas être affectée à l'emplacement de réduction et la tâche de réduction ne peut pas être affectée à l'emplacement de carte. JT surveille la disponibilité des créneaux pour attribuer les tâches appropriées dans TT.

TASK TRACKER (TT)

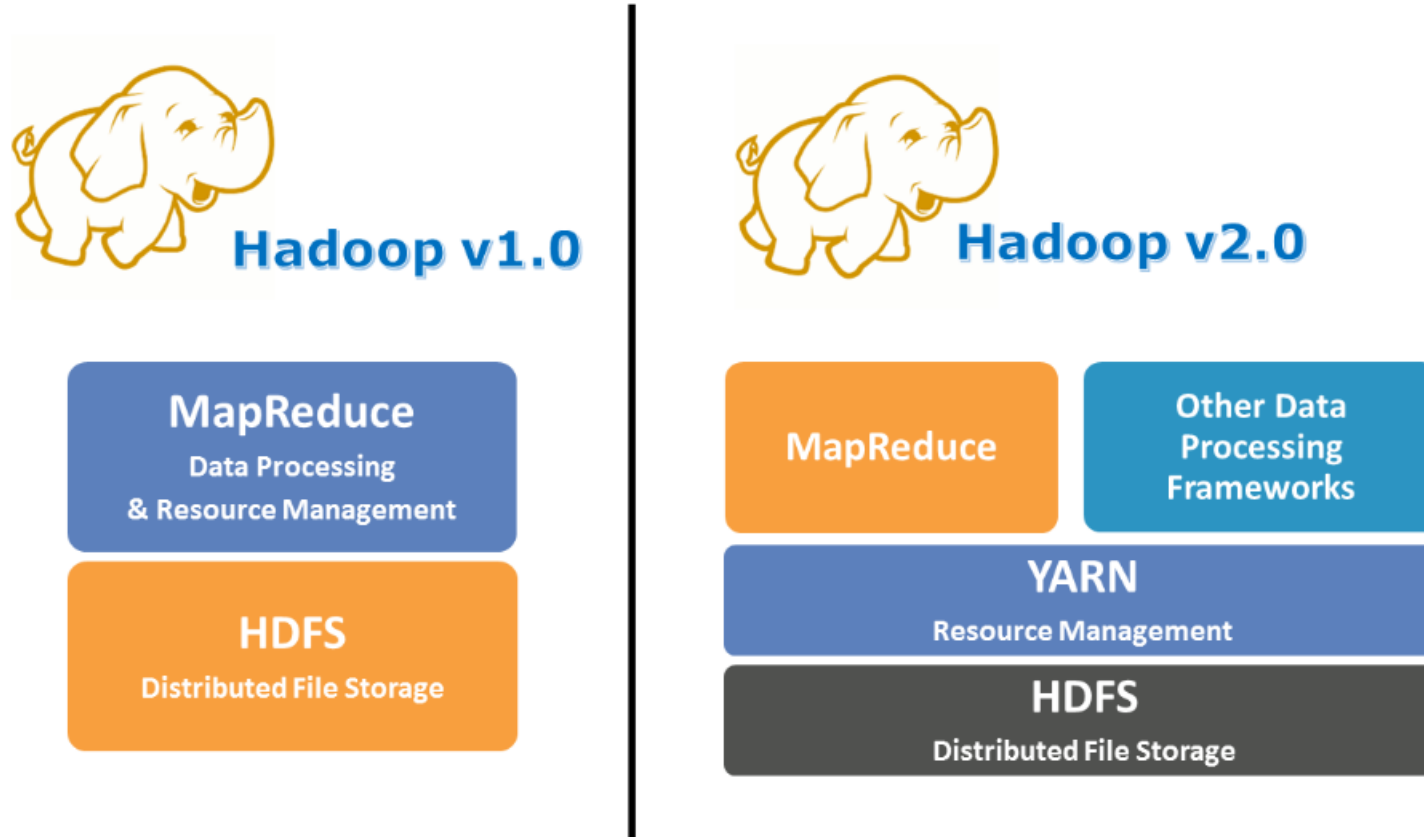
TT est responsable du démarrage et de la gestion des tâches de carte/réduction individuelles dans ses emplacements dédiés. TT envoie périodiquement un rapport à JT sur la disponibilité des ressources, la progression de la tâche et son état. TT envoie HB à JT pour indiquer qu'il est prêt à être utilisé. Au fur et à mesure que vous ajoutez plus de nœuds dans le cluster Hadoop, le nombre d'emplacements de tâches augmente linéairement et davantage de blocs de données peuvent être traités en parallèle. Par conséquent, le nombre de blocs traités par unité de temps (débit) augmente linéairement.

Hadoop YARN

YARN est un gestionnaire de ressources et planification de tâches. YARN exécute toutes les activités de traitement en allouant des ressources et en planifiant des tâches. L'architecture Apache Hadoop YARN se compose des composants principaux suivants :

- 1. Resource Manager** : s'exécute sur un démon maître et gère l'allocation des ressources dans le cluster.
- 2. Node Manager** : ils s'exécutent sur les démons esclaves et sont responsables de l'exécution d'une tâche sur chaque nœud de données.
- 3. Application Master** : gère le cycle de vie des tâches utilisateur et les besoins en ressources des applications individuelles. Il fonctionne avec le gestionnaire de nœuds et surveille l'exécution des tâches.
- 4. Conteneur** : ensemble de ressources comprenant la RAM, le processeur, le réseau, le disque dur, etc. sur un seul nœud.

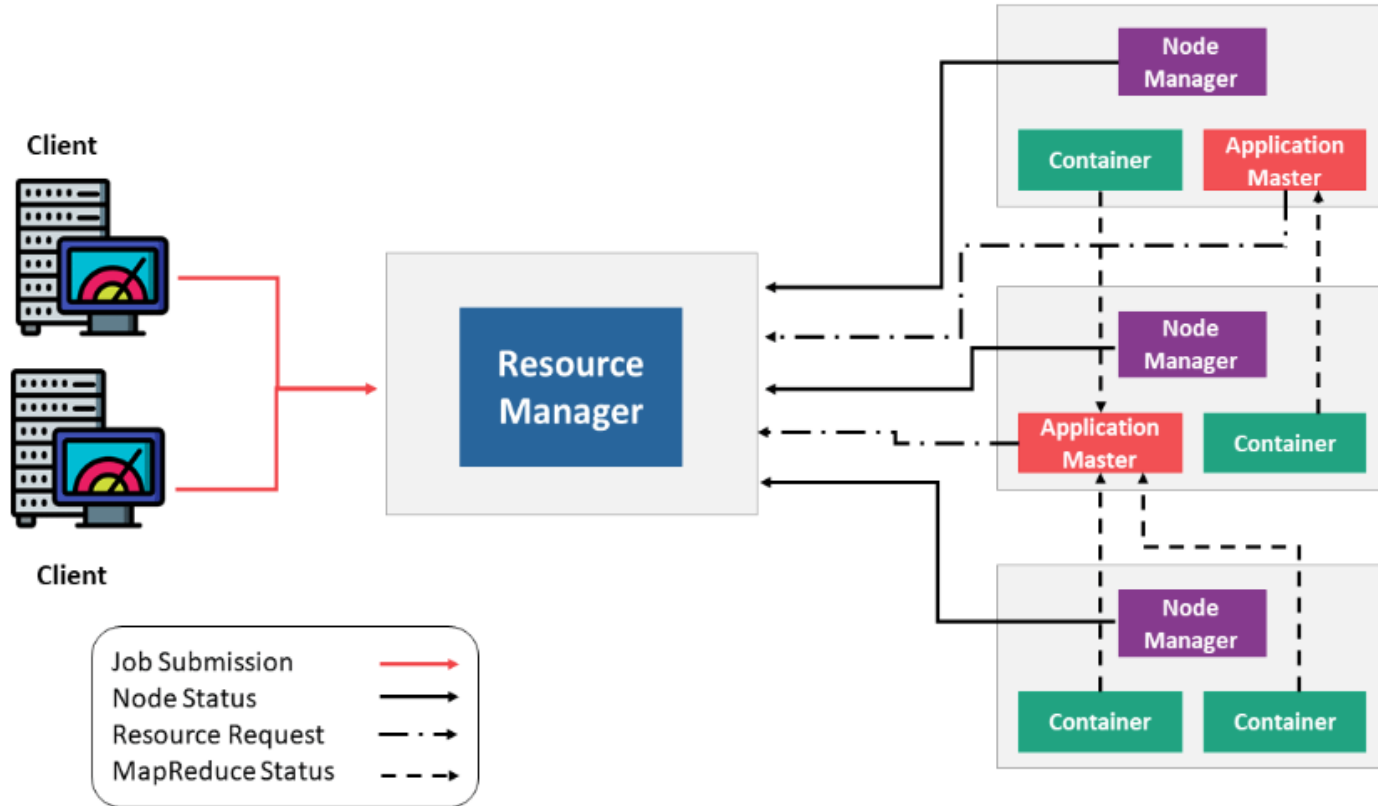
Hadoop YARN



YARN a permis aux utilisateurs d'effectuer des opérations selon les besoins en utilisant une variété d'outils tels que Spark pour le traitement en temps réel, Hive pour SQL, HBase pour NoSQL et autres.

Les Composants de Hadoop YARN

YARN peut être considéré comme le cerveau de l'écosystème Hadoop. L'image ci-dessous représente l'architecture YARN.



Les Composants de Hadoop YARN

1. Resource Manager

C'est l'autorité ultime en matière d'allocation des ressources.

- Lors de la réception des demandes de traitement, il transmet en conséquence des parties de demandes aux gestionnaires de nœuds correspondants, où le traitement réel a lieu.
- Il est l'arbitre des ressources du cluster et décide de l'allocation des ressources disponibles pour les applications concurrentes.
- Optimise l'utilisation du cluster en gardant toutes les ressources utilisées en permanence contre diverses contraintes telles que les garanties de capacité, l'équité et les SLA(Service Level Agreement).
- Il comporte deux composants principaux : a) Planificateur b) Gestionnaire d'applications

Les Composants de Hadoop YARN

a) Planificateur

- Le planificateur (l'ordonnanceur) est responsable de l'allocation des ressources aux différentes applications en cours d'exécution sous réserve de contraintes de capacités, de files d'attente, etc.
- Il s'agit d'un planificateur (ordonnanceur) pur dans Resource Manager, ce qui signifie qu'il n'effectue aucune surveillance ou suivi de l'état des applications.
- En cas de défaillance d'une application ou d'une défaillance matérielle, le planificateur ne garantit pas le redémarrage des tâches ayant échoué.
- Effectue la planification en fonction des besoins en ressources des applications.
- Il dispose d'un plug-in de politique enfichable, qui est chargé de partitionner les ressources du cluster entre les différentes applications. Il existe deux plug-ins de ce type : Capacity Scheduler et Fair Scheduler, qui sont actuellement utilisés comme planificateurs dans ResourceManager.

Les Composants de Hadoop YARN

b) Application Manager

- Il est chargé d'accepter les soumissions de tâches (job submissions)
- Négocie le premier conteneur du Resource Manager pour l'exécution de l'Application Master spécifique à l'application.
- Gère l'exécution des Application Master dans un cluster et fournit un service pour redémarrer le conteneur du maître d'application en cas d'échec.

Les Composants de Hadoop YARN

2. Node Manager

- Il prend en charge les nœuds individuels d'un cluster Hadoop et gère les tâches et le flux de travail des utilisateurs sur le nœud donné.
- Il s'enregistre auprès du Resource Manager et envoie des pulsations (heartbeats) avec l'état de santé du nœud.
- Son objectif principal est de gérer les conteneurs d'applications qui lui sont assignés par le Resource Manager.
- Il tient à jour avec le Resource Manager.
- L' Application Master demande le conteneur attribué au Node Manager en lui envoyant un contexte de lancement de conteneur (CLC) qui comprend tout ce dont l'application a besoin pour s'exécuter. Le gestionnaire de nœuds crée le processus de conteneur demandé et le démarre.
- Surveille l'utilisation des ressources (mémoire, CPU) des conteneurs individuels.
- Effectue la gestion des journaux.
- Il tue également le conteneur comme indiqué par le gestionnaire de ressources.

Les Composants de Hadoop YARN

3. Application Master

- Une application est une simple tâche soumise au framework. Chacune de ces applications est associée à un Application Master unique, qui est une entité spécifique au framework.
- C'est le processus qui coordonne l'exécution d'une application dans le cluster et gère également les pannes.
- Sa tâche est de négocier les ressources auprès du Resource Manager et de travailler avec le Node Manager pour exécuter et surveiller les tâches des composants.
- Il est responsable de la négociation des conteneurs de ressources appropriés à partir du Resource Manager, du suivi de leur statut et du suivi des progrès.
- Une fois démarré, il envoie périodiquement des messages de battements de cœur au Resource Manager pour confirmer son état de santé et mettre à jour l'enregistrement de ses demandes de ressources.

Les Composants de Hadoop YARN

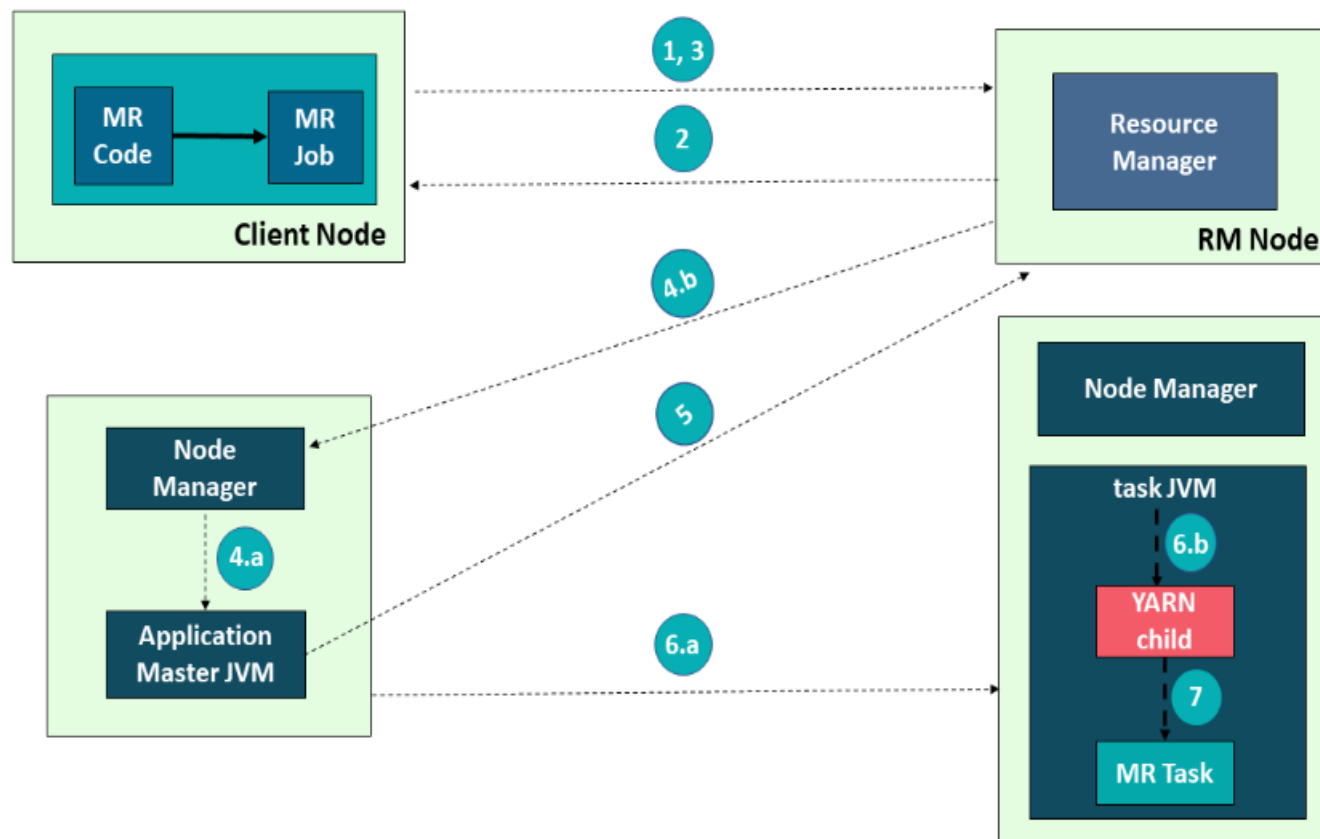
4. Conteneur

- Il s'agit d'un ensemble de ressources physiques telles que la RAM, les cœurs de processeur et les disques sur un seul nœud.
- Les conteneurs YARN sont gérés par un contexte de lancement de conteneur qui est le cycle de vie du conteneur (CLC). Cet enregistrement contient une carte des variables d'environnement, des dépendances stockées dans un stockage accessible à distance, des jetons de sécurité, une charge utile pour les services Node Manager et la commande nécessaire pour créer le processus.
- Il accorde le droit à une application d'utiliser une quantité spécifique de ressources (mémoire, CPU, etc.) sur un hôte spécifique.

Application Submission in YARN

Les étapes impliquées dans la soumission de l'application de Hadoop YARN sont énumérées ci-dessous:

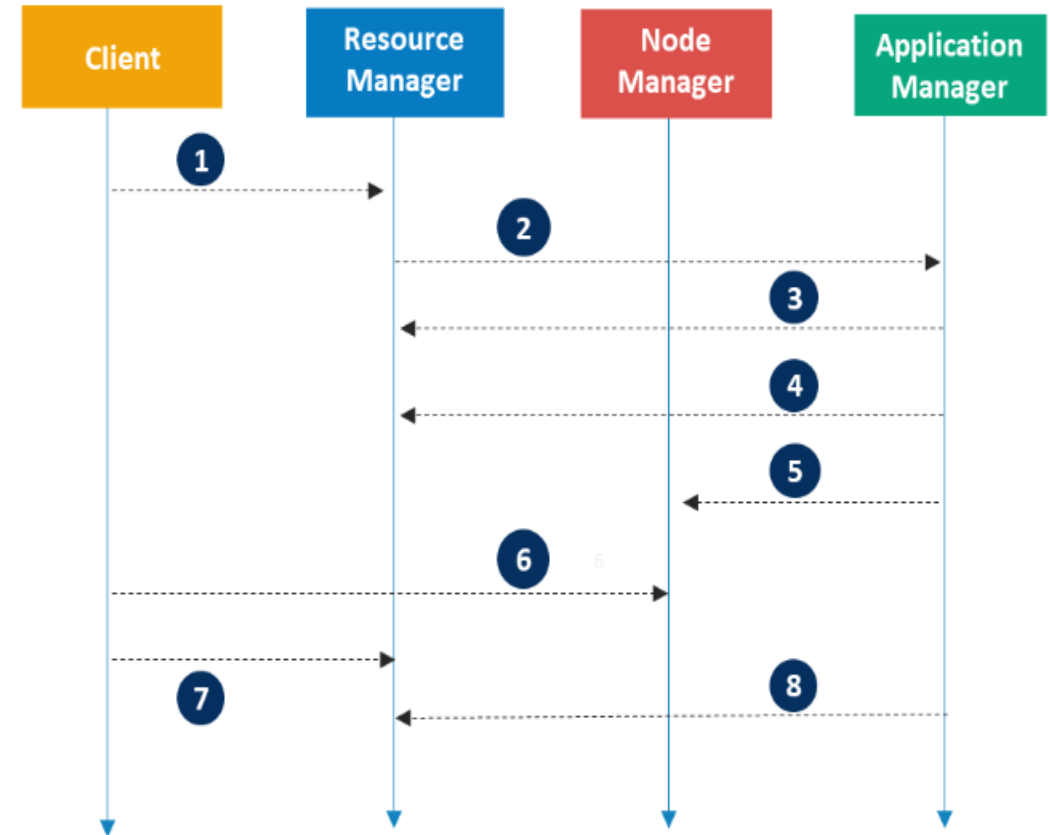
- 1) Submit the job
- 2) Get Application ID
- 3) Application Submission Context
- 4 a) Start Container Launch
- b) Launch Application Master
- 5) Allocate Resources
- 6 a) Container
- b) Launch
- 7) Execute



Application Workflow in Hadoop YARN

Les étapes suivantes impliquées dans le workflow d'application d'Apache Hadoop YARN :

1. Client submits an application
2. Resource Manager allocates a container to start Application Manager
3. Application Manager registers with Resource Manager
4. Application Manager asks containers from Resource Manager
5. Application Manager notifies Node Manager to launch containers
6. Application code is executed in the container
7. Client contacts Resource Manager/Application Manager to monitor application's status
8. Application Manager unregisters with Resource Manager



Apache Spark

Apache Spark est développé par le laboratoire UC Berkeley AMP, et c'est une source ouverte. Il est plus puissant en termes de fonctionnalités que **Hadoop** et **MapReduce** [17]. Il utilise Resilient Distributed Dataset 2 qui est de nature immuable et qui est une collection d'objets distribués. Il y a deux variables principales dans le Spark, qui sont la variable de diffusion et les accumulateurs. En raison de l'évaluation différée, les actions sont évaluées et les transformations sont stockées pour une exécution ultérieure. **Apache Hadoop et Spark traitent l'énorme quantité de données sur la base de la technique MapReduce, qui utilise des algorithmes d'exploration de données et d'apprentissage automatique dans différents domaines. Spark Mlib est également conçu sur la base d'algorithmes d'apprentissage automatique. Présentation, traitement, stockage, analyse des données peut être fait dans Mlib. Apache Spark est bien meilleur que les autres en termes de performances, d'analyse et de polyvalence.** Certains problèmes tels que l'adéquation architecturale, le développement des compétences, l'adoption et la mesure des avantages commerciaux doivent encore être résolus dans Spark, car cela semble très prometteur.

Volume

- ❑ La croissance exponentielle du stockage de données car les données sont désormais plus que des données textuelles.
- ❑ Les données peuvent être trouvées sous forme de **vidéos**, de **musique** et de **grandes images** sur nos réseaux sociaux.
- ❑ Il est très courant d'avoir des **téraoctets** et des **pétaoctets** de système de stockage pour les entreprises.
- ❑ Au fur et à mesure que la base de données se développe, les applications et l'architecture conçues pour prendre en charge les données doivent être réévaluées assez souvent.
- ❑ Parfois, les mêmes données sont réévaluées sous plusieurs angles et même si les données d'origine sont les mêmes, la nouvelle intelligence trouvée crée une explosion des données.
- ❑ Le gros volume représente en effet le Big Data.

Vélocité

- ❑ La croissance des données et l'explosion des médias sociaux ont changé notre façon de considérer les données.
- ❑ Le fait est que les journaux suivent toujours cette logique.
- ❑ Cependant, les chaînes d'information et les radios ont changé la rapidité avec laquelle nous recevons les informations.
- ❑ Aujourd'hui, les gens répondent sur les réseaux sociaux pour les tenir au courant des derniers événements. Sur les réseaux sociaux, parfois, des messages vieux de quelques secondes (un tweet, des mises à jour de statut, etc.) n'intéressent pas les utilisateurs.
- ❑ Ils ignorent souvent les anciens messages et prêtent attention aux mises à jour récentes. Le mouvement des données se fait désormais presque en temps réel et la fenêtre de mise à jour est réduite à des fractions de secondes.
- ❑ Ces données à grande vitesse représentent le Big Data.

Variété

- ❑ Les données peuvent être stockées dans plusieurs formats. Par exemple base de données, **excel**, **csv**, **access** ou **d'ailleurs**, il peut être stocké dans un simple **fichier texte**.
- ❑ **Parfois, les données ne sont même pas au format traditionnel** comme nous le supposons, elles peuvent être sous forme **de vidéo**, SMS, pdf ou quelque chose auquel nous n'y avons peut-être pas pensé. C'est la nécessité pour l'organisation de l'organiser et de lui donner un sens.
- ❑ Cela sera facile à faire si nous avons des données dans le même format, cependant ce n'est pas le cas la plupart du temps. Le monde réel dispose de données dans de nombreux formats différents et c'est le défi que nous devons relever avec le Big Data. Cette variété de données représente le Big Data.

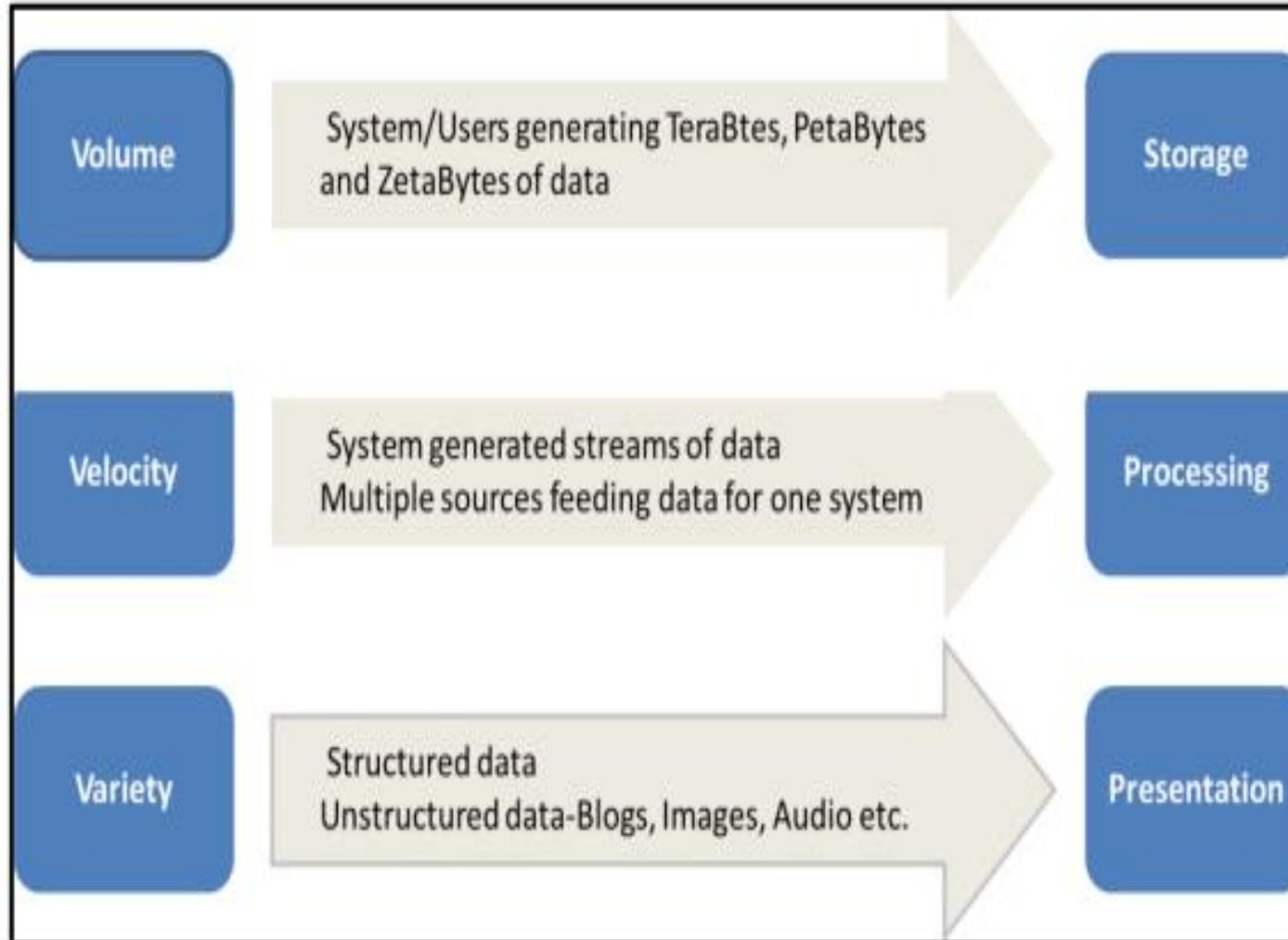


Figure : Volume, vélocité, variété

Apache Hadoop

- ❑ Apache Hadoop est l'un des principaux éléments de support des technologies Big Data. Il simplifie le traitement d'une grande quantité de données structurées ou non structurées à moindre coût.
- ❑ Hadoop est un projet open source d'Apache qui s'améliore continuellement au fil des années.
- ❑ "Hadoop est essentiellement un ensemble de bibliothèques et de frameworks logiciels permettant de gérer et de traiter de grandes quantités de données depuis un seul serveur vers des milliers de machines.
- ❑ Il fournit un mécanisme de détection d'erreurs efficace et puissant basé sur la couche application plutôt que sur le matériel.
- ❑ En décembre 2012, Apache publie Hadoop 1.0.0, plus d'informations et un guide d'installation peuvent être trouvés dans la documentation Apache Hadoop. Hadoop n'est pas un projet unique mais inclut un certain nombre d'autres technologies.

MapReduce

- ❑ **MapReduce** a été introduit par Google pour créer une grande quantité d'index de recherche sur le Web.
- ❑ Il s'agit essentiellement d'un framework pour écrire des applications qui traitent une grande quantité de données structurées ou non structurées sur le Web.
- ❑ MapReduce prend la requête et la divise en parties pour l'exécuter sur plusieurs nœuds.
- ❑ Grâce au traitement distribué des requêtes, il est facile de conserver une grande quantité de données en divisant les données en plusieurs machines différentes.
- ❑ **Hadoop MapReduce est un framework logiciel permettant d'écrire facilement des applications permettant de gérer une grande quantité d'ensembles de données avec une grande tolérance aux pannes.**
- ❑ D'autres didacticiels et un guide de démarrage sont disponibles dans la documentation Apache.

HDFS (système de fichiers distribué Hadoop)

- ❑ HDFS est un système de fichiers basé sur Java utilisé pour stocker des données structurées ou non structurées sur de grands clusters de serveurs distribués.
- ❑ Les données stockées dans HDFS n'ont aucune restriction ni règle à appliquer, les données peuvent être soit totalement non structurées, soit purement structurées.
- ❑ Dans HDFS, le travail visant à rendre les données significatives est effectué uniquement par le code du développeur.
- ❑ Le système de fichiers distribué Hadoop offre une atmosphère hautement tolérante aux pannes avec un déploiement sur des machines matérielles à faible coût.
- ❑ HDFS fait désormais partie du projet Apache Hadoop, plus d'informations et un guide d'installation peuvent être trouvés dans la documentation Apache HDFS.

Hive

- ❑ Hive a été initialement développé par Facebook, il est maintenant rendu open source depuis un certain temps.
- ❑ Hive fonctionne comme un pont entre SQL et Hadoop, il est essentiellement utilisé pour effectuer des requêtes SQL sur les clusters Hadoop.
- ❑ Apache Hive **est essentiellement un entrepôt de données** qui fournit des requêtes ad hoc, une synthèse des données et une analyse d'énormes ensembles de données stockés dans des systèmes de fichiers compatibles Hadoop.
- ❑ Hive fournit une implémentation basée sur des requêtes de type SQL appelée HiveQL d'une énorme quantité de données stockées dans des clusters Hadoop. En janvier 2013, Apache publie Hive 0.10.0. Pour plus d'informations et un guide d'installation, consultez la documentation Apache Hive.

Pig

- ❑ Pig a été introduit par Yahoo et plus tard, il a été rendu entièrement open source.
- ❑ Il fournit également un pont pour interroger les données sur les clusters Hadoop, mais contrairement à Hive, il implémente une implémentation de script pour permettre aux développeurs et aux hommes d'affaires d'accéder aux données Hadoop.
- ❑ Apache pig fournit une plate-forme de programmation de haut niveau permettant aux développeurs de traiter et d'analyser le Big Data à l'aide de fonctions et d'efforts de programmation définis par l'utilisateur. En janvier 2013, Apache a publié Pig 0.10.1, défini pour être utilisé avec Hadoop 0.10.1 ou des versions ultérieures. Plus d'informations et un guide d'installation sont disponibles dans la documentation de démarrage d'Apache Pig.

Approche classique Vs Approche Big Data

1. Bases de données sans schéma et orientées colonnes (No SQL)

- i. Nous utilisons des bases de données relationnelles basées sur des tables et des lignes au fil des ans, ces bases de données conviennent parfaitement aux transactions en ligne et aux mises à jour rapides.
- ii. Lorsque des données non structurées et volumineuses entrent en jeu, nous avons besoin de certaines bases de données qui ne repose pas principalement sur un schéma.
- iii. Il existe un certain nombre de bases de données à intégrer dans cette catégorie, ces bases de données peuvent stocker des données non structurées, semi-structurées ou même entièrement structurées.
- iv. Outre d'autres avantages, la meilleure des bases de données sans schéma est qu'elles rendent la migration des données très facile.
- v. MongoDB est une base de données NoSQL très populaire et largement utilisée de nos jours.
- vi. Les bases de données NoSQL et sans schéma sont utilisées lorsque la préoccupation principale est de stocker une énorme quantité de données et non de maintenir des relations entre les éléments.
- vii. "NoSQL (Not only SQL) est un type de base de données qui ne repose pas principalement sur une structure basée sur un schéma et n'utilise pas SQL pour le traitement des données."

Approche classique Vs Approche Big Data

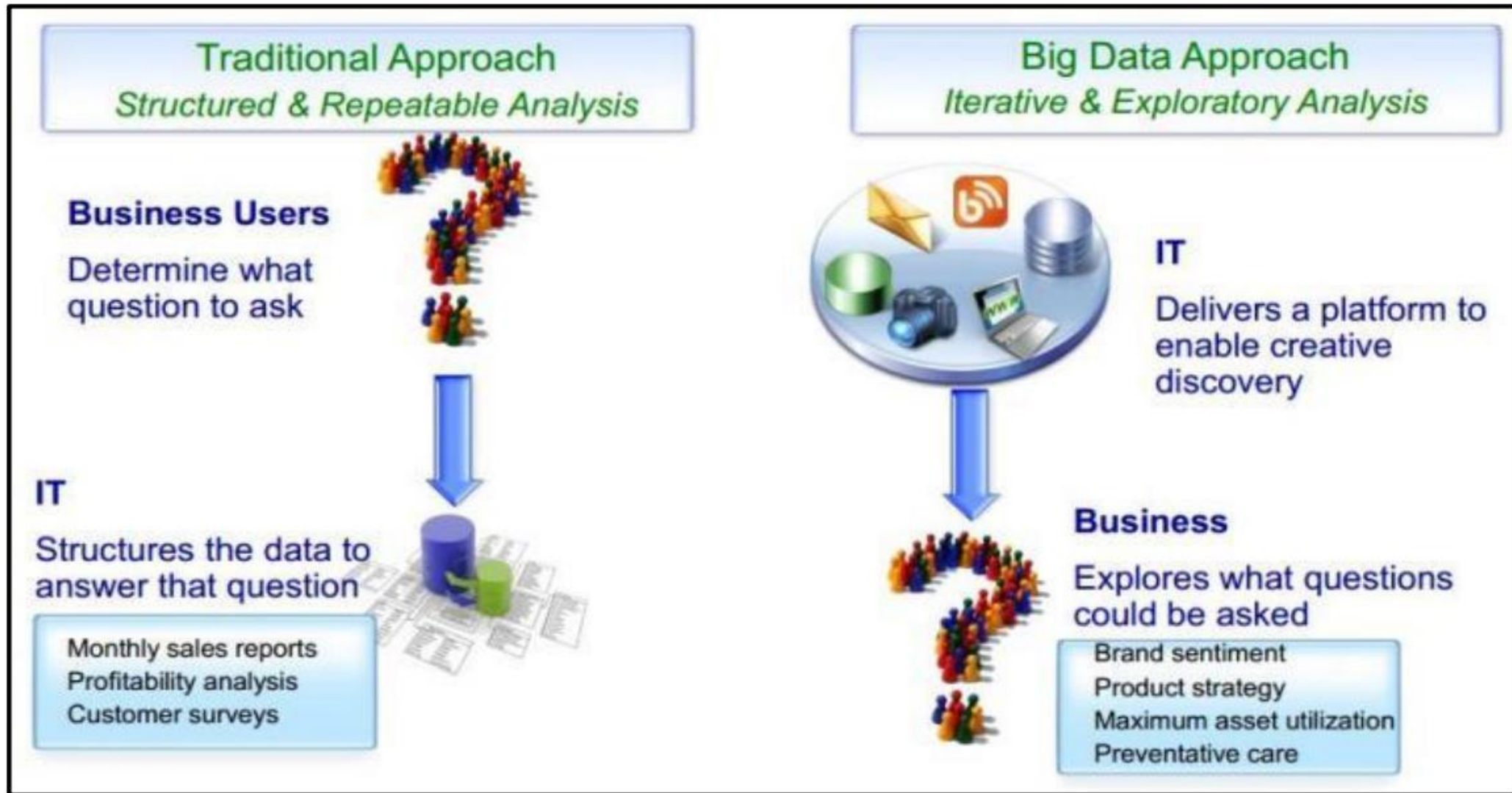


Figure: Architecture Big Data

Approche classique Vs Approche Big Data

viii. L'approche traditionnelle fonctionne sur les données structurées qui ont une présentation de base et la structure fournie.

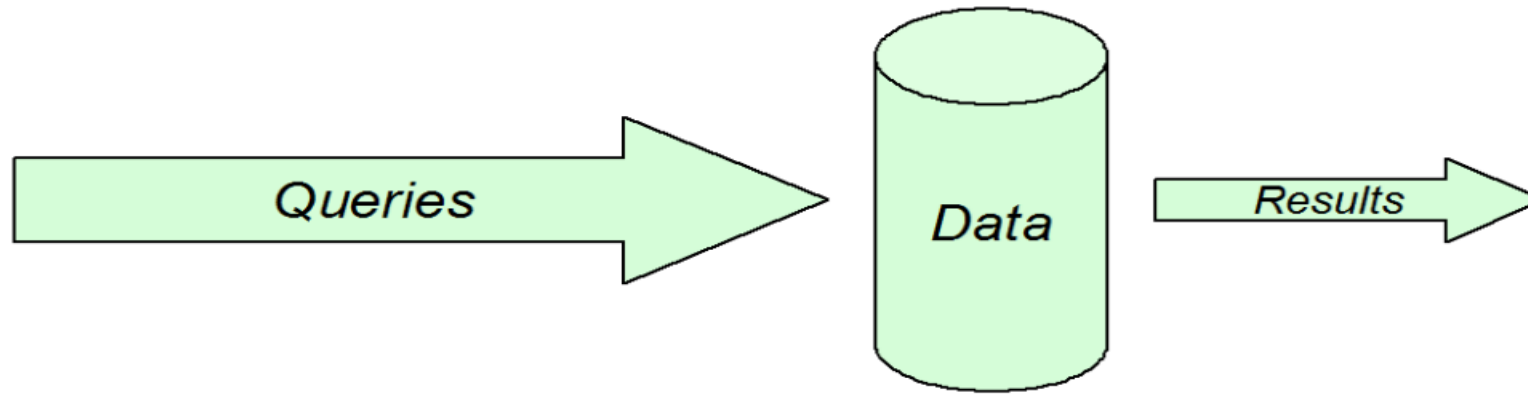


Figure: Données statiques

ix. L'approche structurée conçoit la base de données selon les exigences en tuples et en colonnes.

Approche classique Vs Approche Big Data

x. L'approche traditionnelle ne peut pas traiter les données qui arrivent en temps réel (Streaming Data).

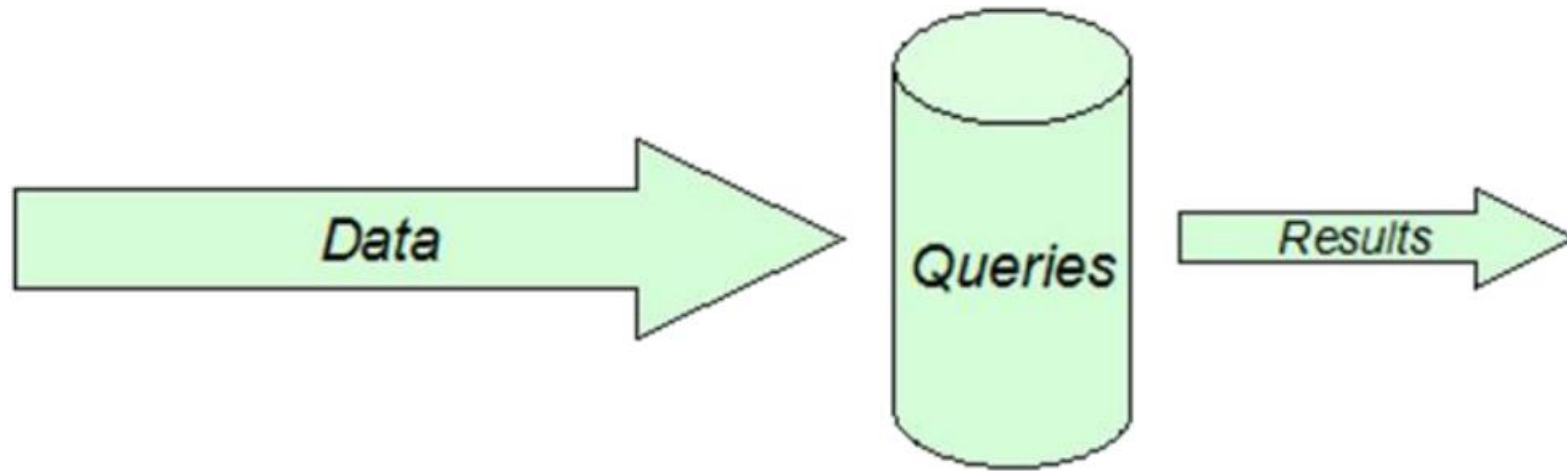


Figure: Flux de données en continu (Streaming Data)

xi. L'approche Big data est itérative.

Approche classique Vs Approche Big Data

- xii.** L'analyse du Big Data fonctionne sur les données non structurées, pour lesquelles aucun modèle spécifique de données n'est défini.
- xiii.** Les données ne sont pas organisées en lignes et colonnes.
- xiv.** Le flux de données en continu (Streaming Data) est capturé et l'analyse est effectuée dessus.
- xv.** L'efficacité augmente lorsque les données à analyser sont volumineuses.

Approche classique Vs Approche Big Data

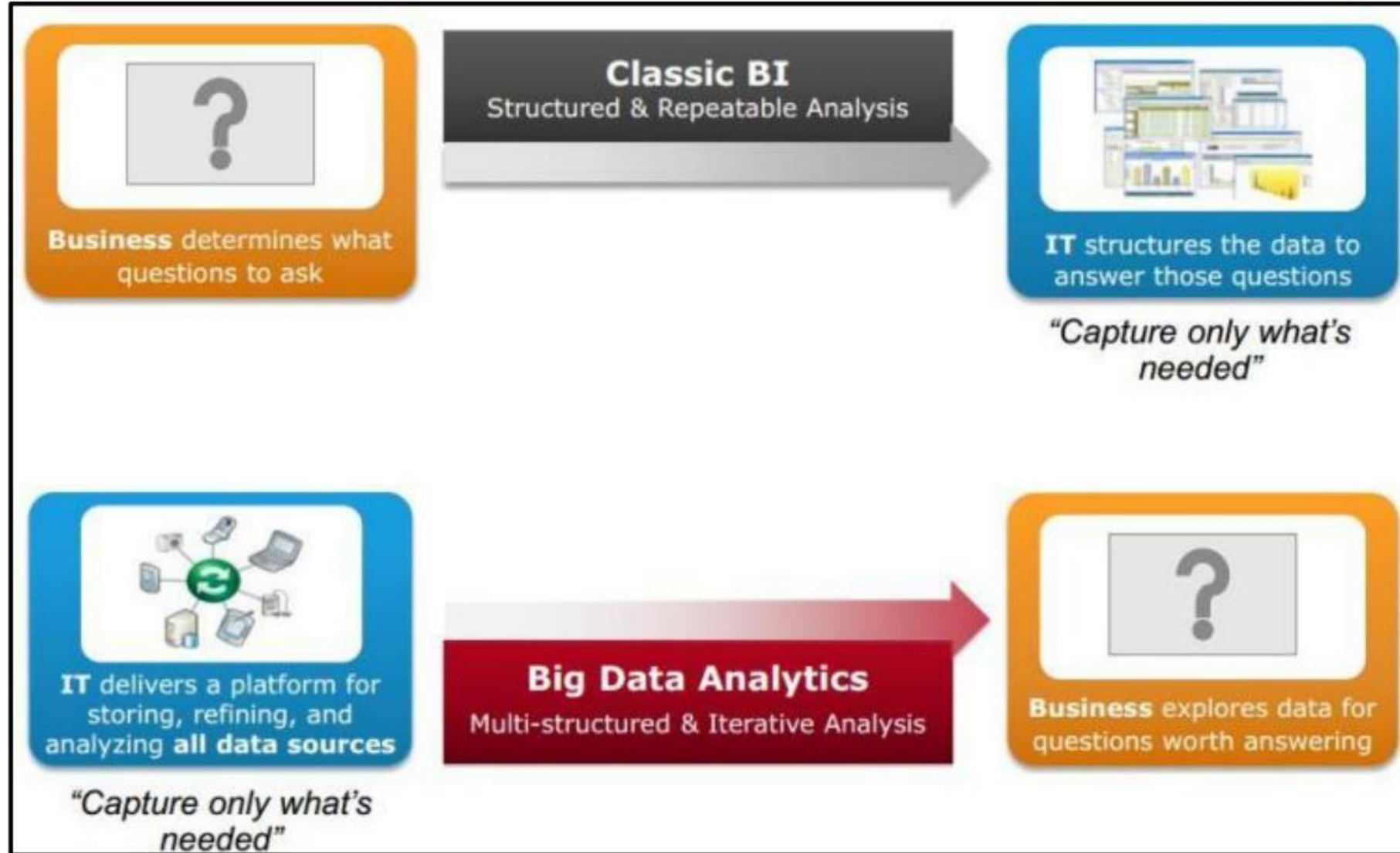


Figure: Architecture Big Data

Étude de cas de solutions Big Data

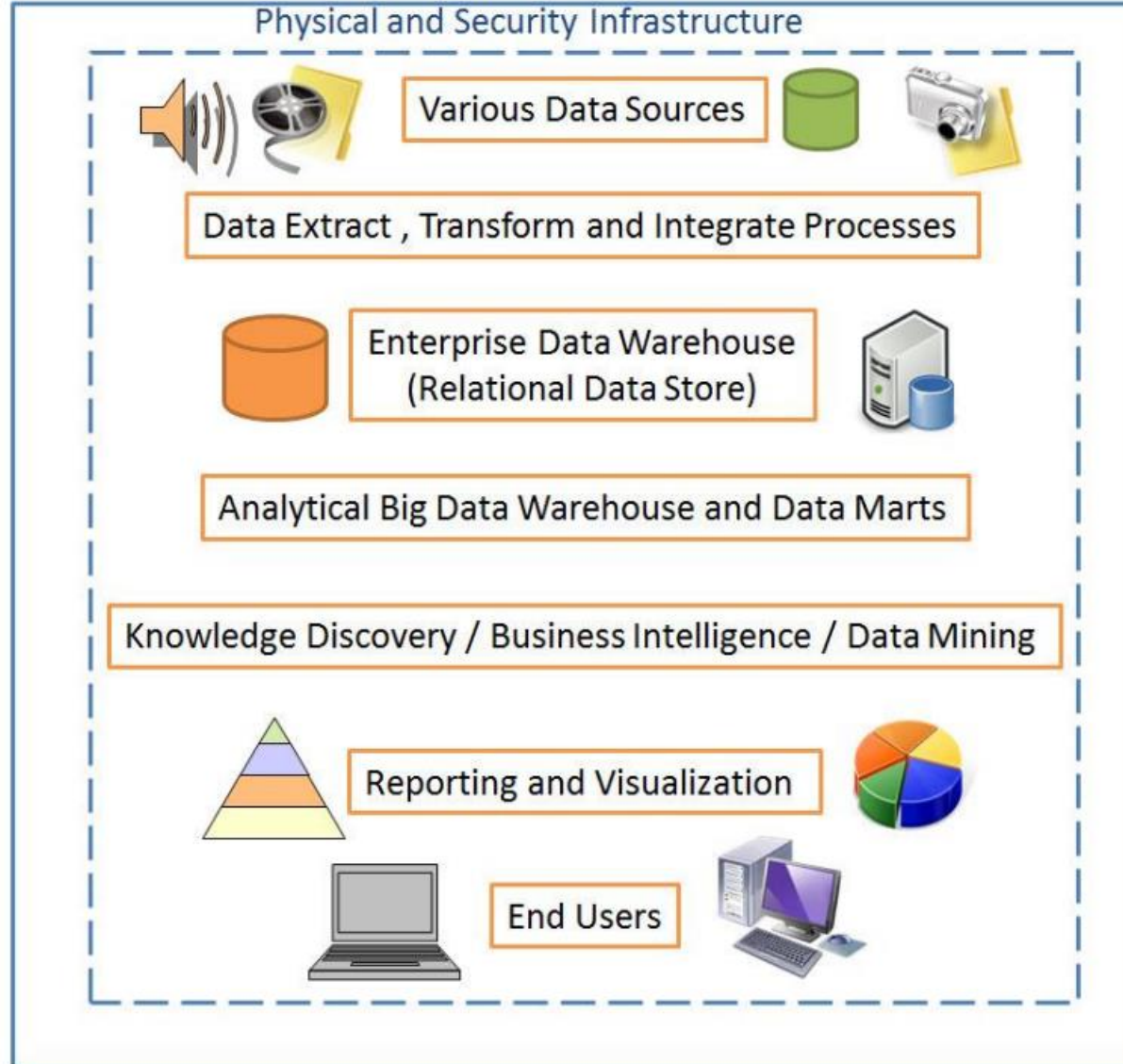


Figure : Infrastructure Big Data

Étude de cas de solutions Big Data

1. L'image ci-dessus donne un bon aperçu de la façon dont, dans l'infrastructure Big Data, divers composants sont associés les uns aux autres.
2. Dans le Big Data, différentes sources de données font partie de l'architecture. L'extraction, la transformation et l'intégration sont donc l'une des couches les plus essentielles de l'architecture.
3. La plupart des données sont stockées dans des data marts relationnels et non relationnels et des solutions d'entreposage de données.
4. Selon les besoins de l'entreprise, diverses données sont traitées et converties en rapports et visualisations appropriés pour les utilisateurs finaux.
5. Tout comme les logiciels, le matériel constitue presque la partie la plus importante de l'infrastructure Big Data.
6. Dans l'architecture Big Data, l'infrastructure matérielle est extrêmement importante et les pannes d'instances ainsi qu'une infrastructure physique redondante sont généralement mises en œuvre.

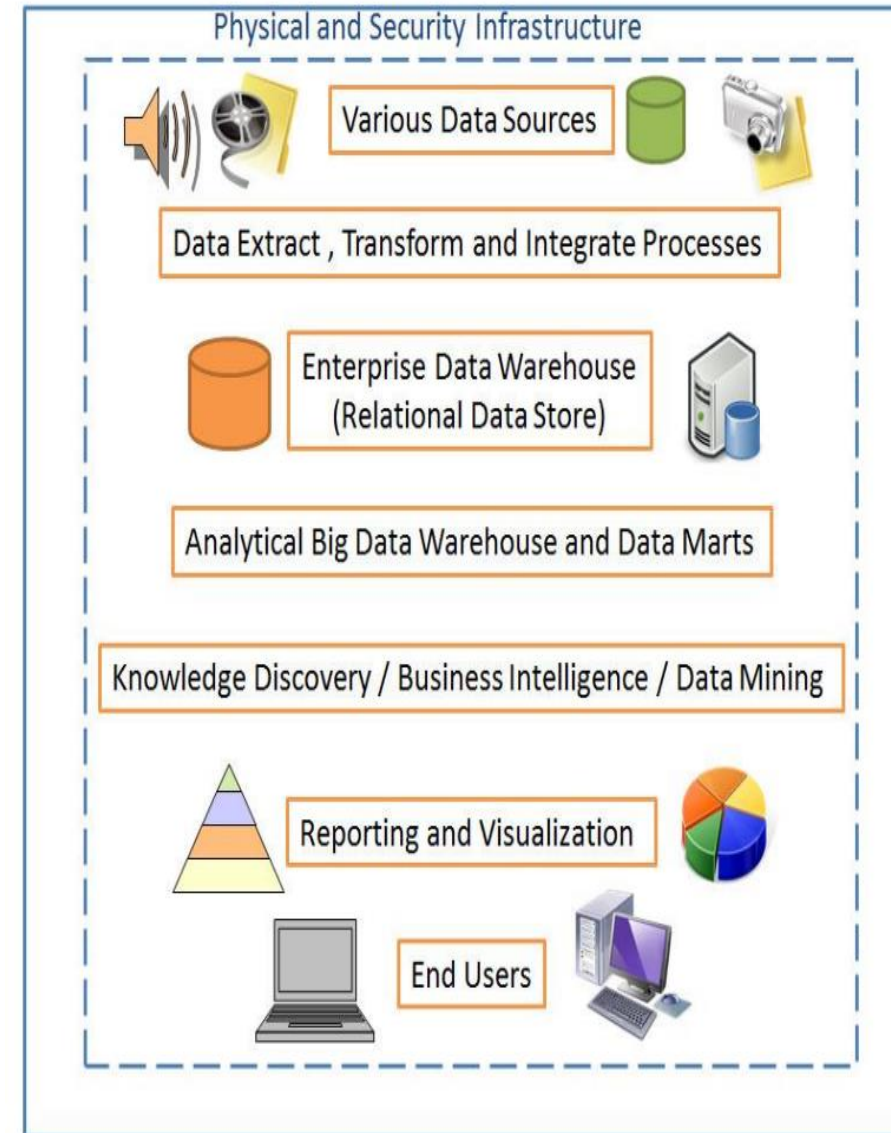


Figure : Infrastructure Big Data

